



Departamento de
Computação - **UFSCar**



Algoritmos e Estruturas de Dados I

Apostila sobre estruturas de dados elementares e análise de complexidade

Alexandre L. M. Levada
Departamento de Computação - UFSCar

6 de março de 2026

Prefácio

Vivemos em uma era em que a eficiência de soluções computacionais é cada vez mais determinante para o sucesso de aplicações em diversas áreas, desde ciência de dados até sistemas embarcados. Nesse contexto, dominar algoritmos e estruturas de dados não é apenas uma exigência acadêmica: é um diferencial crucial para qualquer profissional da computação. Entender como construir algoritmos corretos e eficientes e como organizar e manipular dados de maneira otimizada permite não só resolver problemas complexos, mas também compreender os fundamentos que sustentam as tecnologias modernas.

Esta apostila foi cuidadosamente elaborada para servir como material de apoio introdutório ao fascinante universo dos algoritmos e estruturas de dados elementares. Ao longo dos capítulos, o leitor encontrará conceitos teóricos essenciais, acompanhados de exemplos práticos e análises detalhadas de eficiência, sempre buscando um equilíbrio entre clareza didática e rigor técnico. A estrutura do conteúdo é organizada da seguinte forma:

- **Capítulo 1:** Complexidade de Algoritmos: apresenta as bases para analisar e comparar a eficiência de algoritmos, introduzindo conceitos fundamentais como notação Big-O, análise de pior, melhor e caso médio, além de exemplos clássicos de análise de loops e somatórios.

- **Capítulo 2:** Recursão: explora o paradigma da recursão, destacando sua expressividade e desafios, como a análise de complexidade recursiva. Traz exemplos clássicos como Fibonacci, fatorial, MDC, e problemas mais elaborados como a Torre de Hanói.

- **Capítulo 3:** Tipos Abstratos de Dados (TADs): introduz o conceito de abstração aplicada a estruturas de dados, discutindo como o encapsulamento e a definição de operações auxiliam no desenvolvimento de programas mais organizados e modulares.

- **Capítulo 4:** Estruturas de Dados Lineares Estáticas: detalha pilhas (LIFO), filas (FIFO), deque e filas de prioridade, abordando implementações em arrays e principais operações associadas.

- **Capítulo 5:** Estruturas de Dados Lineares Dinâmicas: apresenta listas encadeadas simples, duplamente encadeadas, com sentinelas e aplicações práticas como matrizes esparsas e listas ortogonais.

- **Capítulo 6:** Árvores Binárias: aborda a estrutura hierárquica de árvores binárias, focando em árvores binárias de busca e suas aplicações no armazenamento e recuperação eficiente de dados.

- **Capítulo 7:** Heaps Binários: explora a estrutura de heap, suas variações (min-heap e max-heap), uso em filas de prioridade

Espero que este material auxilie no seu desenvolvimento como profissional da computação e desperte o interesse em aprofundar o estudo de algoritmos e estruturas de dados, temas que seguirão sendo relevantes ao longo de toda sua carreira.

Bons estudos!

Sumário

1	Introdução à complexidade de algoritmos	5
1.1	Notação Big-O	6
1.2	Classes de Complexidade de Algoritmos	8
1.3	Somatórios	9
1.4	Analisando Algoritmos	11
1.5	Considerações Finais	18
2	Recursão	19
2.1	A sequência de Fibonacci	19
2.2	Problemas e exercícios	22
2.3	O Problema da Multiplicação de Inteiros	28
2.4	O Problema da Torre de Hanói	31
2.5	Busca Sequencial vs. Busca Binária	34
2.6	Considerações Finais	37
3	Tipos Abstratos de Dados (TAD's)	38
3.1	Tipos Abstratos de Dados e Programação Orientada a Objetos	39
3.2	Implementação de um TAD em C: TAD Ponto	40
3.3	Exercício: Implementação de um TAD Circulo	43
3.4	Exercício: Implementação de um TAD NumeroComplexo	46
3.5	Implementação de um TAD em Python por meio de Classes	49
3.6	Exercício: Classe Equacao_Quadratica em Python	51
3.7	Considerações Finais	52
4	Estruturas de dados lineares	54
4.1	Pilhas (LIFO)	54
4.1.1	Interface e Implementação Estática de uma Pilha	55
4.1.2	Aplicação de Pilhas: Verificação de Balanceamento de Parêntesis	57
4.1.3	Aplicação de Pilhas: Conversão de Decimal para Binário	59
4.1.4	O Problema da Celebridade	62
4.2	Filas (FIFO)	66
4.2.1	Aplicação de Filas: Escalonamento Round-Robin com Passagem de Token	70
4.3	Deque: Double-Ended Queue (Fila de Duas Extremidades)	76
4.3.1	Um TAD Deque	77
4.3.2	Aplicação do Deque: Verificação de Palíndromos	80
4.3.3	Problema: Sliding Window Maximum	82
4.4	Fila de Prioridades	86
4.4.1	Problema: Disk Tower	87

4.5	Considerações Finais	90
5	Estruturas de dados lineares dinâmicas	91
5.1	Listas encadeadas	91
5.1.1	Listas Encadeadas Simples	93
5.1.2	Pilhas com Estruturas Dinâmicas	99
5.1.3	Filas com Estruturas Dinâmicas	100
5.2	Listas Encadeadas Ordenadas	103
5.3	Listas Duplamente Encadeadas	105
5.3.1	Listas Duplamente Encadeadas com Sentinelas	108
5.3.2	Listas Duplamente Encadeadas Circulares	111
5.4	Matrizes Esparsas com Arrays e Listas Encadeadas	112
5.4.1	Método 1: Arrays Estáticos	113
5.4.2	Método 2: Listas Encadeadas	114
5.5	Listas Ortogonais e Matrizes Dinâmicas	116
5.6	Considerações Finais	118
6	Árvores Binárias	119
6.1	Limitações das Árvores Binárias Simples	123
6.2	Árvores Binárias de Busca (Binary Search Trees - BST)	124
6.2.1	Busca em Árvores Binárias de Busca	125
6.2.2	Sucessor e Predecessor	127
6.2.3	Inserção em Árvores Binárias de Busca	130
6.2.4	Remoção em Árvores Binárias de Busca	131
6.2.5	Remoção em Árvores Binárias de Busca	133
6.3	Árvores AVL	136
6.4	Operações em Árvores AVL	138
6.4.1	Rotação à Esquerda	138
6.4.2	Rotação à Direita	140
6.4.3	Rotação Esquerda-Direita	142
6.4.4	Rotação Direita-Esquerda	143
6.5	Inserção de nós em árvores AVL	144
6.6	Remoção em Árvores AVL	148
6.7	Complexidade das Operações em Árvores AVL	151
6.8	Considerações Finais	152
7	Heaps Binários e Filas de Prioridade	154
7.1	Árvores Binárias Completas	154
7.2	A Estrutura de Dados Heap	156
7.3	Filas de Prioridade com min-heaps	157

7.4	Análise da Complexidade	162
7.5	Considerações Finais	163

1 Introdução à complexidade de algoritmos

Na Ciência da Computação, mais especificamente no estudo de algoritmos, uma das primeiras perguntas que surgem é: como medir a eficiência de algoritmos? Ou seja, dados dois algoritmos A_1 e A_2 , como determinar qual deles é mais eficiente?

Em geral, estamos interessados na análise de duas grandezas fundamentais:

- (a) Tempo de execução (*complexidade de tempo*);
- (b) Quantidade de memória utilizada (*complexidade de espaço*).

A quantidade de memória exigida por um algoritmo depende basicamente das variáveis alocadas durante sua execução, o que é relativamente simples de estimar. Formalmente, podemos representar a complexidade de espaço como uma função $S(n)$, onde n representa o tamanho da entrada do problema. Por outro lado, o tempo de execução requer uma análise mais formal e aprofundada. A complexidade de tempo pode ser representada por uma função $T(n)$, que indica o número de operações elementares executadas pelo algoritmo em função do tamanho da entrada n .

Uma dúvida comum é: por que não podemos simplesmente cronometrar o tempo de execução de uma implementação do algoritmo? A resposta é que o tempo medido experimentalmente depende de diversos fatores externos ao algoritmo, tais como:

- A linguagem de programação utilizada (por exemplo, C, Python ou Java);
- O hardware da máquina (processador, memória, arquitetura, etc.);
- O compilador ou interpretador empregado.

Assim, o tempo observado t_{real} poderia ser modelado de forma simplificada como

$$t_{\text{real}} = c \cdot T(n),$$

onde c é uma constante que depende do ambiente de execução. Como essa constante varia conforme a linguagem e o hardware, ela não permite uma comparação justa e universal entre algoritmos.

Portanto, desejamos realizar uma análise independente de fatores externos, isto é, uma análise assintótica baseada apenas no número de operações realizadas pelo algoritmo. O objetivo central da análise de tempo consiste em contar o número de operações elementares executadas por um algoritmo.

Para realizar a análise formal da complexidade de tempo, assumimos as seguintes hipóteses:

1. Cada comando de atribuição executa em tempo constante, isto é, possui complexidade $O(1)$.
2. A função $T(n)$ retorna o número de atribuições executadas quando a entrada do problema possui tamanho n .

Considere o seguinte algoritmo que calcula a soma dos n primeiros números inteiros:

```
// Soma os n primeiros inteiros
sum_of_n(n) {
    soma = 0
    for i = 1 to n
        soma = soma + i
    return soma
}
```

Observe que:

- A instrução de inicialização $\text{soma} = 0$ é executada uma única vez;
- A instrução dentro do laço **for** é executada n vezes.

Como estamos contando apenas operações de atribuição, temos:

$$T(n) = 1 + n$$

ou, equivalentemente,

$$T(n) = n + 1.$$

Quando n cresce muito, a constante aditiva torna-se irrelevante diante do termo dominante. Assim, do ponto de vista assintótico,

$$T(n) \sim n \quad \text{quando } n \rightarrow \infty.$$

Portanto, a complexidade de tempo do algoritmo é linear no tamanho da entrada, isto é,

$$T(n) \in O(n).$$

1.1 Notação Big-O

Em vez de nos preocuparmos em contar exatamente o número de instruções de um programa, é matematicamente mais conveniente analisar a *ordem de magnitude* da função $T(n)$, ou seja, investigar o comportamento de $T(n)$ quando n cresce arbitrariamente.

Suponha que exista uma função $f(n)$, definida para todos os inteiros $n \geq 0$, tal que existam constantes positivas c e m com

$$T(n) \leq c f(n), \quad \forall n \geq m.$$

Nesse caso, dizemos que $T(n)$ é $O(f(n))$, ou ainda,

$$T(n) = O(f(n)).$$

Essa notação fornece um limite superior assintótico para a função $T(n)$.

Considere o seguinte algoritmo que soma os elementos de uma matriz quadrada $M \in \mathbb{R}^{n \times n}$:

```
sum_of_matrix(M, n) {
    soma = 0
    for i = 1 to n {
        soma_linha = 0
        for j = 1 to n
            soma_linha += M[i, j]
        soma += soma_linha
    }
    return soma
}
```

Vamos calcular o número de atribuições executadas por esse algoritmo. Observe que:

- No laço mais interno (índice j), temos n iterações;
- Para cada valor de i , o bloco interno executa n atribuições referentes à instrução `soma_linha += M[i, j]`;
- Além disso, para cada i , temos duas atribuições adicionais: `soma_linha = 0` e `soma += soma_linha`.

Logo, para cada valor de i , temos $n + 2$ atribuições. Como o laço externo executa n vezes, obtemos:

$$T(n) = n(n + 2) + 1.$$

Expandindo,

$$T(n) = n^2 + 2n + 1.$$

Agora, tomando $f(n) = n^2$ e $c = 2$, temos:

$$n^2 + 2n + 1 \leq 2n^2,$$

para todo $n > 2$. Portanto,

$$T(n) = O(n^2).$$

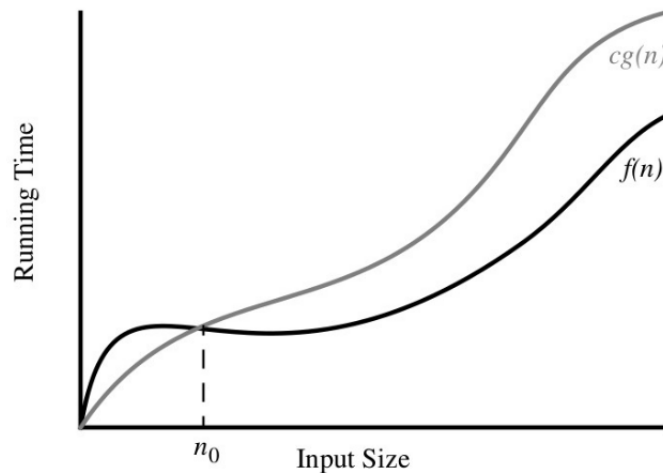


Figura 1: A função $g(n)$ multiplicada por uma constante c fornece um limite superior para a função $f(n)$ para todo n maior que um limite n_0 .

Poderíamos escolher, por exemplo, $f(n) = n^3$, pois também seria um limite superior válido. Entretanto, o objetivo da notação Big-O é fornecer o limite superior *mais justo possível*, isto é, aquele que melhor caracteriza o crescimento assintótico da função.

1.2 Classes de Complexidade de Algoritmos

Os algoritmos costumam ser classificados de acordo com a ordem de crescimento assintótico da função $T(n)$ que descreve sua complexidade de tempo. Essa classificação permite comparar algoritmos independentemente de constantes multiplicativas e termos de menor ordem. A seguir, apresentamos as principais classes de complexidade, organizadas conforme a função $f(n)$ associada ao seu crescimento assintótico.

Tabela 1: Principais classes de complexidade assintótica

Função $f(n)$	Classe de Complexidade
1	Constante
$\log n$	Logarítmica
n	Linear
$n \log n$	Log-linear
n^2	Quadrática
n^3	Cúbica
n^k	Polinomial
2^n	Exponencial
$n!$	Fatorial

Observações

- Algoritmos de complexidade constante $O(1)$ possuem tempo de execução independente do tamanho da entrada.

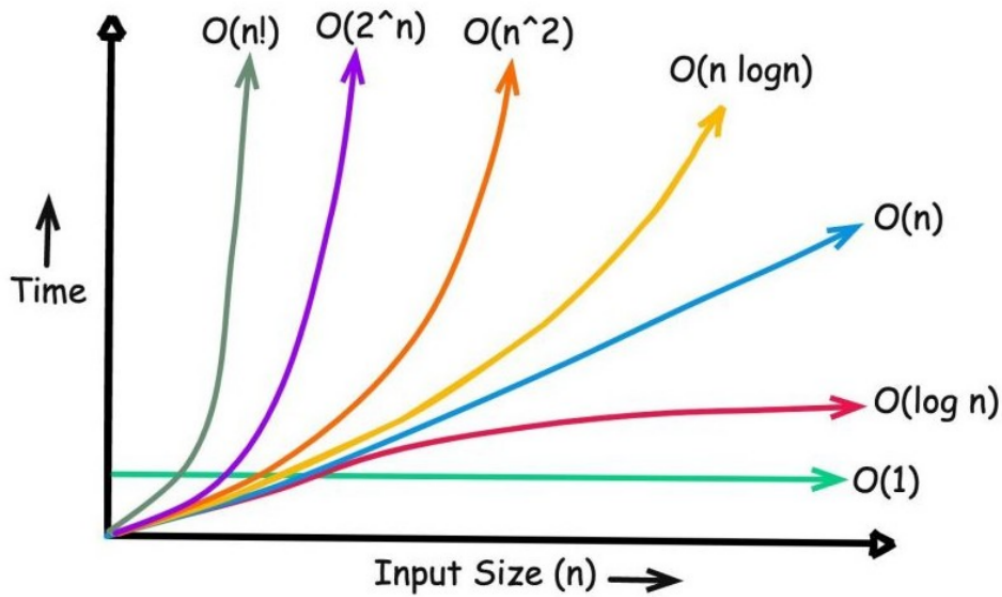


Figura 2: O gráfico mostra a taxa de crescimento de diversos tipos de funções usadas na complexidade de algoritmos.

- Algoritmos logarítmicos $O(\log n)$ são típicos de estratégias que dividem o problema sucessivamente pela metade, como na busca binária.
- Algoritmos lineares $O(n)$ percorrem a entrada uma única vez.
- Algoritmos $O(n \log n)$ aparecem frequentemente em métodos eficientes de ordenação baseados em divisão e conquista.
- Complexidades polinomiais $O(n^k)$ são consideradas tratáveis do ponto de vista computacional.
- Complexidades exponenciais $O(2^n)$ e fatoriais $O(n!)$ crescem muito rapidamente, tornando-se inviáveis mesmo para valores moderados de n .

1.3 Somatórios

No cálculo da complexidade de algoritmos é comum analisar estruturas de repetição (*loops*). Para isso, é fundamental saber manipular e resolver somatórios. Considere o seguinte exemplo:

$$2^1 + 2^2 + 2^3 + 2^4 + \dots + 2^{10}.$$

Podemos expressá-lo na forma compacta como:

$$\sum_{k=1}^{10} 2^k.$$

A seguir, apresentamos propriedades importantes para a manipulação de somatórios.

1) Substituição de Variáveis

Considere o somatório:

$$\sum_{k=1}^n 2^k.$$

Definindo a mudança de variável $i = k - 1$, temos:

$$k = i + 1.$$

Ajustando os limites:

- Limite inferior (LI): se $k = 1$, então $i = 0$;
- Limite superior (LS): se $k = n$, então $i = n - 1$.

Logo, o somatório pode ser reescrito como:

$$\sum_{i=0}^{n-1} 2^{i+1}.$$

2) Propriedade Distributiva

Para toda constante c , vale:

$$\sum_{k \in A} c f(k) = c \sum_{k \in A} f(k).$$

Ou seja, constantes podem ser colocadas em evidência, fora do somatório.

3) Propriedade Associativa

A soma de funções pode ser separada em soma de somatórios:

$$\sum_{k \in A} (f(k) + g(k)) = \sum_{k \in A} f(k) + \sum_{k \in A} g(k).$$

4) Somas Telescópicas

Considere uma sequência de números reais $x_1, x_2, x_3, \dots, x_n, x_{n+1}$. Então, vale a identidade:

$$\sum_{k=1}^n (x_{k+1} - x_k) = x_{n+1} - x_1.$$

Ou seja, o somatório das diferenças sucessivas é igual à diferença entre o último e o primeiro elemento.

1. Pela propriedade associativa, temos:

$$S = \sum_{k=1}^n (x_{k+1} - x_k) = \sum_{k=1}^n x_{k+1} - \sum_{k=1}^n x_k.$$

2. Aplicando substituição de variáveis no primeiro somatório:

$$i = k + 1 \quad \Rightarrow \quad k = i - 1.$$

Ajustando os limites:

- Se $k = 1$, então $i = 2$;
- Se $k = n$, então $i = n + 1$.

Logo,

$$S = \sum_{i=2}^{n+1} x_i - \sum_{k=1}^n x_k.$$

3. Separando o último termo do primeiro somatório e o primeiro termo do segundo:

$$S = \left(\sum_{i=2}^n x_i + x_{n+1} \right) - \left(x_1 + \sum_{k=2}^n x_k \right).$$

Cancelando os termos comuns:

$$S = x_{n+1} - x_1.$$

A prova está concluída.

1.4 Analisando Algoritmos

Considere inicialmente um exemplo com duas estruturas de repetição em série.

```
Func_A(n) {  
    c = 0  
    for i = 1 to n  
        c += 1  
    for j = 1 to n  
        c += 2  
    return c  
}
```

Temos uma atribuição inicial (1 operação) e dois loops com n iterações cada. Cada laço contribui com n atribuições. Logo,

$$T(n) = 2n + 1,$$

o que implica

$$T(n) = O(n).$$

Agora considere um algoritmo com duas estruturas de repetição aninhadas.

```
Func_B(n) {
  c = 0
  for i = 1 to n {
    for j = 1 to n
      c = c + 1
  }
  return c
}
```

O loop interno executa n operações. Como o loop externo executa n vezes, temos $n \cdot n = n^2$ incrementos, além da inicialização. Assim,

$$T(n) = n^2 + 1,$$

portanto,

$$T(n) = O(n^2).$$

Se no loop interno o limite fosse 10 em vez de n , teríamos

$$T(n) = 10n + 1,$$

o que implica

$$T(n) = O(n).$$

Considere agora um exemplo em que o loop interno depende da variável do loop externo.

```
Func_C(n) {
  c = 0
  for i = 1 to n {
    for j = 1 to i
      c = c + 1
  }
  return c
}
```

Observe que:

- Quando $i = 1$, o loop interno executa uma vez;
- Quando $i = 2$, executa duas vezes;
- ...
- Quando $i = n$, executa n vezes.

Logo, o número total de incrementos é:

$$1 + 2 + 3 + \cdots + n = \sum_{k=1}^n k.$$

Para resolver esse somatório, note que:

$$(k+1)^2 = k^2 + 2k + 1,$$

o que implica

$$(k+1)^2 - k^2 = 2k + 1.$$

Somando ambos os lados de $k = 1$ até n :

$$\sum_{k=1}^n [(k+1)^2 - k^2] = \sum_{k=1}^n (2k + 1).$$

O lado esquerdo é uma soma telescópica:

$$\sum_{k=1}^n [(k+1)^2 - k^2] = (n+1)^2 - 1.$$

Logo,

$$(n+1)^2 - 1 = \sum_{k=1}^n (2k + 1).$$

Aplicando a propriedade associativa:

$$(n+1)^2 - 1 = 2 \sum_{k=1}^n k + \sum_{k=1}^n 1.$$

Como

$$\sum_{k=1}^n 1 = n,$$

temos:

$$(n+1)^2 - 1 = 2 \sum_{k=1}^n k + n.$$

Expandindo:

$$n^2 + 2n + 1 - 1 = 2 \sum_{k=1}^n k + n,$$

$$n^2 + 2n = 2 \sum_{k=1}^n k + n.$$

Isolando o somatório:

$$2 \sum_{k=1}^n k = n^2 + n,$$

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}.$$

Portanto,

$$T(n) = \frac{n(n+1)}{2} + 1 = \frac{1}{2}(n^2 + n) + 1.$$

Assim, o termo dominante é n^2 , e concluímos que

$$T(n) = O(n^2).$$

O próximo exemplo apresenta uma função em que a variável controladora é dividida por 2 a cada iteração.

```
Func_D(n) {
    c = 0
    i = n
    while i > 1 {
        c = c + 1
        i = floor(i/2)    // divisao inteira
    }
    return c
}
```

Essa função calcula quantas vezes um número pode ser dividido por 2 até se tornar menor ou igual a 1. Por exemplo, para $n = 16$:

$$16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$$

A variável c termina valendo 4, pois

$$2^4 = 16.$$

Para $n = 25$:

$$25 \rightarrow 12 \rightarrow 6 \rightarrow 3 \rightarrow 1$$

Temos $c = 4$, pois

$$2^4 < 25 < 2^5.$$

Para $n = 40$:

$$40 \rightarrow 20 \rightarrow 10 \rightarrow 5 \rightarrow 2 \rightarrow 1$$

Temos $c = 5$, pois

$$2^5 < 40 < 2^6.$$

De forma geral, o número de iterações do laço é:

$$\lfloor \log_2 n \rfloor.$$

Como dentro do `while` existem duas instruções, o número total de operações é:

$$T(n) = 2 + 2\lfloor \log_2 n \rfloor,$$

onde $\lfloor x \rfloor$ representa a função piso, isto é, o maior inteiro menor ou igual a x .

Logo,

$$T(n) = O(\log_2 n).$$

Considere agora o seguinte algoritmo:

```
Func_E(n) {  
    c = 0  
    for i = 1 to n  
        c = c + Func_D(n)  
    return c  
}
```

Como a função `Func_D(n)` possui complexidade $O(\log_2 n)$ e o laço externo executa n vezes, o número total de operações é da ordem de:

$$T(n) = n \log_2 n.$$

Portanto, a complexidade assintótica é:

$$T(n) = O(n \log_2 n).$$

A seguir apresentamos duas funções para verificar se um número inteiro n é primo. Vamos analisar a complexidade de cada uma e justificar qual delas é mais eficiente.

```
prime_A(n) {
    if n == 1
        return False
    for i = 2 to n-1 {
        resto = n % i
        if resto == 0
            return False
    }
    return True
}
```

Primeiramente, analisemos o algoritmo `prime_A`. Se $n \in \{0, 1, 2, 3\}$, o algoritmo executa apenas algumas instruções condicionais, resultando em complexidade:

$$T(n) = O(1),$$

que corresponde ao melhor caso. Caso contrário, o laço executa de $i = 2$ até $i = n - 1$. Logo, o número de iterações é:

$$(n - 1) - 2 + 1 = n - 2.$$

Em cada iteração é realizada uma operação de resto da divisão. Assim, no pior caso:

$$T(n) = n - 2,$$

o que implica:

$$T(n) = O(n).$$

Agora analisemos o algoritmo `prime_B`. Novamente, se $n \in \{0, 1, 2, 3\}$, temos:

$$T(n) = O(1).$$

Caso contrário, o laço `while` executa enquanto $i \leq \sqrt{n}$.

```

prime_B(n) {
    if n == 1
        return False
    if n == 2
        return True
    if n % 2 == 0
        return False
    i = 3
    while i <= sqrt(n) {
        resto = n % i
        if resto == 0
            return False
        else
            i += 2
    }
    return True
}

```

Como i inicia em 3 e é incrementado de 2 em 2, o número de iterações é aproximadamente:

$$\frac{\sqrt{n} - 3}{2}.$$

Em cada iteração são realizadas duas operações principais: o cálculo do resto e o incremento de i . Assim, o número total de operações é proporcional a:

$$T(n) \approx 2 \cdot \frac{\sqrt{n} - 3}{2}.$$

Assintoticamente, temos:

$$T(n) = O(\sqrt{n}) = O(n^{1/2}).$$

Comparando as duas complexidades:

$$O(n) \quad \text{vs} \quad O(n^{1/2}),$$

observa-se que

$$\lim_{n \rightarrow \infty} \frac{\sqrt{n}}{n} = 0.$$

Logo, $n^{1/2}$ cresce muito mais lentamente que n . Portanto, o algoritmo `prime_B` é assintoticamente mais eficiente que `prime_A`. Em termos práticos, isso significa que, para valores suficientemente grandes de n , nenhum computador fará com que `prime_A` seja mais rápido que `prime_B`.

(embora, para valores pequenos de n , diferenças de constante possam eventualmente inverter essa relação).

1.5 Considerações Finais

Neste capítulo apresentamos os fundamentos da análise de complexidade de algoritmos, com foco na mensuração formal do tempo de execução e, secundariamente, do espaço utilizado. Mostramos que a simples medição empírica do tempo não é suficiente para comparar algoritmos de maneira universal, pois depende de fatores externos como linguagem de programação, compilador e hardware. Por esse motivo, introduzimos uma abordagem assintótica baseada na contagem de operações elementares.

A notação $O(\cdot)$ foi formalmente definida como um limite superior assintótico, permitindo classificar algoritmos de acordo com sua ordem de crescimento. Vimos que, quando $n \rightarrow \infty$, termos de menor ordem e constantes multiplicativas tornam-se irrelevantes, sendo o termo dominante aquele que determina o comportamento assintótico da função $T(n)$.

Foram analisados diversos exemplos envolvendo estruturas sequenciais, laços aninhados e laços com crescimento não linear, evidenciando como diferentes padrões de repetição levam a complexidades distintas, tais como:

$$O(1), \quad O(\log n), \quad O(n), \quad O(n \log n), \quad O(n^2), \quad O(\sqrt{n}).$$

Também exploramos a importância da manipulação de somatórios na análise de algoritmos, especialmente em estruturas aninhadas e dependentes, onde a resolução de expressões como

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}$$

é essencial para determinar o comportamento assintótico.

A comparação entre diferentes estratégias algorítmicas, como no exemplo dos testes de primalidade, ilustrou um princípio central da Ciência da Computação: algoritmos assintoticamente mais eficientes tornam-se inevitavelmente superiores para entradas suficientemente grandes, independentemente de constantes de implementação.

Por fim, a análise de complexidade não deve ser vista apenas como um exercício matemático, mas como uma ferramenta conceitual fundamental para o projeto de algoritmos eficientes. Compreender a taxa de crescimento de $T(n)$ permite prever escalabilidade, estimar viabilidade prática e tomar decisões fundamentadas no desenvolvimento de soluções computacionais.

No próximo capítulo, aprofundaremos a discussão abordando técnicas de projeto de algoritmos e estratégias clássicas como divisão e conquista, programação dinâmica e algoritmos gulosos, analisando suas respectivas implicações em termos de complexidade.

2 Recursão

Recursão é uma técnica fundamental de projeto de algoritmos na qual uma função é definida em termos de si mesma. Em vez de resolver diretamente um problema de tamanho n , a estratégia recursiva consiste em reduzi-lo a uma ou mais instâncias menores do mesmo problema, até que se atinja um caso base cuja solução seja conhecida explicitamente. Formalmente, uma definição recursiva envolve duas componentes essenciais: (i) um ou mais *casos base*, que encerram o processo recursivo, e (ii) uma *regra de recorrência*, que expressa a solução para tamanho n em função de soluções para tamanhos menores, por exemplo,

$$T(n) = T(n - 1) + c,$$

ou, de maneira mais geral,

$$T(n) = a T\left(\frac{n}{b}\right) + f(n),$$

onde $a \geq 1$, $b > 1$ e $f(n)$ representa o custo adicional do processamento. A recursão está intimamente ligada a estruturas matemáticas como indução e recorrências, além de ser a base de importantes paradigmas algorítmicos, como divisão e conquista. Entender seu funcionamento é essencial tanto para a implementação correta quanto para a análise formal de sua complexidade.

Dizemos que uma função é recursiva quando é definida em termos dela mesma. Em matemática e computação, uma classe de objetos ou métodos exhibe comportamento recursivo quando pode ser caracterizada por duas propriedades fundamentais:

1. **Caso base:** condição de término da recursão, na qual o processo produz diretamente uma resposta.
2. **Passo recursivo:** conjunto de regras que reduz todos os demais casos a instâncias menores do problema, aproximando-os do caso base.

2.1 A sequência de Fibonacci

Um exemplo clássico é a série de Fibonacci, definida por:

$$F(0) = 0 \quad (\text{caso base})$$

$$F(1) = 1 \quad (\text{caso base})$$

$$F(2) = 1 \quad (\text{caso base})$$

e, para todo $n > 1$,

$$F(n) = F(n - 1) + F(n - 2).$$

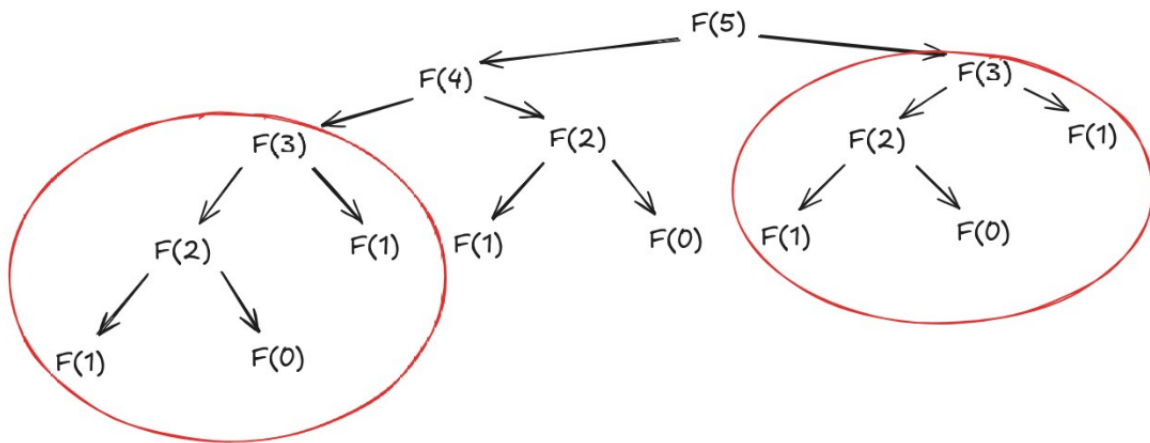


Figura 3: A árvore de recursão para o cálculo de $F(5)$.

A seguir, apresentamos um algoritmo recursivo para calcular o n -ésimo número de Fibonacci.

```
Fib(n) {
  if n == 0
    return 0
  else {
    if n == 1 or n == 2
      return 1
    else
      return Fib(n - 1) + Fib(n - 2)
  }
}
```

Embora a definição seja elegante do ponto de vista matemático, essa implementação é computacionalmente muito ineficiente. Suponha que desejamos calcular $F(7)$. Durante o cálculo de $F(n)$, os valores $F(n-2)$, $F(n-3)$, etc., são recalculados diversas vezes. Por exemplo, $F(5)$ é computado tanto no cálculo de $F(6)$ quanto no de $F(7)$.

Podemos modelar o tempo de execução da versão recursiva ingênua do Fibonacci pela seguinte recorrência:

$$T(n) = T(n-1) + T(n-2) + O(1).$$

Para n suficientemente grande, temos aproximadamente

$$T(n-1) \approx T(n-2),$$

o que permite obter a seguinte aproximação:

$$T(n) \approx 2T(n-1) + O(1).$$

Aplicando substituições sucessivas:

$$T(n) = 2(2T(n-2) + O(1)) + O(1) = 2^2T(n-2) + 2O(1) + O(1),$$

$$T(n) = 2(2(2T(n-3) + O(1)) + O(1)) + O(1) = 2^3T(n-3) + O(1) \sum_{i=0}^2 2^i.$$

Prosseguindo com essas substituições até o k -ésimo termo, obtemos:

$$T(n) = 2^kT(n-k) + O(1) \sum_{i=0}^{k-1} 2^i.$$

Sabemos que

$$\sum_{i=0}^{k-1} 2^i = 2^k - 1.$$

Logo,

$$T(n) = 2^kT(n-k) + (2^k - 1)O(1).$$

Fatorando:

$$T(n) = 2^kT(n-k) + 2^kO(1) = 2^k(T(n-k) + O(1)).$$

A condição de parada ocorre quando atingimos $T(0)$, ou seja, quando $k = n$. Portanto,

$$T(n) = 2^n(T(0) + O(1)).$$

Como $T(0) = 0$, temos:

$$T(n) = 2^nO(1),$$

ou simplesmente,

$$T(n) = O(2^n).$$

Concluimos que a complexidade é exponencial, o que é computacionalmente proibitivo para valores grandes de n . Para ilustrar, considere $n = 100$. O número de operações necessárias seria aproximadamente:

$$2^{100} \approx 1.2676506 \times 10^{30}.$$

Se cada instrução fosse executada em 1 nanosegundo (10^{-9} segundos), o tempo total seria:

$$1.2676506 \times 10^{30} \times 10^{-9} = 1.2676506 \times 10^{21} \text{ segundos.}$$

Convertendo para anos:

$$\frac{1.2676506 \times 10^{21}}{60 \times 60 \times 24 \times 365} \approx 4.01 \times 10^{13} \text{ anos.}$$

Para comparação, a idade estimada do universo é aproximadamente

$$26.7 \times 10^9 \text{ anos.}$$

Isso evidencia que a abordagem recursiva ingênua para o Fibonacci torna-se completamente inviável para valores moderadamente grandes de n .

2.2 Problemas e exercícios

Problema: Desenvolva um algoritmo recursivo para calcular o fatorial de um inteiro positivo n e determine sua complexidade. Recorde que o fatorial é definido matematicamente por:

$$n! = \begin{cases} 1, & \text{se } n = 0 \text{ ou } n = 1, \\ n \cdot (n-1)!, & \text{se } n > 1. \end{cases}$$

Uma implementação recursiva é dada por:

```
fatorial(n) {  
    if n == 1  
        return 1  
    else  
        return n * fatorial(n - 1)  
}
```

A cada chamada recursiva, o valor de n é reduzido de uma unidade. Assim, temos a seguinte recorrência para o tempo de execução:

$$T(n) = T(n-1) + O(1),$$

onde $O(1)$ representa o custo constante da multiplicação e do teste condicional. Expandindo a recorrência:

$$T(n) = T(n-1) + O(1)$$

$$T(n) = T(n-2) + 2O(1)$$

$\vdots$

$$T(n) = T(1) + (n - 1)O(1).$$

Como $T(1) = O(1)$, obtemos:

$$T(n) = nO(1).$$

Logo, a complexidade assintótica do algoritmo é:

$$T(n) = O(n).$$

Como cada chamada recursiva permanece na pilha de execução até que o caso base seja atingido, a profundidade da recursão é n . Portanto, o consumo de memória também é linear:

$$S(n) = O(n).$$

Concluimos que o algoritmo recursivo para cálculo do fatorial possui complexidade linear tanto em tempo quanto em espaço.

Problema: Desenvolva um algoritmo recursivo para calcular o somatório

$$S = \sum_{i=1}^n i$$

Recorde que esse somatório pode ser definido recursivamente como:

$$S(n) = \begin{cases} 1, & \text{se } n = 1, \\ n + S(n - 1), & \text{se } n > 1. \end{cases}$$

Uma possível implementação recursiva é dada por:

```
somatorio(n) {
    if n == 1
        return 1
    else
        return n + somatorio(n - 1)
}
```

A cada chamada recursiva, o valor de n é reduzido em uma unidade. Assim, o tempo de execução pode ser descrito pela recorrência:

$$T(n) = T(n - 1) + O(1).$$

Expandindo a recorrência:

$$T(n) = T(n - 2) + 2O(1)$$

$\vdots$

$$T(n) = T(1) + (n - 1)O(1).$$

Como $T(1) = O(1)$, concluímos que:

$$T(n) = O(n).$$

A profundidade da recursão é n , pois cada chamada permanece na pilha de execução até que o caso base seja atingido. Logo, o consumo de memória também é linear:

$$S(n) = O(n).$$

Portanto, o algoritmo recursivo para cálculo do somatório possui complexidade linear em tempo e espaço.

Problema: Desenvolva um algoritmo recursivo para calcular o valor de e^x utilizando a expansão em série de Taylor, dada por:

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}.$$

Considere a seguinte implementação recursiva:

```
exponencial(x, n) {  
    if n == 0  
        return 1  
    else  
        return (x**n)/fatorial(n) + exponencial(x, n-1)  
}
```

Observe que o algoritmo calcula a soma parcial da série até a ordem n . A função realiza uma chamada recursiva para cada valor de n até atingir o caso base. Assim, temos inicialmente n chamadas recursivas. Entretanto, em cada nível da recursão é calculado:

$$\frac{x^n}{fatorial(n)}.$$

Se assumirmos que o cálculo de $fatorial(n)$ custa $O(n)$, então cada chamada recursiva possui custo $O(n)$. Dessa forma, a recorrência aproximada é:

$$T(n) = T(n - 1) + O(n).$$

Expandindo:

$$T(n) = O(n) + O(n-1) + \dots + O(1).$$

Logo,

$$T(n) = O\left(\sum_{k=1}^n k\right) = O\left(\frac{n(n+1)}{2}\right).$$

Portanto, a complexidade temporal é $T(n) = O(n^2)$. A profundidade da recursão é n , pois a função chama a si mesma até que $n = 0$. Assim, o consumo de memória é linear:

$$S(n) = O(n).$$

A conclusão é que o algoritmo proposto não é eficiente, pois recalcula $fatorial(n)$ e x^n repetidamente. Uma abordagem mais eficiente poderia reutilizar resultados previamente calculados, reduzindo a complexidade para $O(n)$.

Problema: O máximo divisor comum (MDC) entre dois inteiros a e b é o maior inteiro d que divide simultaneamente a e b . Considere $a = 1071$ e $b = 462$. Primeiramente, calculamos quantas vezes b cabe em a e tomamos o resto:

$$1071 \div 462 = 2 \quad \text{e} \quad 1071 \bmod 462 = 147.$$

O processo é repetido, agora dividindo 462 por 147:

$$462 \div 147 = 3 \quad \text{e} \quad 462 \bmod 147 = 21.$$

Repetindo novamente:

$$147 \div 21 = 7 \quad \text{e} \quad 147 \bmod 21 = 0.$$

Como o resto tornou-se zero, o algoritmo termina e concluímos que $\text{mdc}(1071, 462) = 21$.

Esse procedimento é conhecido como **Algoritmo de Euclides**.

```
mdc(a, b) {
    while b > 0 {
        tmp = a
        a = b
        b = tmp % b
    }
    return a
}
```

Podemos implementar o algoritmo anterior de maneira recursiva.

```

mdc_r(a, b) {
    if b == 0
        return a
    else
        return mdc_r(b, a % b)
}

```

O Algoritmo de Euclides reduz significativamente o tamanho dos argumentos a cada iteração. Pode-se demonstrar que o número de iterações é limitado por:

$$O(\log \min(a, b)).$$

Mais precisamente, o pior caso ocorre quando os valores de entrada são números consecutivos da sequência de Fibonacci. Assim, tanto a versão iterativa quanto a recursiva possuem:

$$T(n) = O(\log n),$$

onde $n = \min(a, b)$. A complexidade de espaço é dada por:

- **Versão iterativa:** utiliza apenas um número constante de variáveis auxiliares. Logo,

$$S(n) = O(1).$$

- **Versão recursiva:** cada chamada recursiva permanece na pilha até atingir o caso base. Como a profundidade é $O(\log n)$, temos:

$$S(n) = O(\log n).$$

Portanto, ambas as implementações possuem complexidade temporal logarítmica, sendo a versão iterativa mais eficiente em termos de uso de memória.

Problema: Desenvolva um algoritmo recursivo para inverter os elementos de uma lista L . A ideia consiste em utilizar dois índices:

$$\text{inicio} = 0 \quad \text{e} \quad \text{fim} = \text{len}(L) - 1.$$

Em cada chamada recursiva, trocamos o primeiro e o último elementos da sublista considerada e, em seguida, chamamos a função novamente com inicio incrementado de 1 e fim decrementado de 1. A condição de parada ocorre quando:

$$\text{fim} \leq \text{inicio}.$$

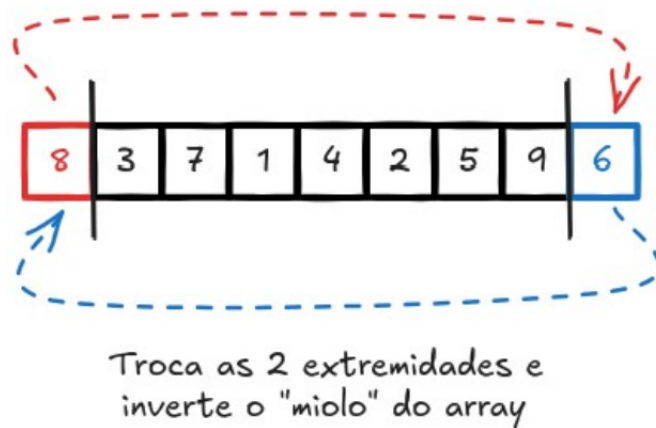


Figura 4: Procedimento recursivo utilizado na inversão de uma lista.

```

inverte(L, inicio, fim) {
  if fim <= inicio
    return L
  else {
    swap(L[inicio], L[fim])
    inverte(L, inicio+1, fim-1)
  }
}

```

Seja n o tamanho da lista. A cada chamada recursiva, dois elementos são posicionados corretamente (um no início e outro no fim). Assim, o número de chamadas recursivas é aproximadamente:

$$\frac{n}{2}.$$

Podemos modelar o tempo de execução pela recorrência:

$$T(n) = T(n - 2) + O(1),$$

pois a operação de troca (`swap`) possui custo constante. Expandindo a recorrência:

$$T(n) = O(1) + O(1) + \dots + O(1),$$

com aproximadamente $\frac{n}{2}$ termos. Logo,

$$T(n) = O(n).$$

A profundidade da recursão é $\frac{n}{2}$, ou seja, proporcional a n . Portanto, o consumo de memória na pilha de execução é:

$$S(n) = O(n).$$

3 8 4	3 dígitos	(n = 3)
x 1 5 6	3 dígitos	
2 3 0 4	9 multiplicações	
1 9 2 0 +	+	
3 8 4 +	2 adições	
5 9 9 0 4	$O(n^2)$	

Figura 5: A multiplicação de dois inteiros de n dígitos tem complexidade computacional $O(n^2)$.

Concluimos que o algoritmo recursivo para inversão de uma lista possui complexidade linear em tempo e espaço.

2.3 O Problema da Multiplicação de Inteiros

Nesta seção, iremos projetar e analisar um algoritmo recursivo para a multiplicação de números inteiros. Recorde que a multiplicação tradicional de dois inteiros com n dígitos requer aproximadamente n^2 multiplicações elementares, além de um número proporcional de adições. A pergunta natural é: *podemos fazer melhor do que $O(n^2)$* ?

Sem perda de generalidade, assumimos que $n = 2^m$ (ou seja, n é potência de 2), de modo a simplificar a divisão recursiva do problema. Considere:

$$a = 12345678, \quad b = 87654321.$$

Podemos escrever:

$$a = 12340000 + 5678,$$

$$b = 87650000 + 4321.$$

Ou, de forma estruturada,

$$a = 1234 \times 10^4 + 5678,$$

$$b = 8765 \times 10^4 + 4321.$$

Logo,

$$a \times b = (1234 \times 10^4 + 5678)(8765 \times 10^4 + 4321).$$

Aplicando a propriedade distributiva:

$$a \times b = 1234 \cdot 8765 \cdot 10^8 + (1234 \cdot 4321 + 5678 \cdot 8765) \cdot 10^4 + 5678 \cdot 4321.$$

De modo geral, podemos escrever:

$$a = a_1 10^{n/2} + a_2,$$

$$b = b_1 10^{n/2} + b_2,$$

onde a_1 e a_2 são as partes superior e inferior de a , e b_1 e b_2 são as partes superior e inferior de b . A multiplicação pode então ser expressa como:

$$a \times b = A \cdot 10^n + (B + C) \cdot 10^{n/2} + D,$$

onde

$$A = a_1 b_1, \quad B = a_1 b_2, \quad C = a_2 b_1, \quad D = a_2 b_2.$$

Assim, podemos elaborar o seguinte algoritmo recursivo:

```
Multiply(a, b) {
    if len(a) <= 1
        return a * b

    // Particionamento custa O(n)
    Particione a e b em
        a = a1 * 10^(n/2) + a2
        b = b1 * 10^(n/2) + b2

    A = Multiply(a1, b1)
    B = Multiply(a1, b2)
    C = Multiply(a2, b1)
    D = Multiply(a2, b2)
    P = A*10^n + (B + C)*10^(n/2) + D

    return P
}
```

Um problema de tamanho n é dividido em quatro subproblemas de tamanho $n/2$, além de um particionamento com custo $O(n)$. Assim, a recorrência é:

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n).$$

Expandindo:

$$T(n) = 4 \left[4T\left(\frac{n}{4}\right) + \frac{n}{2} \right] + n.$$

Expandindo novamente:

$$T(n) = 4^2 T\left(\frac{n}{4}\right) + 2n + n.$$

Generalizando para um nível arbitrário k :

$$T(n) = (2^2)^k T\left(\frac{n}{2^k}\right) + n \sum_{i=0}^{k-1} 2^i.$$

O somatório geométrico é:

$$\sum_{i=0}^{k-1} 2^i = 2^k - 1.$$

Logo,

$$T(n) = (2^{2k}) T\left(\frac{n}{2^k}\right) + n(2^k - 1).$$

No caso base, temos $n = 2^k$, portanto:

$$k = \log_2 n.$$

Substituindo:

$$T(n) = 2^{2 \log_2 n} T(1) + n(2^{\log_2 n} - 1).$$

Como

$$2^{\log_2 n} = n,$$

obtemos:

$$T(n) = n^2 + n(n - 1) = 2n^2 - n.$$

Portanto,

$$T(n) = O(n^2).$$

Portanto, a estratégia recursiva apresentada mantém complexidade assintótica quadrática. Apesar da divisão em subproblemas, o número de multiplicações recursivas (quatro) impede a redução abaixo de $O(n^2)$.

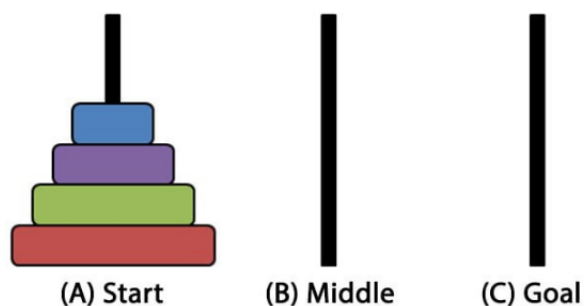


Figura 6: Exemplo de uma instância do problema da torre de Hanoi com $n = 3$ discos.

Para obter uma melhoria assintótica, seria necessário reduzir o número de multiplicações recursivas, como ocorre no algoritmo de Karatsuba.

2.4 O Problema da Torre de Hanói

Imagine que temos três hastes (A , B e C) e, inicialmente, n discos de tamanhos distintos empilhados na haste A , de modo que discos maiores nunca podem ser colocados sobre discos menores. O objetivo é mover todos os discos para outra haste, respeitando as seguintes regras:

1. Apenas um disco pode ser movido por vez;
2. Nunca pode haver um disco menor abaixo de um disco maior.

Iremos analisar alguns casos em que n é pequeno.

Caso $n = 1$:

Apenas um movimento é necessário:

Move $A \rightarrow B$

Caso $n = 2$:

São necessários 3 movimentos:

Move $A \rightarrow B$

Move $A \rightarrow C$

Move $B \rightarrow C$

Caso $n = 3$:

São necessários 7 movimentos:

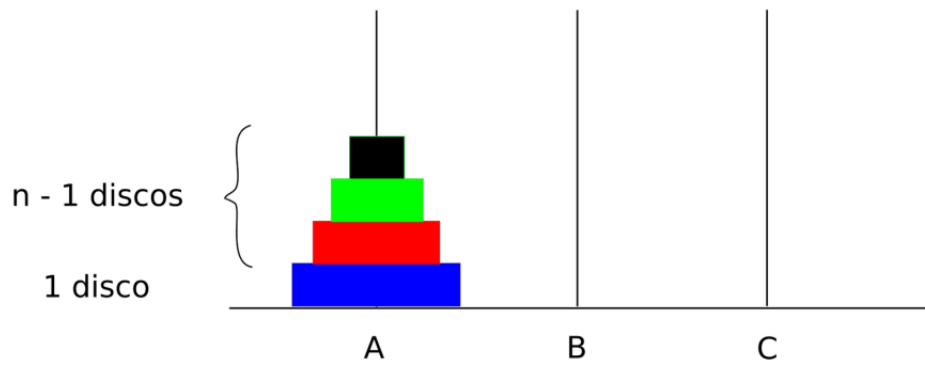


Figura 7: Lógica para solução recursiva do problema da torre de Hanói.

Move $A \rightarrow B$

Move $A \rightarrow C$

Move $B \rightarrow C$

Move $A \rightarrow B$

Move $C \rightarrow A$

Move $C \rightarrow B$

Move $A \rightarrow B$

Observe que, utilizando uma abordagem recursiva, temos:

3 movimentos (para os dois menores) + 1 movimento (para o maior) + 3 movimentos.

Caso $n = 4$:

$$7 + 1 + 7 = 15 \text{ movimentos.}$$

Caso $n = 5$:

$$15 + 1 + 15 = 31 \text{ movimentos.}$$

A lógica geral é:

- Para mover os $n - 1$ discos menores: $T(n - 1)$ movimentos;
- Para mover o maior disco: 1 movimento;
- Para reposicionar os $n - 1$ discos: $T(n - 1)$ movimentos.

A seguir apresentamos um algoritmo recursivo para solução do problema da torre de Hanói.

```

// Move n discos de from para to usando aux
Hanoi(n, from, to, aux) {
    if n == 0:
        return
    // Move recursivamente os n-1 discos superiores
    Hanoi(n-1, from, aux, to)
    // Move o maior disco
    print("Move disk", n, "from rod", from, "to rod", to)
    // Move recursivamente os n-1 discos
    Hanoi(n-1, aux, to, from)
}

// Exemplo de chamada
// Mover N discos de A para C usando B
N = 5
Hanoi(N, 'A', 'C', 'B')

```

A recorrência associada ao algoritmo é:

$$T(n) = 2T(n-1) + O(1).$$

Expandindo:

$$T(n) = 2(2T(n-2) + O(1)) + O(1) = 2^2T(n-2) + O(1).$$

Expandindo novamente:

$$T(n) = 2^3T(n-3) + O(1).$$

Generalizando para o k -ésimo passo:

$$T(n) = 2^kT(n-k) + O(1).$$

A condição de parada ocorre quando $n - k = 0$, ou seja, $k = n$. Logo:

$$T(n) = 2^nT(0) + O(1).$$

Como $T(0)$ é constante:

$$T(n) = O(2^n).$$

Portanto, o algoritmo da Torre de Hanói possui complexidade exponencial, o que o torna impraticável para valores grandes de n .

2.5 Busca Sequencial vs. Busca Binária

Uma tarefa fundamental na computação consiste em buscar um elemento em um vetor (array). A forma mais simples de realizar essa tarefa é por meio da **busca sequencial**, na qual percorremos o vetor elemento por elemento até encontrar o valor desejado.

A seguir apresentamos um algoritmo iterativo para a busca sequencial.

```
busca_sequencial(L, n, x) {  
    i = 1  
    while i <= n {  
        if L[i] == x  
            return i  
        else  
            i = i + 1  
    }  
    return False  
}
```

No pior caso, o elemento procurado encontra-se na última posição do vetor (ou não está presente). Nesse cenário, o laço executa $n - 1$ incrementos.

Logo,

$$T(n) = 1 + (n - 1) = n,$$

o que implica:

$$T(n) = O(n).$$

Portanto, a busca sequencial possui complexidade linear.

Busca Binária

A busca binária exige que o vetor esteja ordenado. O método imita o processo de procurar uma palavra em um dicionário: abrimos aproximadamente no meio e descartamos metade do espaço de busca a cada passo. A ideia consiste em:

- acessar o elemento central;
- se for o elemento desejado, encerrar;
- se o valor procurado for menor, buscar na metade esquerda;
- caso contrário, buscar na metade direita.

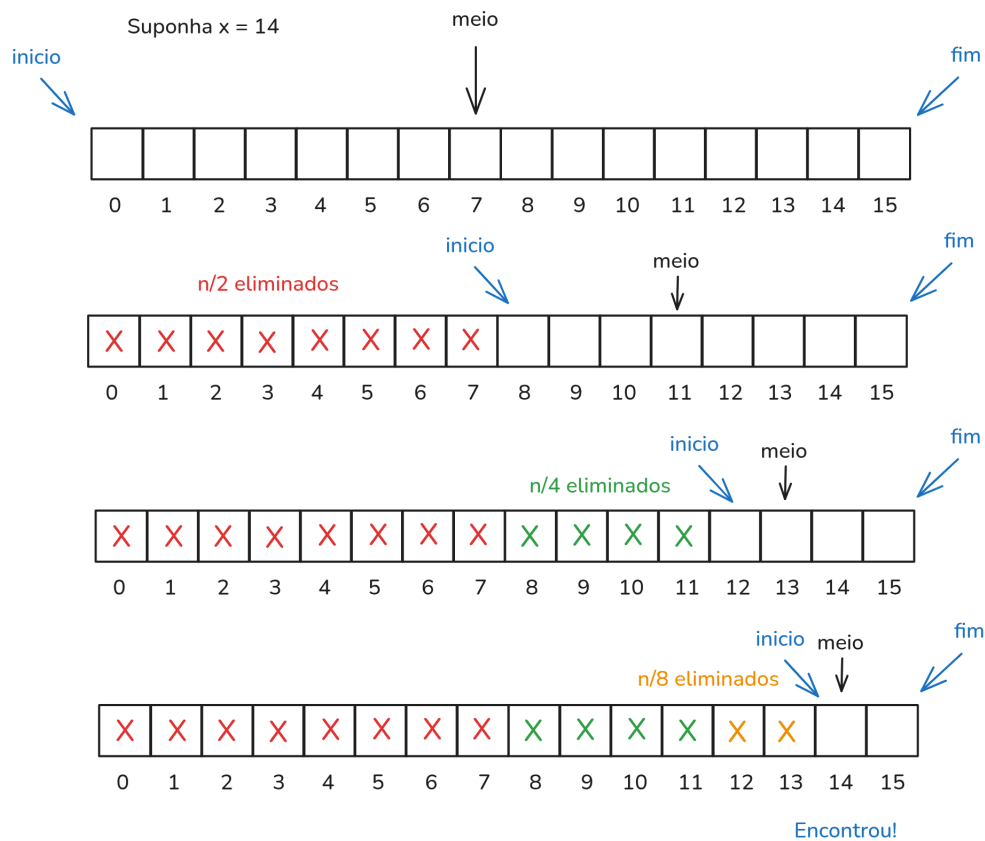


Figura 8: Ilustração gráfica da busca binária pela chave $x = 14$ em um array de inteiros.

```
busca_binaria(L, x, ini, fim) {
    meio = (ini + fim) // 2
    if ini > fim
        return -1 // elemento nao encontrado
    else {
        if L[meio] == x
            return meio
        else {
            if L[meio] > x
                return busca_binaria(L, x, ini, meio-1)
            else
                return busca_binaria(L, x, meio+1, fim)
        }
    }
}
```

Na busca binária, a cada acesso descartamos metade dos elementos restantes. Suponha, para simplificar, que:

$$n = 2^m.$$

A cada passo:

$$\begin{aligned} 1^\circ \text{ acesso} &\rightarrow \frac{n}{2} \text{ elementos restantes} \\ 2^\circ \text{ acesso} &\rightarrow \frac{n}{4} \\ 3^\circ \text{ acesso} &\rightarrow \frac{n}{8} \\ 4^\circ \text{ acesso} &\rightarrow \frac{n}{16} \\ &\vdots \end{aligned}$$

A recorrência associada é:

$$T(n) = T\left(\frac{n}{2}\right) + O(1).$$

Expandindo:

$$T(n) = T\left(\frac{n}{4}\right) + 2,$$

$$T(n) = T\left(\frac{n}{8}\right) + 3,$$

Generalizando:

$$T(n) = T\left(\frac{n}{2^k}\right) + k.$$

No limite da recursão:

$$\frac{n}{2^k} = 1.$$

Logo,

$$n = 2^k, \quad k = \log_2 n.$$

Substituindo:

$$T(n) = T(1) + \log_2 n.$$

Como $T(1)$ é constante:

$$T(n) = O(\log n).$$

Sendo assim, temos que:

- Busca sequencial: $O(n)$
- Busca binária: $O(\log n)$

A função $\log n$ cresce muito mais lentamente que a função linear n . Enquanto a derivada da função linear é constante e igual a 1, a derivada de $\log n$ é:

$$\frac{1}{n},$$

que decresce conforme n aumenta. Por exemplo, em um vetor com 1024 elementos, $\log_2 1024 = 10$. Assim:

- Busca sequencial: até 1023 acessos;
- Busca binária: no máximo 10 acessos.

Ou seja, aproximadamente 1% do custo da busca sequencial. Isso ocorre porque, na busca binária, a cada passo metade do vetor é descartada.

2.6 Considerações Finais

Neste capítulo, exploramos o conceito de recursão como uma estratégia fundamental para a construção de algoritmos. A ideia central consiste em resolver um problema decompondo-o em subproblemas menores da mesma natureza, até que uma condição de parada simples seja atingida. Essa estrutura permite formular soluções elegantes e frequentemente mais próximas da definição matemática dos problemas.

Ao longo dos exemplos apresentados, como fatorial, somatório, cálculo de séries, máximo divisor comum, inversão de listas, multiplicação de inteiros, Torre de Hanói e busca binária, observamos que a recursão pode levar tanto a algoritmos eficientes (complexidade $O(\log n)$, como na busca binária e no algoritmo de Euclides) quanto a soluções com custo elevado (como a Torre de Hanói, com complexidade exponencial $O(2^n)$).

Um ponto essencial na análise de algoritmos recursivos é a formulação de recorrências. A partir delas, é possível expandir a recursão, identificar padrões e determinar o comportamento assintótico da solução. Essa etapa é crucial para avaliar se a abordagem recursiva é apenas conceitualmente elegante ou também computacionalmente eficiente.

Também discutimos que a recursão possui custo adicional de memória, devido à pilha de chamadas. Em muitos casos, uma versão iterativa pode reduzir o consumo de espaço. Por outro lado, existem situações em que a formulação recursiva é mais natural e conduz a implementações mais claras e estruturadas.

Concluimos que a recursão é uma ferramenta poderosa no projeto de algoritmos, especialmente quando combinada com técnicas como divisão e conquista, programação dinâmica e redução de problemas. Entretanto, sua aplicação deve sempre ser acompanhada de uma análise criteriosa de complexidade temporal e espacial.

3 Tipos Abstratos de Dados (TAD's)

Na programação de computadores, o conceito de **abstração** é fundamental para o desenvolvimento de software em níveis mais elevados. Um exemplo clássico de abstração são as funções, que representam abstrações de processos computacionais. Uma vez definida uma função, toda a lógica interna fica encapsulada do usuário. Sempre que necessário, o programador apenas invoca a função, sem precisar conhecer os detalhes de sua implementação.

À medida que a complexidade dos sistemas cresce, torna-se necessário aplicar o mesmo princípio não apenas a processos, mas também aos dados. É nesse contexto que surgem os **Tipos Abstratos de Dados (TAD's)**, que permitem estruturar e organizar informações de maneira controlada e modular.

Um Tipo Abstrato de Dados (TAD) é um tipo definido pelo programador que especifica:

- um conjunto de variáveis responsáveis por armazenar informações;
- um conjunto de operações bem definidas que podem ser aplicadas sobre essas variáveis.

A principal característica de um TAD é o *encapsulamento*. Sua implementação interna é ocultada, de modo que o acesso aos dados ocorre exclusivamente por meio de uma **interface**, definida pelo conjunto de operações disponíveis.

Assim, o usuário do TAD interage apenas com essa interface, sem necessidade de conhecer como os dados são armazenados ou como as operações são implementadas internamente. Essa separação entre *interface* e *implementação* favorece modularidade, reutilização de código e manutenção de sistemas.

A grande vantagem desse modelo é que, uma vez definido o TAD, sua utilização depende apenas do conhecimento das operações oferecidas pela interface, independentemente dos detalhes internos de implementação.

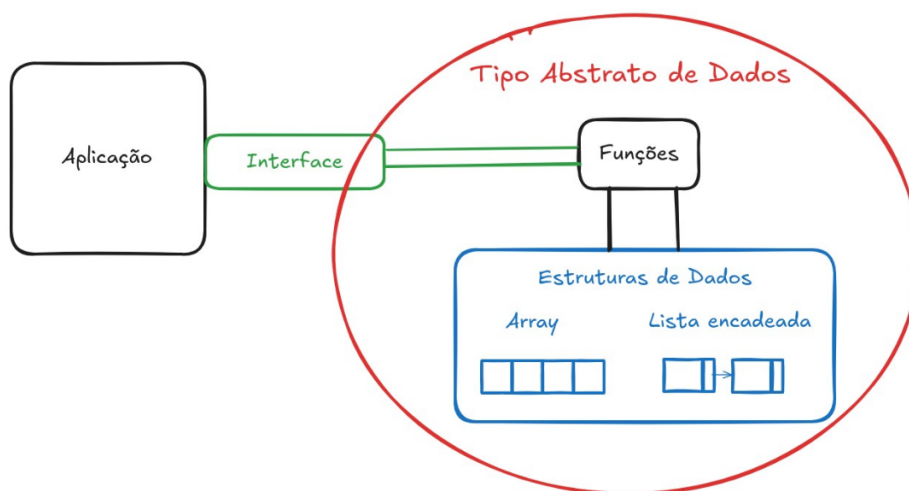


Figura 9: Diagrama ilustrativo esquemático de um Tipo Abstrato de Dados (TAD).

3.1 Tipos Abstratos de Dados e Programação Orientada a Objetos

Os Tipos Abstratos de Dados (TAD's) são considerados precursores da Programação Orientada a Objetos (POO). Um exemplo de TAD implementado nativamente na linguagem Python por meio da orientação a objetos são as listas. Uma lista em Python consiste, essencialmente, em uma variável composta heterogênea (isto é, capaz de armazenar informações de diferentes tipos de dados) associada a um conjunto de operações para manipulação dessa estrutura.

Não é necessário conhecer os detalhes internos da implementação de uma lista em Python para utilizá-la corretamente. Basta conhecer sua interface, ou seja, as operações disponíveis e os parâmetros exigidos por cada uma delas. Entre os principais métodos disponíveis para listas em Python, destacam-se:

- `L.append(x)`: adiciona o elemento `x` ao final da lista `L`;
- `L.pop()`: remove o último elemento da lista `L`;
- `L.pop(i)`: remove o elemento da posição `i`;
- `L.insert(i, x)`: insere o elemento `x` na posição `i`;
- `L.reverse()`: inverte a ordem dos elementos da lista `L`;
- `L.sort()`: ordena os elementos da lista `L`.

Observe que, para o programador, a implementação desses métodos permanece encapsulada. Não é necessário compreender seus detalhes internos, sendo suficiente conhecer a interface das funções, isto é, quais parâmetros devem ser fornecidos e qual comportamento esperar. Por exemplo, na função `L.insert(i, x)`, o primeiro parâmetro representa o índice da posição na lista, enquanto o segundo parâmetro corresponde ao valor a ser inserido.

Existem diversas vantagens na utilização de TAD's em programação, especialmente na implementação de estruturas de dados:

1. Foco na resolução do problema, e não nos detalhes de implementação;
2. Redução de erros por meio do encapsulamento de código previamente validado;
3. Facilidade de correção de *bugs* e manutenção de código, pois a implementação interna de um TAD pode ser modificada sem impactar os programas que utilizam sua interface;
4. Redução da complexidade no desenvolvimento de software, já que sistemas extensos podem ser divididos em módulos independentes, permitindo que equipes trabalhem de forma paralela.

Um TAD é uma abstração que pode ser implementada tanto em linguagens procedurais, como C e Pascal, quanto em linguagens orientadas a objetos, como C++, Java e Python. No contexto da POO, a implementação de um TAD conduz ao conceito de classe, que, após ser instanciada, origina um

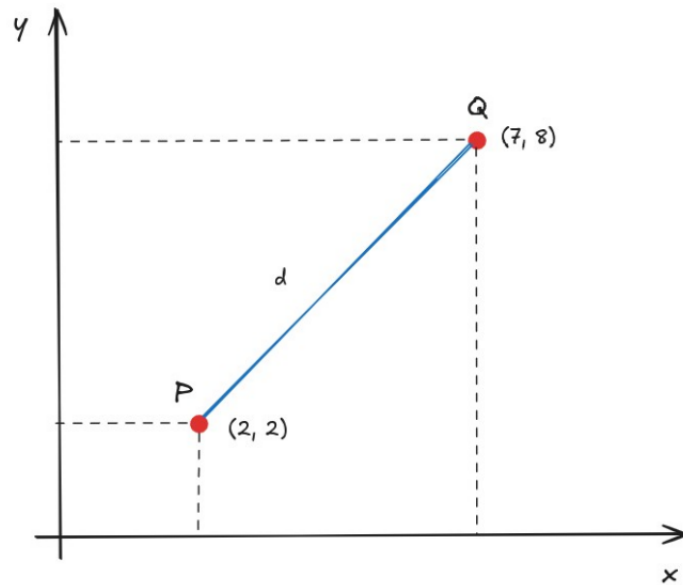


Figura 10: Ilustração gráfica de um ponto no plano cartesiano.

objeto. Uma classe pode ser entendida como um *template* composto por um conjunto de atributos — responsáveis pelo armazenamento de informações — e um conjunto de métodos — responsáveis por operar sobre esses atributos.

3.2 Implementação de um TAD em C: TAD `Ponto`

Considere a criação de um Tipo Abstrato de Dados denominado `Ponto`, cujo objetivo é representar um ponto no plano cartesiano e disponibilizar um conjunto de operações básicas para sua manipulação.

As operações definidas para o TAD `Ponto` são:

1. **cria**: cria um ponto com coordenadas (x, y) ;
2. **libera**: libera a memória alocada para um ponto;
3. **acessa**: retorna as coordenadas de um ponto;
4. **atribui**: atribui novos valores às coordenadas de um ponto;
5. **distancia**: calcula a distância entre dois pontos.

Em linguagem C, inicialmente define-se o arquivo de interface `ponto.h`, que especifica o tipo exportado e as funções disponíveis.

Arquivo `ponto.h` (Interface)

```

// TAD: Ponto (interface)

//***** Tipo exportado
typedef struct ponto Ponto;

//***** Funções exportadas

// Função cria
// Aloca e retorna um ponto com coordenadas (x, y)
Ponto *pto_cria(float x, float y);

// Função libera
// Libera a memória de um ponto previamente criado
void pto_libera(Ponto *p);

// Função acessa (get)
// Retorna os valores das coordenadas de um ponto
void pto_acessa(Ponto *p, float *x, float *y);

// Função atribui (set)
// Atribui novos valores às coordenadas de um ponto
void pto_atribui(Ponto *p, float x, float y);

// Função distancia
// Retorna a distância entre dois pontos
float pto_distancia(Ponto *p1 , Ponto *p2);

```

Observe que, no arquivo de interface, a estrutura interna não é revelada. Apenas o tipo abstrato `Ponto` e as assinaturas das funções são expostos. Em seguida, define-se a implementação do TAD no arquivo `ponto.c`.

Arquivo `ponto.c` (Implementação)

```

#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include "ponto.h"

//*****
// Implementação do TAD
//*****

```

```

// Definição da estrutura ponto
struct ponto {
    float x;
    float y;
};

// Função para criar ponto
Ponto *pto_cria(float x, float y) {
    Ponto *p = (Ponto *)malloc(sizeof(Ponto));
    if (p == NULL) {
        printf("Memória insuficiente!\n");
        exit(1);
    }
    p->x = x;
    p->y = y;
    return p;
}

// Função para liberar memória
void pto_libera(Ponto *p) {
    free(p);
}

// Função get
void pto_acessa(Ponto *p, float *x, float *y) {
    *x = p->x;
    *y = p->y;
}

// Função set
void pto_atribui(Ponto *p, float x, float y) {
    p->x = x;
    p->y = y;
}

// Função para calcular a distância entre 2 pontos
float pto_distancia(Ponto *p1, Ponto *p2) {
    float dx = p2->x - p1->x;
    float dy = p2->y - p1->y;
    return sqrt(dx*dx + dy*dy);
}

```

```

}

// Função main (usa o TAD criado)
int main(void) {
    Ponto *p = pto_cria(2.0, 1.0);
    Ponto *q = pto_cria(3.4, 2.1);
    float d = pto_distancia(p, q);
    printf("Distancia entre pontos: %f\n", d);

    pto_libera(q);
    pto_libera(p);
    return 0;
}

```

Note que a definição da estrutura `struct ponto` aparece apenas no arquivo de implementação, garantindo o encapsulamento dos dados. O usuário do TAD interage exclusivamente por meio das funções definidas na interface, sem acesso direto aos campos internos `x` e `y`. Esse mecanismo assegura modularidade, ocultação de informação e maior segurança na manipulação da estrutura.

3.3 Exercício: Implementação de um TAD `Circulo`

Considere a criação de um Tipo Abstrato de Dados denominado `Circulo`, cujo objetivo é representar um círculo no plano cartesiano e disponibilizar funções para sua manipulação. As operações definidas para o TAD `Circulo` são:

1. **cria**: cria um círculo com centro (x, y) e raio r ;
2. **libera**: libera a memória alocada para um círculo;
3. **area**: calcula a área do círculo;
4. **interior**: verifica se um determinado ponto pertence ao interior do círculo.

Observe que um círculo é composto por um `Ponto`, que representa seu centro, e por um valor real correspondente ao raio. Assim, o TAD `Circulo` reutiliza o TAD `Ponto`, evidenciando o princípio de composição entre abstrações. Inicialmente, define-se o arquivo de interface `circulo.h`, conforme apresentado a seguir.

Arquivo `circulo.h` (Interface)

```

// TAD: Circulo (interface)

#include "ponto.h"

```

```

//***** Tipo exportado
typedef struct circulo Circulo;

//***** Funções exportadas

// Função cria
// Aloca e retorna um círculo com centro (x, y) e raio r
Circulo *circ_cria(float x, float y, float r);

// Função libera
// Libera a memória de um círculo previamente criado
void circ_libera(Circulo *c);

// Função area
// Retorna a área do círculo
float circ_area(Circulo *c);

// Função interior
// Verifica se um dado ponto está no interior do círculo
int circ_interior(Circulo *c, Ponto *p);

```

Em seguida, define-se a implementação do TAD no arquivo `circulo.c`.

Arquivo `circulo.c` (Implementação)

```

#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include "circulo.h"

// malloc, free, exit
// printf
// sqrt

#define PI 3.14159

//*****
// Implementação do TAD
//*****

// Definição da estrutura circulo

```

```

struct circulo {
    Ponto *p;    // Centro
    float r;     // Raio
};

// Função para criar círculo
Circulo *circ_cria(float x, float y, float r) {
    Circulo *c = (Circulo *)malloc(sizeof(Circulo));
    c->p = pto_cria(x, y);
    c->r = r;
    return c;
}

// Função para liberar memória
void circ_libera(Circulo *c) {
    pto_libera(c->p);
    free(c);
}

// Função para calcular a área
float circ_area(Circulo *c) {
    return PI * (c->r) * (c->r);
}

// Função para verificar se um ponto está dentro do círculo
int circ_interior(Circulo *c, Ponto *p) {
    float d = pto_distancia(c->p, p);
    return (d < c->r);
}

// Função main (usa o TAD criado)
int main(void) {
    Circulo *c = circ_cria(5, 5, 2);
    float area = circ_area(c);
    printf("Área do círculo: %f\n", area);

    Ponto *p = pto_cria(4, 4);
    if (circ_interior(c, p) == 0)
        printf("Ponto p fora do círculo\n");
    else
        printf("Ponto p dentro do círculo\n");
}

```

```
pto_libera(p);  
circ_libera(c);  
  
return 0;  
}
```

Observe que, assim como no TAD `Ponto`, a estrutura interna `struct circulo` é definida apenas no arquivo de implementação, garantindo o encapsulamento dos dados. O usuário interage com o círculo exclusivamente por meio das funções declaradas na interface. Além disso, este exemplo ilustra claramente o princípio de composição de TAD's, uma vez que o `Circulo` utiliza internamente um objeto do tipo `Ponto` para representar seu centro.

3.4 Exercício: Implementação de um TAD `NumeroComplexo`

Considere a criação de um Tipo Abstrato de Dados denominado `NumeroComplexo`, cujo objetivo é representar um número complexo da forma

$$z = a + bi,$$

em que a corresponde à parte real e b à parte imaginária. Deseja-se definir as seguintes operações:

1. Função para criar um número complexo, dados a e b ;
2. Função para liberar um número complexo previamente criado;
3. Função para somar dois números complexos;
4. Função para subtrair dois números complexos;
5. Função para multiplicar dois números complexos;
6. Função para dividir dois números complexos.

Recorde as seguintes propriedades algébricas:

$$(a + bi) + (c + di) = (a + c) + (b + d)i,$$

$$(a + bi) - (c + di) = (a - c) + (b - d)i,$$

$$(a + bi)(c + di) = (ac - bd) + (bc + ad)i.$$

A divisão é definida por:

$$\frac{a + bi}{c + di} = \frac{(a + bi)(c - di)}{c^2 + d^2} = \frac{ac + bd}{c^2 + d^2} + \frac{bc - ad}{c^2 + d^2}i, \quad \text{com } c^2 + d^2 \neq 0.$$

Arquivo `complexo.h` (Interface)

```
// TAD: NumeroComplexo (interface)

//***** Tipo exportado
typedef struct numeroComplexo NumeroComplexo;

//***** Funções exportadas

// Cria número complexo (a + bi)
NumeroComplexo *nc_cria(float a, float b);

// Libera memória
void nc_libera(NumeroComplexo *z);

// Soma: z1 + z2
NumeroComplexo *nc_soma(NumeroComplexo *z1, NumeroComplexo *z2);

// Subtração: z1 - z2
NumeroComplexo *nc_subtrai(NumeroComplexo *z1, NumeroComplexo *z2);

// Multiplicação: z1 * z2
NumeroComplexo *nc_multiplica(NumeroComplexo *z1, NumeroComplexo *z2);

// Divisão: z1 / z2
NumeroComplexo *nc_divide(NumeroComplexo *z1, NumeroComplexo *z2);
```

Arquivo `complexo.c` (Implementação)

```
#include <stdlib.h>
#include <stdio.h>
#include "complexo.h"

// Definição da estrutura
struct numeroComplexo {
    float real;
    float imag;
};
```

```

// Criação
NumeroComplexo *nc_cria(float a, float b) {
    NumeroComplexo *z = (NumeroComplexo *)malloc(sizeof(NumeroComplexo)
    );
    if (z == NULL) {
        printf("Memória insuficiente!\n");
        exit(1);
    }
    z->real = a;
    z->imag = b;
    return z;
}

// Liberação
void nc_libera(NumeroComplexo *z) {
    free(z);
}

// Soma
NumeroComplexo *nc_soma(NumeroComplexo *z1, NumeroComplexo *z2) {
    return nc_cria(z1->real + z2->real, z1->imag + z2->imag);
}

// Subtração
NumeroComplexo *nc_subtrai(NumeroComplexo *z1, NumeroComplexo *z2) {
    return nc_cria(z1->real - z2->real, z1->imag - z2->imag);
}

// Multiplicação
NumeroComplexo *nc_multiplica(NumeroComplexo *z1, NumeroComplexo *z2)
{
    float a = z1->real;
    float b = z1->imag;
    float c = z2->real;
    float d = z2->imag;
    return nc_cria(a*c - b*d, b*c + a*d);
}

// Divisão
NumeroComplexo *nc_divide(NumeroComplexo *z1, NumeroComplexo *z2) {

```

```

float a = z1->real;
float b = z1->imag;
float c = z2->real;
float d = z2->imag;
float denom = c*c + d*d;
if (denom == 0) {
    printf("Erro: divisão por zero!\n");
    exit(1);
}
return nc_cria((a*c + b*d)/denom, (b*c - a*d)/denom);
}

// Exemplo de uso
int main(void) {
    NumeroComplexo *z1 = nc_cria(2, 3);
    NumeroComplexo *z2 = nc_cria(1, -4);

    NumeroComplexo *soma = nc_soma(z1, z2);
    NumeroComplexo *prod = nc_multiplica(z1, z2);

    printf("Soma: %f + %fi\n", soma->real, soma->imag);
    printf("Produto: %f + %fi\n", prod->real, prod->imag);

    nc_libera(z1);
    nc_libera(z2);
    nc_libera(soma);
    nc_libera(prod);

    return 0;
}

```

Observe que a estrutura `struct numeroComplexo` permanece encapsulada no arquivo de implementação. Todas as operações são realizadas por meio das funções exportadas na interface, respeitando o princípio fundamental dos Tipos Abstratos de Dados: ocultação da representação interna e manipulação exclusivamente via operações bem definidas.

3.5 Implementação de um TAD em Python por meio de Classes

Mas como podemos criar um TAD em Python, uma linguagem orientada a objetos? Em Programação Orientada a Objetos (POO), o conceito de Tipo Abstrato de Dados equivale ao conceito de definição de uma classe. Uma classe encapsula atributos (dados) e métodos (operações), fornecendo uma interface bem definida para manipulação dos objetos instanciados. Para criar uma classe `Ponto`

em Python, podemos utilizar a seguinte definição no arquivo `ponto.py`:

```
import math

class Ponto:

    # Construtor: usado para instanciar novos objetos
    def __init__(self, coord_x, coord_y):
        self.x = coord_x
        self.y = coord_y

    # Obtém as coordenadas x e y do ponto
    def pto_acessa(self):
        return (self.x, self.y)

    # Atribui novas coordenadas x e y para o ponto
    def pto_atribui(self, coord_x, coord_y):
        self.x = coord_x
        self.y = coord_y

    # Calcula a distância entre 2 pontos no plano
    def pto_distancia(self, other_point):
        dx = other_point.x - self.x
        dy = other_point.y - self.y
        return math.sqrt(dx**2 + dy**2)

if __name__ == '__main__':
    # Cria objetos
    p = Ponto(2.0, 1.0)
    q = Ponto(3.4, 2.1)
    d = p.pto_distancia(q)
    print('Distância entre pontos: %f' % d)
```

Observe que o método especial `__init__` atua como construtor da classe, sendo automaticamente chamado no momento da criação do objeto. O parâmetro `self` referencia a própria instância da classe, permitindo acesso aos atributos internos. Dessa forma, a classe `Ponto` define uma estrutura de dados composta pelos atributos `x` e `y`, bem como um conjunto de métodos para manipulá-los, caracterizando plenamente um TAD implementado segundo os princípios da Programação Orientada a Objetos.

3.6 Exercício: Classe Equacao_Quadratica em Python

Considere o desenvolvimento de uma classe em Python denominada `Equacao_Quadratica`, cujo objetivo é representar e resolver uma equação do segundo grau da forma

$$ax^2 + bx + c = 0,$$

em que a , b e c são coeficientes reais, com $a \neq 0$. A solução é obtida por meio da fórmula quadrática:

$$\Delta = b^2 - 4ac,$$

$$x_{1,2} = \frac{-b \pm \sqrt{\Delta}}{2a}.$$

A seguir apresenta-se uma possível implementação da classe.

```
from math import sqrt
from math import isnan

class Equacao_Quadratica:

    # Construtor (usado para instanciar novos objetos)
    def __init__(self, a, b, c):
        self.a = float(a)
        self.b = float(b)
        self.c = float(c)
        self.delta = float('nan')
        self.x1 = float('nan')
        self.x2 = float('nan')

    # Retorna string para imprimir equação com comando print
    def __str__(self):
        return str(self.a) + 'x^2 + ' + str(self.b) + 'x + ' + str(self.c)

    # Verifica se é equação do segundo grau (a diferente de zero)
    def eh_quadratica(self):
        return self.a != 0

    # Calcula o valor de delta
    def calcula_delta(self):
        self.delta = self.b**2 - 4 * self.a * self.c

    # Verifica se a equação possui raízes reais
    def raiz_real(self):
```

```

        return self.delta >= 0

# Resolve equação do 2o grau
def resolve(self):
    # Copiando variáveis para simplificar notação
    a = self.a
    b = self.b
    # Verifica se é equação quadrática
    if self.eh_quadratica():
        self.calcula_delta()
        delta = self.delta
        # Verifica se as raízes são reais
        if self.raiz_real():
            self.x1 = (-b - sqrt(delta)) / (2 * a)
            self.x2 = (-b + sqrt(delta)) / (2 * a)
            return (self.x1, self.x2)
        else:
            print('A equação não admite raízes reais')
    else:
        print('A equação não é do segundo grau (coeficiente a = 0)')

if __name__ == '__main__':
    # Cria equação do segundo grau
    equacao = Equacao_Quadratica(1, -5, 6)
    # Imprime na tela
    print(equacao)
    # Resolve a equação utilizando a fórmula quadrática
    print(equacao.resolve())

```

Observe que o método especial `__str__` permite que o objeto seja impresso diretamente com o comando `print`. O método `resolve` encapsula toda a lógica algébrica necessária para calcular as raízes, incluindo as verificações de validade da equação e da existência de raízes reais. Dessa forma, a classe caracteriza um TAD implementado segundo os princípios da Programação Orientada a Objetos, reunindo dados (coeficientes e raízes) e operações em uma única abstração.

Exercício: Refaça o exercício criando um TAD em linguagem C.

3.7 Considerações Finais

Neste capítulo, discutimos o conceito de Tipos Abstratos de Dados (TAD's) como um dos pilares fundamentais da abstração em programação. Vimos que um TAD é definido não apenas por sua estrutura interna de dados, mas principalmente pelo conjunto de operações que compõem sua

interface. Essa separação entre especificação e implementação permite encapsular detalhes internos, promovendo modularidade, reutilização e segurança no desenvolvimento de software.

A partir dos exemplos desenvolvidos, como os TAD's `Ponto`, `Circulo` e `NumeroComplexo` em C, bem como suas versões em Python por meio de classes, foi possível observar como o encapsulamento e a ocultação de informação reduzem o acoplamento entre módulos. Alterações na implementação interna podem ser realizadas sem impactar o código cliente, desde que a interface permaneça inalterada. Esse princípio é essencial para a manutenção, evolução e escalabilidade de sistemas computacionais.

Também foi destacado que os TAD's constituem a base conceitual da Programação Orientada a Objetos. Em linguagens orientadas a objetos, como Python, a definição de uma classe materializa diretamente o conceito de TAD, reunindo atributos (estado) e métodos (comportamento) em uma única abstração coerente. Dessa forma, o programador passa a modelar entidades do problema por meio de estruturas que refletem sua semântica, e não apenas sua representação computacional.

Conclui-se, portanto, que o domínio do conceito de TAD é indispensável para a construção de estruturas de dados corretas, eficientes e bem organizadas. Mais do que uma técnica de implementação, trata-se de uma abordagem conceitual que orienta o projeto de software de maneira sistemática, favorecendo clareza, robustez e qualidade no desenvolvimento de programas.

4 Estruturas de dados lineares

Na programação de computadores, estruturas de dados são abstrações lógicas concebidas para organizar informações na memória de forma eficiente, possibilitando acesso, atualização e manipulação adequados aos requisitos de cada problema. Do ponto de vista conceitual, uma estrutura de dados é composta por uma coleção de elementos, pelas relações existentes entre esses elementos e pelo conjunto de operações definidas para inserção, remoção, consulta e modificação.

Neste capítulo, iniciaremos o estudo das estruturas de dados lineares, com foco em Pilhas, Filas, Deques e Filas de Prioridades. Essas estruturas caracterizam-se por organizar seus elementos segundo regras específicas de inserção e remoção. Uma vez inserido, cada elemento mantém uma posição relativa fixa em relação aos elementos que o precedem e aos que o sucedem. Essa preservação da ordem de encadeamento é a característica fundamental que define uma estrutura linear.

De maneira simplificada, podemos dizer que estruturas lineares possuem duas extremidades bem definidas: início e fim, esquerda e direita, ou ainda base e topo. A terminologia adotada depende do contexto da aplicação. Contudo, o aspecto central não está na nomenclatura, mas sim na política de acesso adotada: isto é, em quais extremidades são permitidas inserções e remoções, e em que ordem os elementos são processados. É precisamente essa política que distingue Pilhas, Filas e Deques, e que determina seu comportamento algorítmico e suas aplicações práticas.

4.1 Pilhas (LIFO)

Uma pilha (*stack*) é uma estrutura de dados linear na qual as operações de inserção e remoção são realizadas sempre na mesma extremidade, usualmente denominada *topo*. A extremidade oposta é chamada de *base*. Quanto mais próximo da base estiver um elemento, maior é o tempo que ele permanece armazenado na estrutura. Em contrapartida, um elemento recém-inserido ocupa necessariamente o topo, sendo portanto o primeiro a ser removido, caso nenhuma nova inserção seja realizada.

Devido a esse princípio de organização, as pilhas são classificadas como estruturas do tipo *LIFO* (*Last In, First Out*), isto é, o último elemento a entrar é o primeiro a sair. Uma analogia simples é a de uma pilha de livros: o primeiro livro colocado ficará na base e será o último a ser retirado. Assim, a ordem de remoção é o inverso da ordem de inserção.

Para implementar um TAD *Pilha*, é necessário definir suas variáveis internas (por exemplo, uma lista para armazenar os elementos) e o conjunto de operações aplicáveis. Podemos utilizar uma lista *P* para armazenar os elementos da pilha, considerando que o final da lista representa o topo.

As principais operações associadas a uma pilha são:

- `is_empty(S)` : verifica se a pilha está vazia;
- `push(S, x)` : insere o elemento *x* no topo da pilha;
- `pop(S)` : remove e retorna o elemento do topo;
- `peek(S)` : consulta o elemento do topo sem removê-lo;

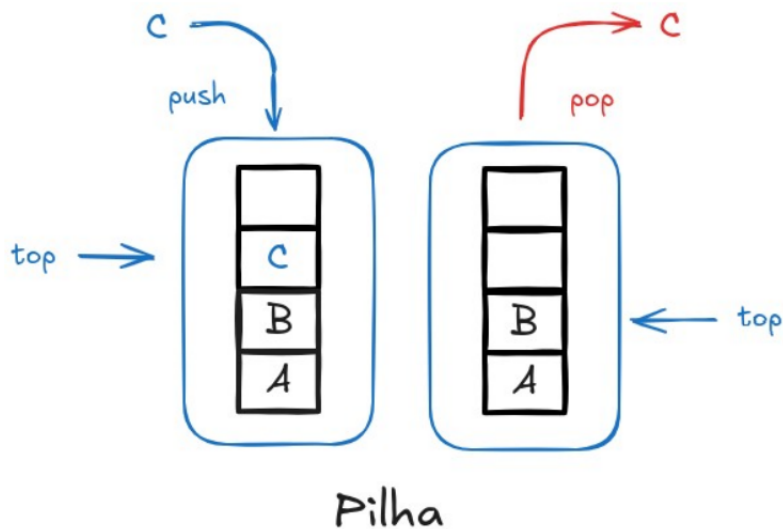


Figura 11: Em uma estrutura de dados Pilha, tanto a inserção (push) quanto a remoção (pop) são realizadas na mesma extremidade, denominada topo.

- `size(S)`: retorna o número de elementos da pilha.

A tabela a seguir ilustra uma sequência de operações realizadas sobre uma pilha inicialmente vazia.

Operação	Conteúdo da Pilha	Retorno	Descrição
<code>is_empty(S)</code>	<code>[]</code>	True	Verifica se a pilha está vazia
<code>push(S, 4)</code>	<code>[4]</code>		Insere elemento no topo
<code>push(S, 9)</code>	<code>[4,9]</code>		Insere elemento no topo
<code>peek(S)</code>	<code>[4,9]</code>	9	Consulta o elemento do topo
<code>push(S, 5)</code>	<code>[4,9,5]</code>		Insere elemento no topo
<code>size(S)</code>	<code>[4,9,5]</code>	3	Número de elementos da pilha
<code>is_empty(S)</code>	<code>[4,9,5]</code>	False	Verifica se a pilha está vazia
<code>push(S, 8)</code>	<code>[4,9,5,8]</code>		Insere elemento no topo
<code>pop(S)</code>	<code>[4,9,5]</code>	8	Remove elemento do topo
<code>pop(S)</code>	<code>[4,9]</code>	5	Remove elemento do topo
<code>size(S)</code>	<code>[4,9]</code>	2	Número de elementos da pilha

Observe que todas as inserções e remoções ocorrem exclusivamente no topo, preservando a propriedade LIFO que caracteriza essa estrutura de dados.

4.1.1 Interface e Implementação Estática de uma Pilha

Antes de implementar as funções necessárias para operar uma estrutura de dados do tipo Pilha, devemos definir sua interface, isto é, o conjunto de primitivas básicas utilizadas para manipular a estrutura. As operações fundamentais são:

```

// Inicializa pilha vazia
init(S)

// Verifica se pilha está vazia
is_empty(S)

// Verifica o topo da pilha
peek(S)

// Insere elemento no topo da pilha
push(S, key)

// Remove elemento do topo da pilha
pop(S)

// Verifica quantos elementos existem na pilha
size(S)

```

Uma pilha S pode ser implementada de maneira estática como um arranjo (array) de tamanho n , em que o último elemento inserido é referenciado por uma variável denominada `top`, que indica o índice do topo da pilha.

```

TAD Stack
    int keys[1..n]    // vetor que armazena os elementos
    int top           // índice do topo da pilha

```

A operação `init(S)` inicializa a pilha, fazendo com que o topo seja igual a zero:

```

init(S) {
    S.top = 0
}

```

A função `is_empty(S)` verifica se a pilha está vazia:

```

is_empty(S) {
    if S.top == 0
        return True
    else
        return False
}

```

A operação `peek(S)` retorna o elemento presente no topo da pilha, sem removê-lo:

```

peek(S) {
    return S.keys[S.top]
}

```

```
}
```

A operação `push(S, key)` insere um novo elemento no topo da pilha, verificando previamente se há espaço disponível (condição de overflow):

```
push(S, key) {  
    if S.top == n  
        error('overflow')  
    else {  
        S.top += 1  
        S.keys[S.top] = key  
    }  
}
```

A operação `pop(S)` remove e retorna o elemento do topo da pilha, verificando se a pilha está vazia (condição de underflow):

```
pop(S) {  
    if is_empty(S)  
        error('underflow')  
    else {  
        key = S.keys[S.top]  
        S.keys[S.top] = NIL  
        S.top -= 1  
        return key  
    }  
}
```

Do ponto de vista de complexidade computacional, observe que as operações de inserção (`push`) e remoção (`pop`) realizam apenas um número constante de operações elementares (atribuições, incrementos e acessos diretos ao vetor). Portanto, ambas possuem complexidade assintótica $O(1)$, isto é, são executadas em tempo constante, independentemente do número de elementos armazenados na pilha.

4.1.2 Aplicação de Pilhas: Verificação de Balanceamento de Parêntesis

Uma aplicação clássica de estruturas de dados do tipo Pilha é a verificação do balanceamento de parêntesis em expressões matemáticas. Dizemos que uma expressão está bem formada quando o número de parêntesis é par e, além disso, para cada símbolo de abertura “(” existe um símbolo de fechamento “)” correspondente.

Por exemplo, a expressão

$$((1 + 2) * (3 + 4)) - (5 + 6)$$

é válida, pois todos os parêntesis estão corretamente balanceados. Já a expressão

$$((1 + 2) * (3 + 4) - (5 + 6)$$

não é válida, pois há um parêntesis de abertura sem o respectivo fechamento.

A estratégia para resolver esse problema consiste em utilizar uma pilha. A ideia é simples:

- Sempre que encontrarmos um parêntesis de abertura “(”, devemos empilhá-lo;
- Sempre que encontrarmos um parêntesis de fechamento “)”, devemos verificar se existe um elemento correspondente no topo da pilha:
 - Se a pilha estiver vazia, a expressão não está balanceada;
 - Caso contrário, desempilhamos o elemento do topo.
- Ao final da varredura, se a pilha estiver vazia, então a expressão é válida.

A seguir, apresenta-se um pseudocódigo para o caso simples em que apenas os símbolos “(” e “)” são permitidos.

```
// Apenas parêntesis são permitidos na expressão: (, )
verifica_expressao_simples(expressao, n) {
    // Cria pilha vazia de tamanho n
    S = Stack()
    balanced = True
    indice = 1    // primeira posição na expressão (de 1 a n)
    // Enquanto não chegar no final e estiver balanceado
    while indice <= n and balanced {
        // Obtém o símbolo atual
        simbolo = expressao[indice]
        // Se for abre parêntesis, empilha
        if simbolo == '('
            push(S, simbolo)
        // Se for fecha parêntesis
        elif simbolo == ')' {
            if is_empty(S)
                balanced = False
            else
                pop(S)
        }
        // Passa para o próximo caracter
        indice += 1
    }
}
```

```

// Verifica condição final
if balanced and is_empty(S)
    return True
else
    return False
}

```

Do ponto de vista de complexidade, o algoritmo percorre a expressão apenas uma vez, realizando operações de empilhar e desempilhar em tempo constante. Logo, sua complexidade temporal é $O(n)$, onde n é o tamanho da expressão.

No caso de expressões compostas, podem existir diferentes tipos de símbolos de parentização, tais como:

(), [], { }.

Nessa situação, além de verificar se a pilha não está vazia ao encontrar um símbolo de fechamento, devemos também conferir se o tipo de abertura corresponde ao fechamento encontrado. Isto é, os pares válidos são:

(), [], {}.

Uma possível estratégia consiste em:

- Empilhar qualquer símbolo de abertura: (, [, {.
- Ao encontrar um símbolo de fechamento:
 - Verificar se a pilha está vazia;
 - Desempilhar o topo;
 - Checar se o par formado é compatível.

Se ao final da varredura a pilha estiver vazia e nenhuma inconsistência tiver sido detectada, a expressão é considerada bem formada. A implementação dessa versão estendida constitui um excelente exercício de programação, pois exige o controle simultâneo de múltiplos tipos de símbolos e reforça o entendimento do comportamento LIFO característico das pilhas.

4.1.3 Aplicação de Pilhas: Conversão de Decimal para Binário

Outro problema interessante que pode ser resolvido com uma estrutura de dados do tipo **Pilha** é a conversão de um número decimal (base 10) para binário (base 2). A representação de números inteiros por computadores digitais é feita na base 2 e não na base 10.

Sabemos que um número na base 10 (decimal) é representado como:

$$23457 = 2 \times 10^4 + 3 \times 10^3 + 4 \times 10^2 + 5 \times 10^1 + 7 \times 10^0$$

Observe que quanto mais à esquerda estiver o dígito, maior o seu valor posicional. Por exemplo, o dígito 2 em 23457 vale, na verdade, 20000, pois é igual a 2×10^4 .

Analogamente, um número binário (base 2) pode ser representado como:

$$110101 = 1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

Calculando:

$$= 32 + 16 + 0 + 4 + 0 + 1 = 53$$

O bit mais à direita é o menos significativo e possui valor $1 \times 2^0 = 1$. O segundo bit a partir da direita tem valor $0 \times 2^1 = 0$. O terceiro bit a partir da direita tem valor $1 \times 2^2 = 4$. O quarto bit a partir da direita tem valor $0 \times 2^3 = 0$. O quinto bit a partir da direita tem valor $1 \times 2^4 = 16$. Por fim, o bit mais à esquerda tem valor $1 \times 2^5 = 32$.

Somando todos os valores, obtemos:

$$1 + 4 + 16 + 32 = 53$$

Essa é a regra para convertermos um número binário para sua notação decimal. Veremos agora o processo inverso: como converter um número decimal para binário. O procedimento consiste em dividir sucessivamente o número por 2, armazenando os restos das divisões.

Considere o número 53:

$$53/2 = 26 \quad \text{resto } 1$$

$$26/2 = 13 \quad \text{resto } 0$$

$$13/2 = 6 \quad \text{resto } 1$$

$$6/2 = 3 \quad \text{resto } 0$$

$$3/2 = 1 \quad \text{resto } 1$$

$$1/2 = 0 \quad \text{resto } 1$$

Quando o quociente se torna zero, o processo pode ser encerrado. Os restos obtidos formam o número binário, sendo que o primeiro resto corresponde ao bit menos significativo. Portanto, lendo os restos de baixo para cima, temos:

$$53_{10} = 110101_2$$

Observe que, se empilharmos os restos em uma pilha, ao final basta desempilhá-los para obter o número binário na ordem correta. O algoritmo a seguir implementa uma função que converte um número inteiro na base 10 para sua representação binária utilizando uma pilha:

```
// Função que converte um número decimal para binário
decimal_binario(numero) {
    S = Stack()
    while numero > 0 {
        resto = numero % 2
        push(S, resto)
        numero = numero // 2
    }
    binario = "" // string vazia
    while not is_empty(S) {
        binario = binario + str(pop(S)) // concatenação
    }
    return binario
}
```

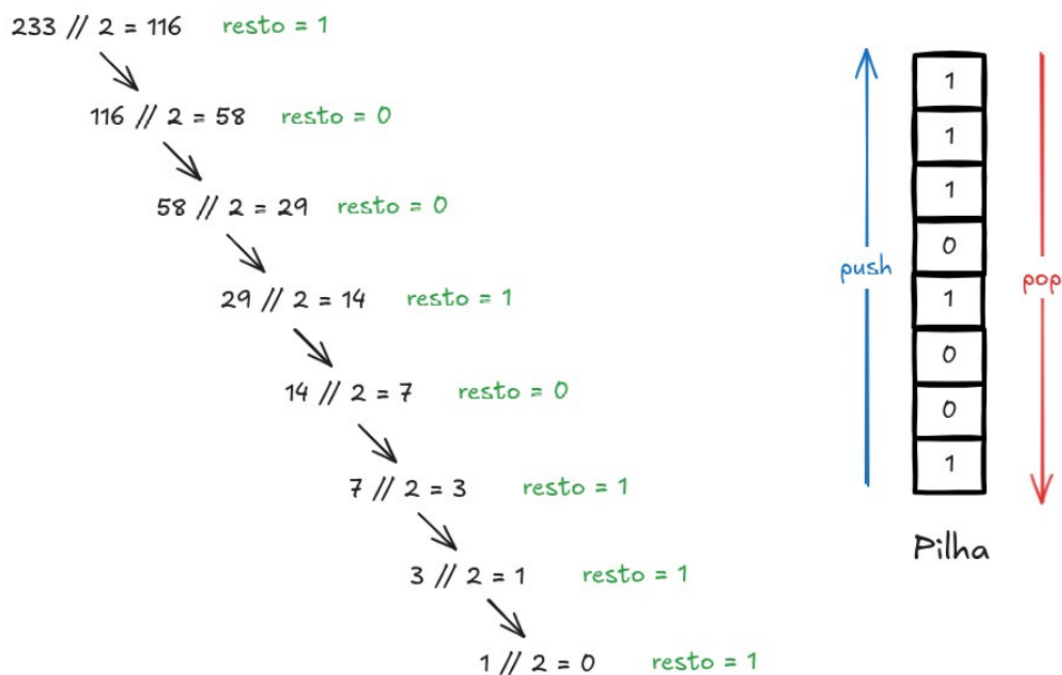


Figura 12: Ao inserir cada bit na Pilha com push e remover cada bit na Pilha com pop, a sequência de bits original é invertida.

Exercício: Escreva uma função para determinar se uma cadeia de caracteres (string) é da forma: xCy onde x e y são cadeias de caracteres compostas por letras 'A' ou 'B', e y é o inverso de x . Isto é, se $x = \text{'AABABBA'}$, então y deve equivaler a 'ABBABAA' . OBS: em cada ponto, você só poderá ler o próximo caractere da cadeia (use uma pilha).

Outro exemplo interessante da aplicação de **Pilhas** é a resolução do seguinte problema: suponha que seja dada uma pilha S contendo números inteiros positivos e negativos. Desenvolva um algoritmo que reorganize a pilha S de modo que todos os números positivos apareçam acima dos números

negativos. A estratégia consiste em utilizar duas pilhas auxiliares:

- P : pilha para armazenar os números positivos (e zero);
- N : pilha para armazenar os números negativos.

Inicialmente, removemos todos os elementos da pilha original S , separando-os em P e N conforme o sinal. Em seguida, reconstruímos S , inserindo primeiro os elementos negativos e depois os positivos, garantindo assim que os positivos fiquem no topo da pilha final.

```
// Função que organiza uma pilha de números inteiros
organiza_pilha(S) {
    P = Stack()    // Pilha dos positivos
    N = Stack()    // Pilha dos negativos
    // Enquanto houver elementos em S
    while not is_empty(S) {
        numero = pop(S)
        if numero >= 0
            push(P, numero)    // Se maior ou igual a zero, vai para P
        else
            push(N, numero)    // Senão, vai para N
    }
    // Transfere os negativos de N para S
    while not is_empty(N)
        push(S, pop(N))
    // Transfere os positivos de P para S
    while not is_empty(P)
        push(S, pop(P))
    return S
}
```

4.1.4 O Problema da Celebridade

Suponha que uma celebridade invada uma festa local. Ele ou ela não conhece ninguém na festa, mas todas as pessoas presentes conhecem a celebridade.

Considere que temos:

- Uma lista com as n pessoas presentes na festa (incluindo a possível celebridade);
- Um mecanismo que permite determinar se uma pessoa A conhece uma pessoa B .

Como podemos utilizar uma **pilha** para determinar de forma eficiente (isto é, em tempo $O(n)$) se existe uma celebridade na festa e, caso exista, identificar quem é essa pessoa? Seja M uma matriz quadrada de dimensões $n \times n$, tal que:

$$M[i, j] = \begin{cases} 1, & \text{se a } i\text{-ésima pessoa conhece a } j\text{-ésima pessoa} \\ 0, & \text{caso contrário} \end{cases}$$

Uma pessoa c é considerada uma **celebridade** se satisfaz simultaneamente:

1. $M[c, j] = 0$, para todo $j \neq c$ (a celebridade não conhece ninguém);
2. $M[i, c] = 1$, para todo $i \neq c$ (todos conhecem a celebridade).

O objetivo consiste em encontrar o índice da celebridade, caso exista. Se não houver nenhuma pessoa que satisfaça essas condições, o algoritmo deve retornar -1 . Um exemplo de matriz M poderia ser:

$$M = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Neste caso, a pessoa de índice 2 é a celebridade, pois:

- A linha 2 contém apenas zeros (ela não conhece ninguém);
- A coluna 2 contém apenas uns, exceto na diagonal principal (todos a conhecem).

Solução Trivial

O algoritmo trivial para resolver o problema da celebridade deve seguir os passos abaixo:

1. Crie dois vetores `indegree` e `outdegree`. O valor de `indegree[i]` deve armazenar quantas pessoas conhecem a i -ésima pessoa, enquanto `outdegree[i]` deve armazenar quantas pessoas a i -ésima pessoa conhece.
2. Execute dois laços aninhados, ambos variando de 0 até $n - 1$:
 - (a) Para cada par (i, j) , se i conhece j , aumente `outdegree[i]` e `indegree[j]`.
 - (b) Para cada par (i, j) , se j conhece i , aumente `outdegree[j]` e `indegree[i]`.
3. Execute um laço de 0 até $n - 1$ e encontre o índice i tal que:

$$\text{indegree}[i] = n - 1 \quad \text{e} \quad \text{outdegree}[i] = 0.$$

```

// Função auxiliar
conhece(i, j, M) {
    return M[i, j]
}

celebridade(M, n) {
    indegree = array(n)
    outdegree = array(n)
    // Laço principal
    for i = 0 to n-1 {
        for j = 0 to n-1 {
            x = conhece(i, j, M)
            indegree[j] += x
            outdegree[i] += x
        }
    }
    // Verifica se há uma celebridade
    for i = 0 to n-1 {
        if indegree[i] == n-1 and outdegree[i] == 0
            return i
    }
    return -1
}

```

É fácil notar que essa solução possui complexidade assintótica $O(n^2)$, pois utiliza dois laços aninhados.

Solução Eficiente com Pilha

Uma abordagem mais eficiente utiliza uma pilha e segue os passos abaixo:

1. Crie uma pilha S e insira todos os identificadores (de 0 até $n - 1$).
2. Enquanto houver mais de um elemento na pilha:
 - (a) Retire dois elementos do topo da pilha, denominando-os A e B .
 - (b) Se A conhece B , então A não pode ser celebridade; insira B de volta na pilha.
 - (c) Caso contrário, B não pode ser celebridade; insira A de volta na pilha.
3. O elemento restante na pilha é o candidato a celebridade.
4. Verifique se esse candidato satisfaz as propriedades: é conhecido por $n - 1$ pessoas e não conhece ninguém.

5. Caso as condições sejam satisfeitas, retorne o índice; caso contrário, retorne -1 .

```
// Função auxiliar
conhece(i, j, M) {
    return M[i, j]
}

celebridade(M, n) {
    S = Stack()
    C = -1
    // Insere todos na pilha
    for i = 0 to n-1
        push(S, i)
    // Encontra a potencial celebridade
    while len(S) > 1 {
        A = pop(S)
        B = pop(S)
        if conhece(A, B, M)
            push(S, B)
        else
            push(S, A)
    }
    // Celebridade potencial
    C = pop(S)
    // Verificação final
    for i = 0 to n-1 {
        if i != C and (conhece(C, i, M) or not conhece(i, C, M))
            return -1
    }
    return C
}
```

Note que as operações primitivas `push` e `pop` possuem complexidade $O(1)$. Como cada elemento é inserido e removido da pilha no máximo uma vez, o algoritmo executa em tempo $O(n)$, sendo portanto mais eficiente do que a solução trivial em termos computacionais.

Exercício: Para um dado número inteiro $n > 1$, o menor inteiro $d > 1$ que divide n é chamado de **fator primo**. É possível determinar a fatora  o prima de n encontrando-se sucessivamente o fator primo d e substituindo n pelo quociente n/d , repetindo esse processo at   que $n = 1$. Utilizando uma estrutura de dados linear para auxiliar na manipula  o dos dados, implemente uma fun  o que compute a fatora  o prima de um n  mero, imprimindo seus fatores em ordem decrescente. Por exemplo, para $n = 3960$, deve ser impresso:

$$11 \times 5 \times 3 \times 3 \times 2 \times 2 \times 2$$

A estrutura de dados escolhida é a **Pilha** (TAD Stack). Durante o processo de fatoração por divisões sucessivas, os fatores primos são obtidos naturalmente em ordem crescente. Por exemplo:

$$3960 = 2 \times 2 \times 2 \times 3 \times 3 \times 5 \times 11$$

Como o exercício solicita que os fatores sejam impressos em ordem decrescente, podemos empilhar os fatores à medida que são encontrados. Como a pilha segue a política LIFO (*Last In, First Out*), ao desempilhar os elementos obteremos automaticamente a ordem decrescente desejada. Essa escolha evita a necessidade de ordenação posterior, mantendo a complexidade adicional constante por operação ($O(1)$ para push e pop).

```
// Função que calcula a fatoração prima de n
fatoracao_prima(n) {
    S = Stack()
    d = 2
    // Encontra fatores primos
    while n > 1 {
        while n % d == 0 {
            push(S, d)
            n = n // d
        }
        d += 1
    }
    // Imprime fatores em ordem decrescente
    while not is_empty(S)
        fator = pop(S)
    if not is_empty(S)
        print(fator)
}
```

O algoritmo realiza divisões sucessivas até que n seja reduzido a 1. No pior caso, o valor de d pode crescer até $\sqrt{n}$, resultando em complexidade temporal $O(\sqrt{n})$. As operações sobre a pilha possuem custo $O(1)$. Portanto, o custo total do algoritmo é dominado pelo processo de fatoração, mantendo-se assintoticamente em $O(\sqrt{n})$.

4.2 Filas (FIFO)

Uma **fila** (*queue*) é uma estrutura de dados linear em que a inserção de elementos é realizada em uma extremidade (final) e a remoção é realizada na outra extremidade (início). Assim como em uma fila de pessoas no mundo real, o primeiro a entrar será o primeiro a sair, o que caracteriza o princí-

pio de ordenação **FIFO** (*First In, First Out*). Além disso, filas são estruturas restritivas no sentido de que um elemento não pode ultrapassar seu antecessor. A ordem de atendimento é estritamente determinada pela sequência de chegada.

Na Ciência da Computação, existem diversos exemplos de aplicações que utilizam filas para gerenciar a ordem de acesso a recursos. Um exemplo clássico é o de um servidor de impressão localizado em um laboratório de pesquisa de um departamento universitário. Esse servidor precisa lidar com múltiplas requisições de impressão que chegam ao longo do tempo. Para organizar essas requisições, o software da impressora mantém uma fila de impressão, garantindo que os trabalhos sejam processados sequencialmente, de acordo com a ordem de chegada.

Implementação de um TAD Fila

Para implementar um **Tipo Abstrato de Dados (TAD) Fila**, devemos definir:

- Suas variáveis internas (estrutura de armazenamento);
- As operações permitidas sobre seus elementos.

Uma fila pode ser implementada a partir de uma lista inicialmente vazia. As operações fundamentais são:

- `enqueue(x)`: insere o elemento x no final da fila;
- `dequeue()`: remove e retorna o elemento no início da fila;
- `front()`: retorna o elemento no início sem removê-lo;
- `is_empty()`: verifica se a fila está vazia.

Observe que, embora conceitualmente diferente, a implementação de uma fila possui operações semelhantes às de uma pilha, diferenciando-se principalmente na política de inserção e remoção de elementos.

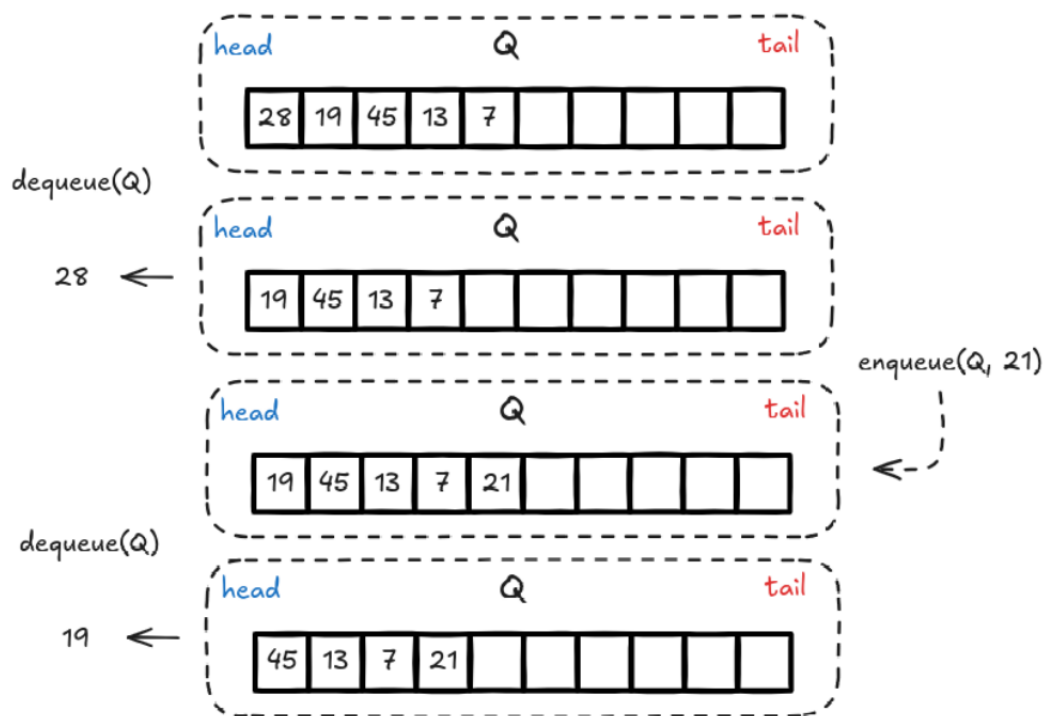


Figura 13: Em uma estrutura de dados do tipo Fila, a inserção (enqueue) é sempre no final e a remoção (dequeue) é sempre no início.

A tabela a seguir ilustra uma sequência de operações realizadas sobre uma fila Q , bem como o conteúdo da fila após cada operação, o valor retornado (quando aplicável) e a descrição correspondente.

Operação	Conteúdo	Retorno	Descrição
is_empty(Q)	[]	True	Verifica se a fila está vazia
enqueue(Q, 4)	[4]		Inserir elemento no final
enqueue(Q, 1)	[1, 4]		Inserir elemento no final
enqueue(Q, 7)	[7, 1, 4]		Inserir elemento no final
size(Q)	[7, 1, 4]	3	Número de elementos da fila
is_empty(Q)	[7, 1, 4]	False	Verifica se a fila está vazia
enqueue(Q, 9)	[9, 7, 1, 4]		Inserir elemento no final
dequeue(Q)	[9, 7, 1]	4	Remove elemento do início
dequeue(Q)	[9, 7]	1	Remove elemento do início
size(Q)	[9, 7]	2	Número de elementos da fila

Projeto de um TAD Fila

Para projetar uma fila, utilizaremos a mesma estratégia adotada na implementação de pilhas. Como atributo principal, teremos um vetor de inteiros inicialmente vazio. Definiremos os seguintes métodos fundamentais:

```

// Inicializa fila vazia
init(Q)

// Verifica se fila está vazia
is_empty(Q)

// Insere elemento no final da fila (equivalente ao push)
enqueue(Q, key)

// Remove elemento do início da fila (equivalente ao pop)
dequeue(Q)

```

Uma fila Q pode ser implementada de forma estática utilizando um vetor de tamanho n , juntamente com dois marcadores:

- head: indica a posição para remoção (início da fila);
- tail: indica a posição para inserção (final da fila);
- size: armazena o número atual de elementos na fila.

Ambas as extremidades podem se mover, caracterizando uma fila circular.

```

TAD Queue
    int keys[1..n]
    int head
    int tail
    int size

```

onde n é o tamanho máximo da fila.

```

init(Q) {
    Q.head = 0
    Q.tail = 0
    Q.size = 0
}

is_empty(Q) {
    if Q.size == 0
        return True
    else
        return False
}

```

```

enqueue(Q, key) {
    if Q.size == n
        error("overflow")
    else {
        if Q.tail == n
            Q.tail = 1          // torna a fila circular
        else
            Q.tail += 1
    }
    Q.keys[Q.tail] = key
    Q.size += 1
}

dequeue(Q) {
    if is_empty(Q)
        error("underflow")
    else {
        if Q.head == n
            Q.head = 1          // torna a fila circular
        else
            Q.head += 1
    }
    key = Q.keys[Q.head]
    Q.size -= 1
    return key
}

```

Tanto a operação de inserção (enqueue) quanto a operação de remoção (dequeue) possuem complexidade temporal $O(1)$, pois envolvem apenas atualizações de índices e acessos diretos ao vetor.

4.2.1 Aplicação de Filas: Escalonamento Round-Robin com Passagem de Token

Uma aplicação clássica de filas na Ciência da Computação é o **escalonamento de processos** em sistemas operacionais, particularmente no algoritmo *Round-Robin*. Podemos compreender esse mecanismo por meio de uma analogia com o jogo infantil *Batata Quente*. Em vez de crianças formando uma roda, temos um conjunto de processos prontos para execução. Em vez de uma batata, temos um **token de execução**, que concede ao processo o direito de utilizar a CPU.

Considere um conjunto de K processos prontos para execução, organizados em uma fila Q . O processo na frente da fila é aquele que atualmente possui o token e pode executar por um intervalo fixo de tempo (quantum).

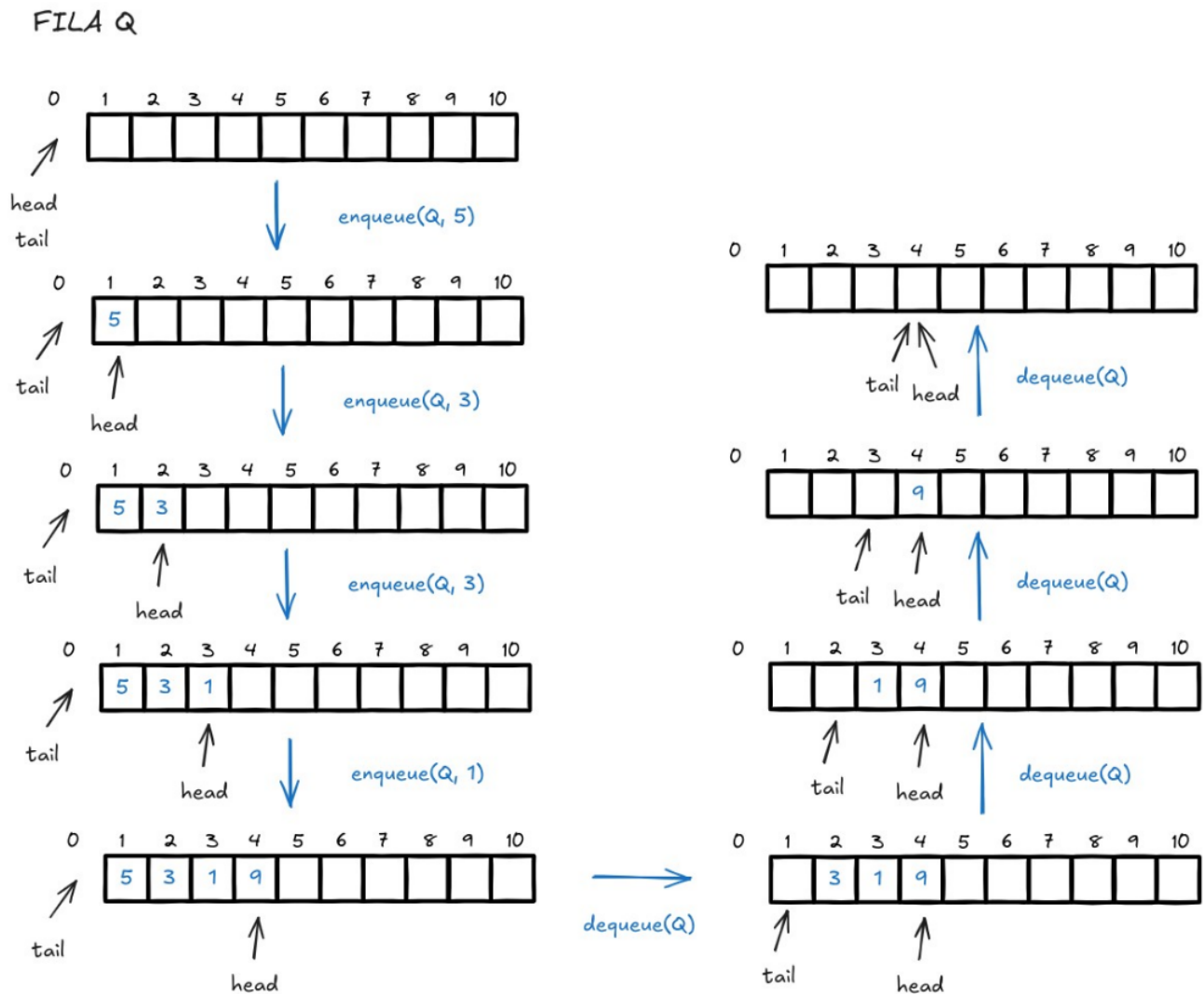


Figura 14: Ilustração de algumas operações elementares realizadas em uma estrutura de dados Fila.

O funcionamento é o seguinte:

- O processo no início da fila executa por um tempo máximo pré-definido MAX (quantum).
- Após esse período, se o processo ainda não terminou, ele é removido do início da fila e reinserido no final (operação `dequeue` seguida de `enqueue`).
- O próximo processo assume o início da fila e recebe o token.
- Se um processo termina sua execução durante seu quantum, ele é removido definitivamente da fila.

O ciclo continua até que reste apenas um processo (ou até que todos finalizem, dependendo do modelo adotado). Nesse contexto:

- A fila modela a lista de processos prontos.

- O token representa o direito exclusivo de acesso à CPU.
- A rotação da fila simula a política de justiça do escalonador.

Esse mecanismo também aparece em sistemas distribuídos baseados em *token passing*, onde apenas o nó que possui o token pode acessar um recurso compartilhado. A seguir, um algoritmo simplificado que simula o escalonamento Round-Robin utilizando uma fila:

```
// Simula escalonamento Round-Robin
// processos = array com identificadores de K processos
// MAX = quantum de execução
round_robin(processos, K, MAX) {
    Q = Queue()
    // Inicializa fila de processos prontos
    for i = 1 to K
        enqueue(Q, processos[i])
    while Q.size > 0 {
        processo = dequeue(Q)
        executar(processo, MAX)
        if not terminou(processo)
            enqueue(Q, processo)
    }
}
```

Cada operação de inserção e remoção na fila possui complexidade $O(1)$. Assim, o custo do algoritmo depende linearmente do número total de quanta necessários para finalizar todos os processos. Embora a analogia com o jogo infantil pareça simples, essa estrutura é fundamental para diversos mecanismos reais em computação, como:

- Escalonamento de CPU em sistemas operacionais;
- Protocolos de acesso a meio compartilhado;
- Controle de concorrência baseado em token;
- Sistemas distribuídos com exclusão mútua.

Exercício: Usando uma pilha P inicialmente vazia, implemente um método para inverter uma fila Q com n elementos utilizando apenas os métodos:

`is_empty()`, `push()`, `pop()`, `enqueue()`, `dequeue()`.

A estratégia baseia-se nas propriedades complementares da fila (FIFO — *First In, First Out*) e da pilha (LIFO — *Last In, First Out*).

- Primeiro, removemos todos os elementos da fila Q e os inserimos na pilha P .
- Em seguida, removemos todos os elementos da pilha P e os reinserimos na fila Q .

Como a pilha inverte a ordem dos elementos devido à sua política LIFO, o resultado final é a fila invertida.

```
// Função para inverter uma fila usando uma pilha
inverte_fila(Q) {
    S = Stack()    // Pilha inicialmente vazia
    // Remove todos os elementos da fila e empilha
    while not is_empty(Q)
        push(S, dequeue(Q))
    // Remove todos os elementos da pilha e reinsere na fila
    while not is_empty(S):
        enqueue(Q, pop(S))
    return Q
}
```

Se inicialmente:

$$Q = [a, b, c, d]$$

Após transferir para a pilha:

$$S = [d, c, b, a]$$

E ao transferir novamente para a fila:

$$Q = [d, c, b, a]$$

ou seja, a ordem foi invertida.

Se a fila possui n elementos:

- O primeiro laço executa n operações de `dequeue()` e n operações de `push()`.
- O segundo laço executa n operações de `pop()` e n operações de `enqueue()`.

Cada uma dessas operações possui custo $O(1)$.

Logo, o número total de operações é proporcional a $2n$, o que resulta em complexidade temporal:

$$O(n)$$

Quanto ao espaço auxiliar, utilizamos uma pilha que armazena até n elementos, portanto a complexidade espacial é:

$$O(n)$$

O algoritmo é linear em tempo e espaço, sendo assintoticamente ótimo, pois qualquer método que inverta uma fila precisa necessariamente processar todos os seus n elementos ao menos uma vez.

Exercício: Dado um número inteiro positivo n , desenvolver um algoritmo que calcule todos os números de 1 até n em representação binária.

Uma abordagem direta consiste em converter cada número individualmente para binário utilizando operações de deslocamento de bits.

```

gera_binarios(n) {
  for i = 1 to n {
    s = ""          // string vazia
    temp = i
    while temp > 0 {
      if temp & 1 == 1      // operador AND lógico
        s = s + "1"
      else
        s = s + "0"
      temp = temp >> 1      // deslocamento para a direita
    }
    print(s)
  }
}

```

A análise do algoritmo revela que:

1. O laço externo executa n vezes.
2. Para cada inteiro i , o número de iterações do laço interno corresponde ao número de bits necessários para representá-lo.

Sabemos que o número de bits necessários para representar i em binário é aproximadamente:

$$\lfloor \log_2 i \rfloor + 1$$

No pior caso, quando $i = n$, temos aproximadamente $\log_2 n$ iterações. Portanto, a complexidade total do algoritmo é:

$$O(n \log n)$$

Podemos obter uma solução mais eficiente utilizando uma **Fila**. A ideia consiste em gerar as representações binárias de forma incremental, explorando o fato de que:

Se x está em binário, então $x0$ e $x1$ também são binários válidos.

```
gera_binarios(n) {  
    Q = Queue()  
    enqueue(Q, "1")    // primeiro número binário  
    while n > 0 {  
        n = n - 1  
        s = dequeue(Q)  
        print(s)  
        enqueue(Q, s + "0")    // adiciona zero a direita (2*n)  
        enqueue(Q, s + "1")    // adiciona um a direita (2*n + 1)  
    }  
}
```

Inicialmente:

$$Q = [1]$$

1. Remove 1 → imprime 1 → insere 10 e 11
2. Remove 10 → imprime 10 → insere 100 e 101
3. Remove 11 → imprime 11 → insere 110 e 111
4. Remove 100 → imprime 100
5. Remove 101 → imprime 101

Os cinco primeiros números binários gerados são:

1, 10, 11, 100, 101

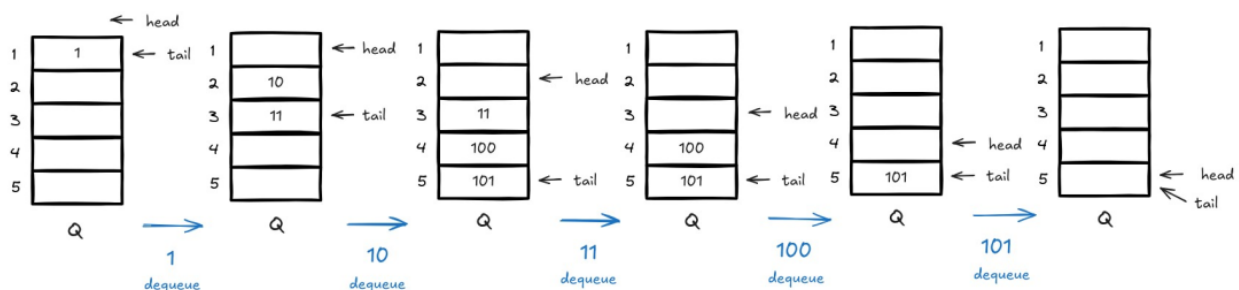


Figura 15: Ilustração da simulação do algoritmo para gerar números binários utilizando uma Fila.

As operações `enqueue()` e `dequeue()` possuem complexidade $O(1)$. O laço principal executa exatamente n vezes. Em cada iteração realizamos:

- uma operação `dequeue()`;
- duas operações `enqueue()`.

Portanto, o número total de operações é proporcional a $3n$, o que implica:

$$O(n)$$

Assim, essa abordagem é assintoticamente mais eficiente que a solução anterior, que possui complexidade $O(n \log n)$. A utilização de uma Fila permite explorar a estrutura incremental dos números binários, reduzindo a complexidade temporal do algoritmo de $O(n \log n)$ para $O(n)$, tornando esta abordagem mais eficiente para gerar os n primeiros números binários.

4.3 Deque: Double-Ended Queue (Fila de Duas Extremidades)

Deque (pronuncia-se *deck*, para diferenciar da operação *dequeue*), ou *Double-Ended Queue*, que traduziremos como **Dupla Fila**, é uma estrutura de dados linear bastante similar a uma fila tradicional, porém com dois inícios e dois finais. O que a torna diferente da fila convencional é justamente a possibilidade de inserir e remover elementos tanto do início quanto do fim da estrutura. Nesse sentido, esse TAD híbrido provê todas as funcionalidades de filas e pilhas em uma única estrutura de dados. Para fixar a notação, denotaremos por *start* a parte da frente da fila (lado esquerdo) e por *end* a parte de trás da fila (lado direito), conforme ilustrado a seguir:

$$\text{start} \rightarrow [1, 2, 3, 4, 5] \leftarrow \text{end}$$

As principais operações de um Deque são listadas a seguir.

- `d.is_empty()` — verifica se o deque está vazio.
- `d.add_start(x)` — insere o elemento x no início.
- `d.add_end(x)` — insere o elemento x no final.
- `d.remove_start()` — remove e retorna o elemento do início.
- `d.remove_end()` — remove e retorna o elemento do final.
- `d.size()` — retorna o número de elementos armazenados.

A tabela a seguir apresenta uma sequência de operações realizadas sobre um deque inicialmente vazio:

Operação	Conteúdo	Retorno	Descrição
<code>d.is_empty()</code>	<code>[]</code>	True	Verifica se o deque está vazio
<code>d.add_start(4)</code>	<code>[4]</code>		Insere elemento na esquerda
<code>d.add_start(7)</code>	<code>[7, 4]</code>		Insere elemento na esquerda
<code>d.add_end(2)</code>	<code>[7, 4, 2]</code>		Insere elemento na direita
<code>d.add_end(5)</code>	<code>[7, 4, 2, 5]</code>		Insere elemento na direita
<code>d.size()</code>	<code>[7, 4, 2, 5]</code>	4	Retorna número de elementos
<code>d.is_empty()</code>	<code>[7, 4, 2, 5]</code>	False	Verifica se o deque está vazio
<code>d.add_start(8)</code>	<code>[8, 7, 4, 2, 5]</code>		Insere elemento na esquerda
<code>d.remove_end()</code>	<code>[8, 7, 4, 2]</code>	5	Remove elemento da direita
<code>d.remove_start()</code>	<code>[7, 4, 2]</code>	8	Remove elemento da esquerda
<code>d.size()</code>	<code>[7, 4, 2]</code>	3	Retorna número de elementos

Observa-se que o deque combina o comportamento de uma fila (inserção no final e remoção no início) com o de uma pilha (inserção e remoção na mesma extremidade), oferecendo maior flexibilidade estrutural. Em implementações eficientes baseadas em listas duplamente encadeadas ou vetores circulares, todas as operações básicas possuem complexidade $O(1)$.

4.3.1 Um TAD Deque

A seguir apresentamos uma especificação de TAD para a estrutura de dados *Deque*, implementada por meio de um vetor circular.

```
TAD Deque
    int keys[1..n]
    int start
    int end
    int size
```

onde:

- n é o tamanho máximo do deque;
- `start` indica a extremidade esquerda;
- `end` indica a extremidade direita;
- `size` armazena o número atual de elementos.

```
init(D) {
    D.start = 0
    D.end   = 0
    D.size  = 0
}
```

```

// Verifica se Deque está cheio
is_full(D) {
    if D.size == n
        return True
    else
        return False
}

// Verifica se Deque está vazio
is_empty(D) {
    if D.size == 0
        return True
    else
        return False
}

// Adiciona um novo nó no início
add_start(D, key) {
    if is_full(D)
        error('overflow')
    else {
        if is_empty(D) {
            D.start = 1
            D.end = 1
        }
        else {
            if D.start == 1
                D.start = n
            else
                D.start = D.start - 1
        }
        D.keys[D.start] = key
        D.size = D.size + 1
    }
}

// Adiciona um novo nó no final
add_end(D, key) {
    if is_full(D)
        error('overflow')

```

```

else {
    if is_empty(D) {
        D.start = 1
        D.end   = 1
    }
    else {
        if D.end == n
            D.end = 1
        else
            D.end = D.end + 1
    }
    D.keys[D.end] = key
    D.size = D.size + 1
}

}

// Remove o nó do início
remove_start(D) {
    if is_empty(D)
        error('underflow')
    else {
        key = D.keys[D.start]
        D.keys[D.start] = NULL
        if D.start == D.end {
            D.start = 0
            D.end   = 0
        }
        else {
            if D.start == n
                D.start = 1
            else
                D.start = D.start + 1
        }
        D.size = D.size - 1
        return key
    }
}

// Remove o nó do final
remove_end(D) {
    if is_empty(D)

```

```

        error('underflow')
    else {
        key = D.keys[D.end]
        D.keys[D.end] = NULL
        if D.start == D.end {
            D.start = 0
            D.end = 0
        }
        else {
            if D.end == 1
                D.end = n
            else
                D.end = D.end - 1
        }
        D.size = D.size - 1
        return key
    }
}

```

Um aspecto importante dessa implementação é que tanto as operações de inserção quanto as de remoção realizam apenas um número constante de operações aritméticas e atribuições, independentemente do tamanho n do vetor. Portanto, as operações `add_start`, `add_end`, `remove_start` e `remove_end` possuem complexidade temporal $O(1)$.

4.3.2 Aplicação do Deque: Verificação de Palíndromos

Um exemplo clássico de aplicação de uma estrutura linear do tipo *Deque* é o problema de verificar se uma dada palavra é palíndroma, isto é, se ela é igual à sua versão reversa. Formalmente, uma palavra é dita palíndroma quando sua sequência de caracteres lida da esquerda para a direita é idêntica à sequência lida da direita para a esquerda. Um exemplo simples é a palavra RADAR.

A ideia para resolver esse problema utilizando um *Deque* consiste em armazenar os caracteres da string de entrada e, em seguida, comparar simetricamente os elementos das duas extremidades. O procedimento pode ser descrito da seguinte forma:

1. Percorrer a string de entrada da esquerda para a direita, inserindo cada caractere no final (*rear*) do deque.
2. Enquanto houver mais de um elemento armazenado, remover simultaneamente um caractere do início (*front*) e outro do final (*rear*).
3. Comparar os dois caracteres removidos:
 - Se forem diferentes, a palavra não é palíndroma.

- Se forem iguais, o processo continua.

4. O algoritmo termina quando:

- o deque ficar vazio (caso de palavra com número par de caracteres), ou
- restar apenas um caractere (caso de palavra com número ímpar de caracteres).

A seguir apresentamos o algoritmo correspondente.

```
// Função que verifica se uma palavra é palíndroma
palindroma(palavra) {
    // Cria Deque
    D = Deque()
    K = len(palavra)
    // Percorre string de entrada
    for i = 1 to K
        add_rear(D, palavra[i])
    // Enquanto houver mais de 1 letra
    while D.size > 1 {
        esquerda = remove_rear(D)
        direita = remove_front(D)
        if esquerda != direita
            return False
    }
    return True
}
```

Seja K o número de caracteres da palavra. A inserção inicial realiza K operações, cada uma com custo $O(1)$. Em seguida, no pior caso, realizamos aproximadamente $K/2$ iterações de remoção e comparação, também com custo constante por operação. Portanto, a complexidade temporal do algoritmo é $O(K)$, sendo linear no tamanho da palavra de entrada.

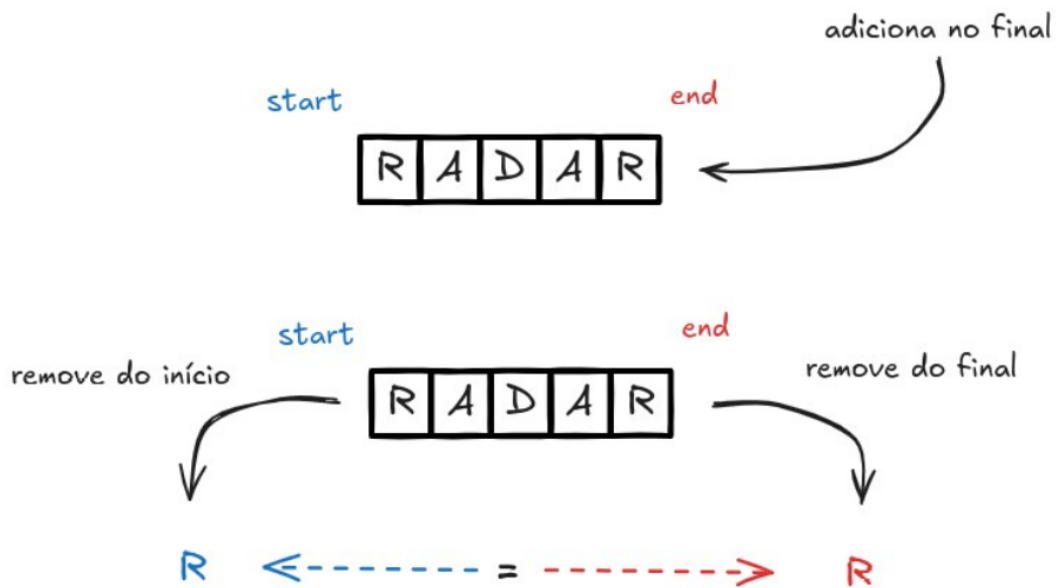


Figura 16: Ilustração do uso de uma estrutura de dados Deque na detecção de uma palavra palíndroma.

4.3.3 Problema: Sliding Window Maximum

Dado um array L de tamanho n e um valor inteiro k , o problema *Sliding Window Maximum* consiste em encontrar o maior elemento de cada subarray contíguo de tamanho k . Em outras palavras, deseja-se determinar o elemento máximo em cada janela deslizante de tamanho k .

Entrada: $L = [1, 2, 3, 1, 4, 5]$, $k = 3$

Saída: 3 3 4 5

Explicação:

- Máximo de $(1, 2, 3)$ é 3;
- Máximo de $(2, 3, 1)$ é 3;
- Máximo de $(3, 1, 4)$ é 4;
- Máximo de $(1, 4, 5)$ é 5.

A primeira solução apresentada é a abordagem direta, que consiste em calcular o máximo de cada janela de forma independente.

```
// Sliding Window Maximum v.1
sliding_window_maximum(L, n, k) {
  for i = 0 to n-k {
    max = L[i]
    for j = 1 to k-1 {
      if L[i + j] > max
        max = L[i + j]
    }
  }
}
```

```
    }
    print(max)
}
}
```

O laço externo executa $(n - k + 1)$ vezes e, para cada iteração, o laço interno executa k vezes. Assim, a complexidade temporal é

$$O(nk).$$

Podemos melhorar significativamente essa eficiência utilizando um *Deque*.

Solução Otimizada com Deque

A ideia do algoritmo consiste em manter um deque D de capacidade k , armazenando apenas os elementos úteis da janela atual. Um elemento é considerado útil se:

- pertence à janela atual, e
- é maior do que todos os elementos à sua direita dentro da janela.

O deque armazena **índices** dos elementos do array e mantém os valores correspondentes em ordem decrescente. Assim:

- O elemento na frente do deque é o maior da janela;
- O elemento na parte traseira é o menor da janela.

Sendo assim, podemos desenvolver o seguinte algoritmo:

1. Criar um deque D .
2. Processar os primeiros k elementos:
 - Remover do final todos os elementos menores que o elemento atual;
 - Inserir o índice do elemento atual no final.
3. Para cada elemento do índice k até $n - 1$:
 - Imprimir o elemento na frente do deque;
 - Remover elementos fora da janela (índices $\leq i - k$);
 - Remover do final todos os elementos menores que o atual;
 - Inserir o índice atual;
4. Ao final, imprimir o máximo da última janela.

```

// Sliding Window Maximum v.2
sliding_window_maximum(L, n, k) {
    D = Deque()
    // Processa primeira janela
    for i = 0 to k-1 {
        while not is_empty(D) and L[i] >= L[D.end]
            remove_rear(D)
        add_rear(D, i)
    }
    // Processa restante do array
    for i = k to n-1 {
        print(L[D.start])
        // Remove elementos fora da janela
        while not is_empty(D) and D.start <= i-k
            remove_front(D)
        while not is_empty(D) and L[i] >= L[D.end]
            remove_rear(D)
        add_rear(D, i)
    }
    print(L[D.start])
}

```

A análise da complexidade baseia-se nas seguintes observações:

1. As operações `add_rear`, `remove_rear` e `remove_front` possuem custo $O(1)$.
2. Cada elemento do array é inserido no deque no máximo uma vez.
3. Cada elemento é removido do deque no máximo uma vez.
4. O primeiro laço executa k vezes e o segundo executa $(n - k)$ vezes, totalizando n iterações.

Assim, temos no máximo $2n$ operações de custo constante, o que implica:

$$O(n).$$

Portanto, essa solução é assintoticamente mais eficiente que a abordagem trivial $O(nk)$, em especial quando a janela é grande.

Exercício: Aplique o algoritmo `sliding_window_maximum` ao array

$$L = [1, 2, 3, 1, 7, 8]$$

considerando o tamanho da janela $k = 3$. Acompanhe passo a passo a evolução do deque D a cada iteração e determine os valores máximos produzidos.

$L = [1, 2, 3, 1, 7, 8]$
 $K = 3$

Ideia: sempre manter o índice do maior no início do Deque

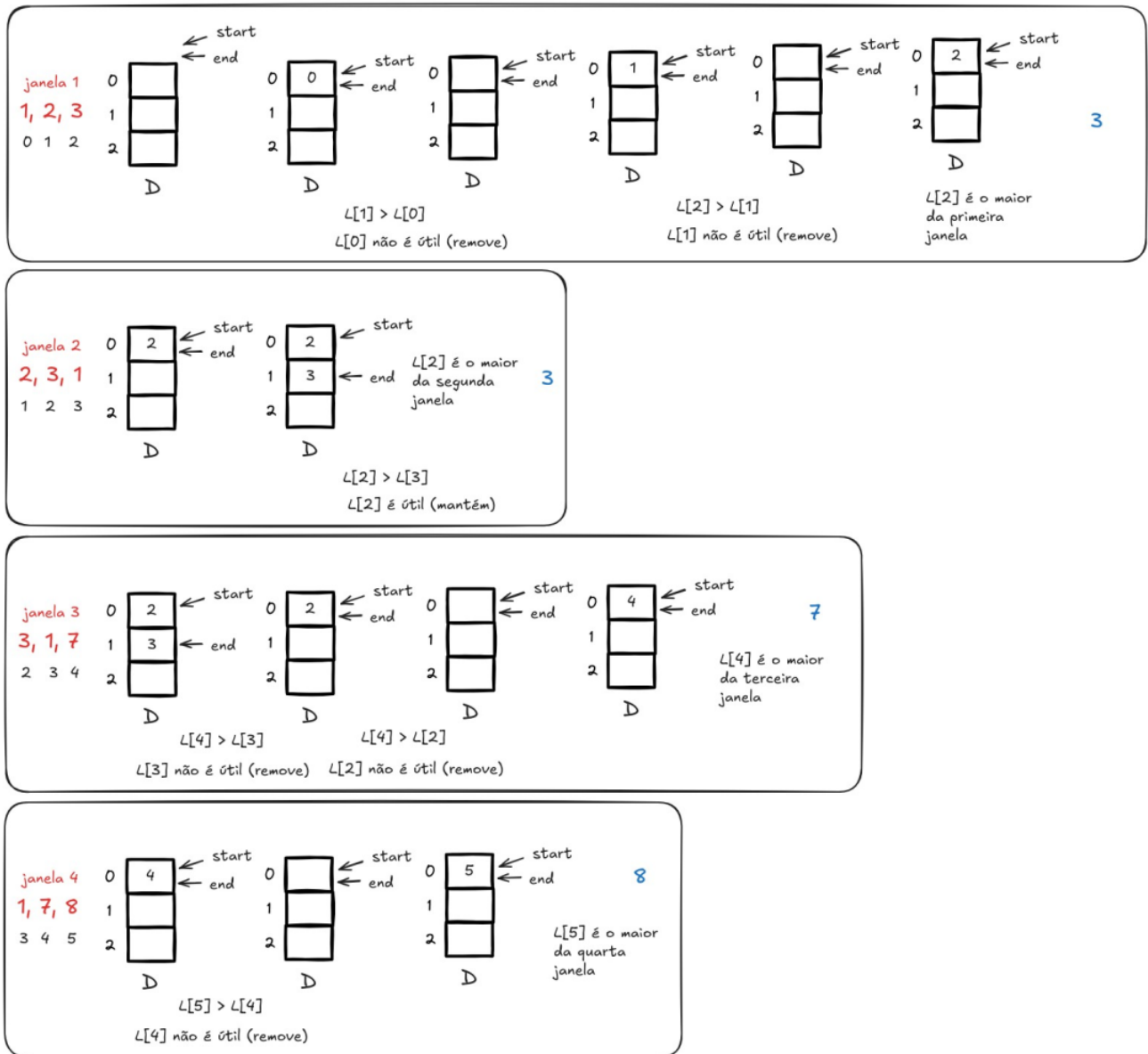


Figura 17: Ilustração do uso de uma estrutura de dados Deque na resolução do problema *sliding window maximum*.

4.4 Fila de Prioridades

Uma fila de prioridades é uma extensão da fila tradicional na qual, para cada elemento inserido, deve-se associar um valor de prioridade p . Por convenção, utilizam-se inteiros positivos para representar a prioridade, sendo que quanto menor o valor inteiro, maior a prioridade. Assim, um elemento com $p = 1$ possui prioridade sobre um elemento com $p = 2$.

A principal diferença em relação à fila tradicional não está na operação `enqueue()`, pois o elemento continua sendo inserido no final da estrutura, mas sim na operação `dequeue()`, já que a remoção não ocorre necessariamente no início da fila. Em vez disso, é necessário localizar o elemento com maior prioridade e removê-lo.

```
TAD Node
    int key
    int priority

TAD PriorityQueue
    Node data[1..n]
    int tail
    int size

init(PQ) {
    PQ.tail = 0
    PQ.size = 0
}

// Adiciona chave com prioridade p no final da fila
enqueue(PQ, key, p) {
    if PQ.size == n
        error('overflow')
    else {
        PQ.tail += 1
        PQ.data[PQ.tail].key = key
        PQ.data[PQ.tail].priority = p
        PQ.size += 1
    }
}

// Encontra elemento de maior prioridade
findMin(PQ) {
    index = 1
    min = PQ.data[index].priority
```

```

    for i = 1 to PQ.tail {
        if PQ.data[i].priority < min {
            min = PQ.data[i].priority
            index = i
        }
    }
    return index
}

// Remove chave de maior prioridade
dequeue(PQ) {
    if PQ.size == 0
        error('underflow')
    else {
        pos = findMax(PQ)
        key = PQ.data[pos].key
        # Realoca elementos
        for i = pos to PQ.tail - 1 {
            PQ.data[i].key = PQ.data[i+1].key
            PQ.data[i].priority = PQ.data[i+1].priority
        }
        PQ.tail -= 1
        PQ.size -= 1
    }
    return key
}

```

Nesta implementação ingênua baseada em array simples, a função `findMax()` percorre todos os elementos da fila, resultando em custo $O(n)$. Além disso, a realocação dos elementos após a remoção também possui custo $O(n)$ no pior caso. Portanto, a operação `dequeue()` apresenta complexidade temporal $O(n)$. Posteriormente, veremos que, ao utilizar uma estrutura do tipo *heap* (por exemplo, um *binary heap*), é possível reduzir a complexidade da operação de remoção para $O(\log n)$.

4.4.1 Problema: Disk Tower

Sua tarefa é construir uma torre ao longo de n dias, obedecendo às seguintes condições:

- A cada dia você recebe um disco de tamanho distinto no intervalo de 1 até n ;
- O disco de maior tamanho deve ser colocado na base da torre;

- O disco de menor tamanho deve ser colocado no topo da torre;
- Um novo disco só pode ser colocado na torre quando todos os discos de tamanho maior que ele já tiverem sido colocados.

Deve-se imprimir n linhas, indicando quais discos podem ser colocados na torre no i -ésimo dia.

- Primeira linha: inteiro n , número total de discos.
- Segunda linha: n inteiros, onde o i -ésimo inteiro representa o tamanho do disco recebido no i -ésimo dia.

Observação: Todos os tamanhos são distintos e pertencem ao intervalo $1 \leq \text{tamanho} \leq n$.

Na saída, você deve imprimir n linhas. Na i -ésima linha, imprimir os tamanhos dos discos que podem ser colocados na torre naquele dia, em ordem decrescente. Caso nenhum disco possa ser colocado, imprimir uma linha vazia (representada por “-” no exemplo). Como restrição, temos que o número de discos deve ser limitado a $1 < n < 10^6$.

Entrada

```
5
4 5 1 2 3
```

Saída

```
-
5 4
-
-
3 2 1
```

1. No primeiro dia, recebe-se o disco de tamanho 4. Ele não pode ser colocado, pois o disco 5 ainda não foi recebido.
2. No segundo dia, recebe-se o disco 5. Agora os discos 5 e 4 podem ser colocados.
3. No terceiro e quarto dias, os discos ainda não podem ser colocados, pois o disco 3 ainda não foi recebido.
4. No quinto dia, finalmente podem ser colocados os discos 3, 2 e 1.

Uma solução consiste em utilizar uma fila de prioridades na qual os campos `key` e `priority` são idênticos. Como desejamos remover sempre o maior elemento disponível, utilizamos a função `findMax()`.

```

// Verifica último elemento inserido
peek(PQ) {
    return PQ.data[PQ.tail].key
}

disk_tower(L, n) {
    max = n
    PQ = PriorityQueue()
    for i = 0 to n-1 {
        enqueue(PQ, L[i], L[i])
        while PQ.size > 0 and findMax(PQ) == max {
            disk = dequeue(PQ)
            print(disk, " ")    // imprime na mesma linha
            max -= 1
        }
        print("\n")
    }
}

```

Cada elemento é inserido exatamente uma vez na fila e removido exatamente uma vez. Assim:

$$\text{Custo de enqueue} = O(1)$$

$$\text{Custo de dequeue} = O(n)$$

Logo, o custo total é:

$$T(n) = n \cdot O(1) + n \cdot O(n) = O(n^2).$$

Posteriormente, ao implementar a fila de prioridades com um *heap binário* (min-heap ou max-heap), é possível reduzir o custo da operação de remoção para:

$$O(\log n),$$

resultando em complexidade total $O(n \log n)$. Nesta aula foram estudadas as principais estruturas lineares estáticas: Pilha, Fila, Deque e Fila de Prioridades. Na próxima aula será introduzida outra estrutura linear fundamental para a computação: as **listas encadeadas**. Diferentemente dos vetores tradicionais, nos quais os elementos são armazenados de forma contígua na memória, em uma lista encadeada cada nó é alocado independentemente, e a ligação entre os elementos é realizada por meio de referências (ponteiros), formando um encadeamento lógico.

4.5 Considerações Finais

Neste capítulo foram apresentadas as principais estruturas de dados lineares estáticas: Pilhas, Filas, Deques e Filas de Prioridades. Embora todas compartilhem a característica fundamental de organizar elementos de forma sequencial, cada uma impõe uma política específica de acesso e manipulação, o que impacta diretamente o tipo de problema para o qual é mais adequada.

As pilhas, regidas pela disciplina LIFO (*Last-In, First-Out*), mostraram-se particularmente úteis em problemas que envolvem retrocesso, avaliação de expressões e controle de chamadas de funções. As filas, por sua vez, seguem a política FIFO (*First-In, First-Out*), sendo amplamente empregadas em escalonamento de processos, simulações e processamento em ordem de chegada. O deque surge como uma generalização dessas duas estruturas, oferecendo maior flexibilidade ao permitir inserções e remoções em ambas as extremidades, sem perda de eficiência assintótica.

Já as filas de prioridades introduzem um critério adicional de ordenação baseado em prioridade, rompendo com a ordem estritamente temporal de inserção. Essa estrutura é essencial em algoritmos clássicos de otimização e grafos, como os métodos de caminho mínimo e escalonamento guloso. Observou-se, ainda, que a escolha da implementação influencia diretamente a complexidade das operações: enquanto uma implementação simples baseada em vetor pode levar a custo $O(n)$ para remoção, o uso de *heaps* permite reduzir esse custo para $O(\log n)$.

Do ponto de vista analítico, um aspecto central discutido ao longo do capítulo foi a relação entre projeto de estrutura de dados e complexidade computacional. Pequenas modificações na organização interna dos elementos podem produzir ganhos significativos de desempenho. Assim, a seleção apropriada da estrutura não deve ser guiada apenas pela funcionalidade desejada, mas também pelas restrições de eficiência impostas pelo problema.

Essas estruturas lineares constituem a base para o estudo de estruturas mais sofisticadas, como listas encadeadas, árvores e grafos. O domínio conceitual e operacional desses TADs é, portanto, fundamental para a construção de algoritmos eficientes e para a compreensão da organização interna de sistemas computacionais.

5 Estruturas de dados lineares dinâmicas

Nos capítulos anteriores estudamos estruturas lineares estáticas, implementadas tipicamente por meio de vetores. Nessas estruturas, a quantidade máxima de elementos deve ser previamente definida, e o armazenamento ocorre de forma contígua na memória. Embora essa abordagem seja simples e eficiente para diversas aplicações, ela apresenta limitações importantes, especialmente quando o número de elementos não é conhecido antecipadamente ou varia significativamente ao longo da execução do programa.

As estruturas de dados lineares dinâmicas surgem justamente para superar essas limitações. Diferentemente dos vetores tradicionais, nessas estruturas os elementos não precisam ocupar posições contíguas de memória. Cada elemento é alocado individualmente e conectado aos demais por meio de referências (ou ponteiros), formando um encadeamento lógico. Essa característica permite que a estrutura cresça ou diminua conforme necessário, utilizando memória de maneira mais flexível.

Entre as principais estruturas lineares dinâmicas destacam-se as listas encadeadas e as listas duplamente encadeadas. Na lista simplesmente encadeada, cada nó armazena um valor e uma referência para o próximo elemento da sequência. Já na lista duplamente encadeada, cada nó mantém duas referências: uma para o próximo elemento e outra para o elemento anterior, permitindo navegação bidirecional.

O estudo dessas estruturas é fundamental para compreender conceitos centrais como alocação dinâmica de memória, manipulação de ponteiros e custo de operações de inserção e remoção. Além disso, listas encadeadas servem como base para a implementação de diversas outras estruturas mais complexas, como pilhas, filas, tabelas de dispersão e árvores.

Neste capítulo, analisaremos detalhadamente a organização interna dessas estruturas, suas operações fundamentais e a complexidade associada a cada uma delas, enfatizando as diferenças conceituais e práticas em relação às estruturas lineares estáticas.

5.1 Listas encadeadas

Em programação de computadores, é possível utilizar estruturas de dados de maneira dinâmica, isto é, projetar estruturas que podem crescer ou diminuir de tamanho em tempo de execução, seja por meio da inserção ou da remoção de elementos. Essa característica é especialmente importante em situações nas quais o número de dados a ser armazenado não é conhecido previamente ou pode variar significativamente ao longo da execução do programa.

Para permitir esse tipo de flexibilidade, diversas linguagens de programação oferecem mecanismos de alocação dinâmica de memória, possibilitando que novos blocos de memória sejam requisitados ou liberados conforme a necessidade da aplicação. Diferentemente das estruturas baseadas em vetores estáticos, nas quais o espaço deve ser previamente reservado de forma contígua, as estruturas dinâmicas exploram o conceito de referências (ou ponteiros) para conectar elementos armazenados em posições arbitrárias da memória.

Nesse contexto, as listas encadeadas constituem um exemplo clássico de estrutura dinâmica. Elas

utilizam um esquema de encadeamentos lógicos sequenciais, no qual cada elemento (nó) mantém uma referência para o próximo elemento da sequência. Dessa forma, elementos vizinhos na estrutura lógica não precisam ocupar posições contíguas na memória física, como ilustrado na figura a seguir.

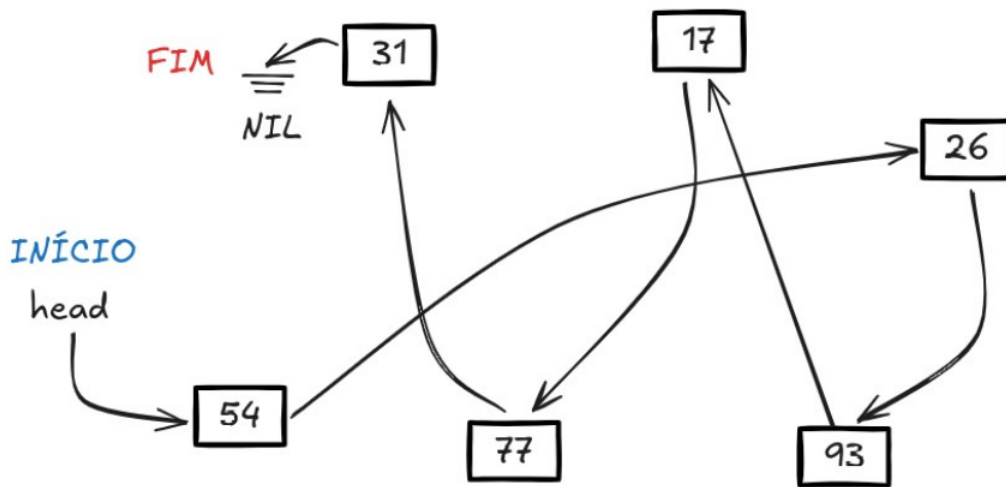


Figura 18: Ilustração do uso de encadeamento lógico de maneira linear.

Primeiramente, antes de definirmos formalmente uma lista encadeada, é necessário especificar seu bloco fundamental de construção: o **nó**. Cada nó de uma lista encadeada deve conter duas informações essenciais: o dado propriamente dito e uma referência para o próximo nó da lista, isto é, o elemento para o qual ele aponta no encadeamento lógico. Podemos definir um TAD `Node`, que representa um nó da lista, da seguinte forma:

TAD Node	
<code>int key</code>	# armazena a chave (dado)
<code>Node next</code>	# referência para o próximo nó

Internamente, ao criarmos um nó, sua representação típica consiste em um bloco de memória dividido em duas partes: uma destinada ao armazenamento da chave e outra destinada ao ponteiro (ou referência) para o próximo nó da sequência.

Ao contrário das estruturas estáticas, como vetores, as estruturas dinâmicas não possuem limitação prévia de tamanho. Em princípio, é possível inserir elementos indefinidamente, até que toda a memória disponível do sistema seja utilizada. Essa característica confere grande flexibilidade às estruturas dinâmicas, tornando-as adequadas para aplicações em que o volume de dados é variável ou desconhecido.

A estrutura linear dinâmica mais simples e fundamental é a **lista encadeada não ordenada**, que será estudada a seguir, juntamente com suas operações básicas de inserção, remoção e busca.

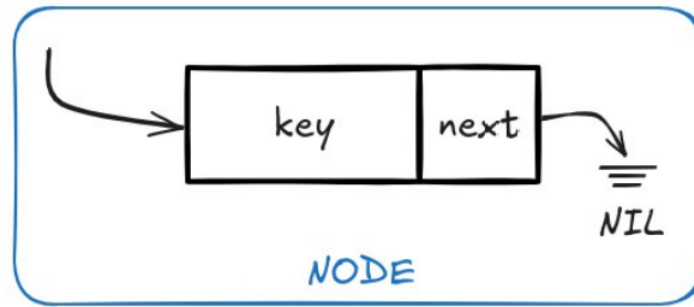


Figura 19: Ilustração do da estrutura básica Node, que define um nó em uma lista encadeada simpels.

5.1.1 Listas Encadeadas Simples

Em uma lista encadeada não ordenada, a principal característica é que a inserção de novos nós é realizada tipicamente no início (ou eventualmente no final) da lista, sem que haja qualquer critério de ordenação entre os elementos armazenados. Assim, os elementos do conjunto não se encontram organizados segundo uma relação de ordem.

Inicialmente, apresentaremos a criação de uma lista encadeada vazia. Adotaremos o seguinte TAD, no qual `head` (cabeça) é uma referência para o primeiro nó da lista:

```
TAD Linked_List
    Node head
    int size
```

Durante a criação da lista encadeada L , devemos fazer com que a cabeça da lista referencie `NIL`, indicando que a estrutura está vazia:

```
init(L) {
    L.head = NIL
    L.size = 0
}
```

A operação mais básica de um TAD lista encadeada é aquela responsável por adicionar um elemento no início da lista. A lógica consiste em:

- Criar um novo nó;
- Fazer o campo `next` desse novo nó apontar para o nó atualmente referenciado por `head`;
- Atualizar `head` para apontar para o novo nó.

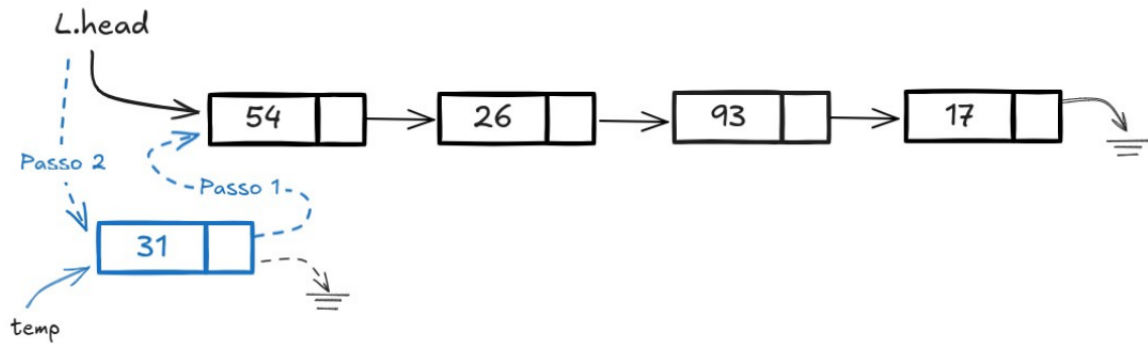


Figura 20: Ilustração da inserção de um novo nó no início de uma lista encadeada simples.

Dessa forma, o elemento recém-inserido passa a ocupar a primeira posição da lista. O algoritmo a seguir implementa essa funcionalidade para uma lista encadeada L :

```
add_head(L, key) {
    // Cria novo nó
    temp = Node(key)
    // Aponta novo nó para a cabeça da lista
    temp.next = L.head
    // Atualiza a cabeça da lista
    L.head = temp
    L.size += 1
}
```

Observe que a inserção no início de uma lista encadeada envolve apenas um número constante de operações de atribuição. Portanto, sua complexidade temporal é $O(1)$. De modo análogo, podemos realizar a inserção no final da lista. Para isso, devemos criar um novo nó e posicioná-lo após o último elemento da estrutura.

Como, na implementação básica, a lista encadeada possui apenas a referência `head`, é necessário percorrer toda a lista até encontrar o último nó, isto é, aquele cujo campo `next` é igual a `NIL`. Esse nó representa o final da lista.

O procedimento consiste em:

- Criar um novo nó;
- Caso a lista esteja vazia (`head == NIL`), fazer a cabeça apontar para o novo nó;
- Caso contrário, utilizar uma referência temporária para percorrer a lista a partir da cabeça;
- Avançar sucessivamente até encontrar o nó cujo `next == NIL`;
- Fazer o último nó apontar para o novo nó.

Ao percorrermos uma lista encadeada, devemos sempre iniciar na cabeça (`head`) e, a cada iteração, atualizar a referência para seu sucessor por meio do campo `next`. A figura correspondente ilustra esse processo de varredura sequencial.

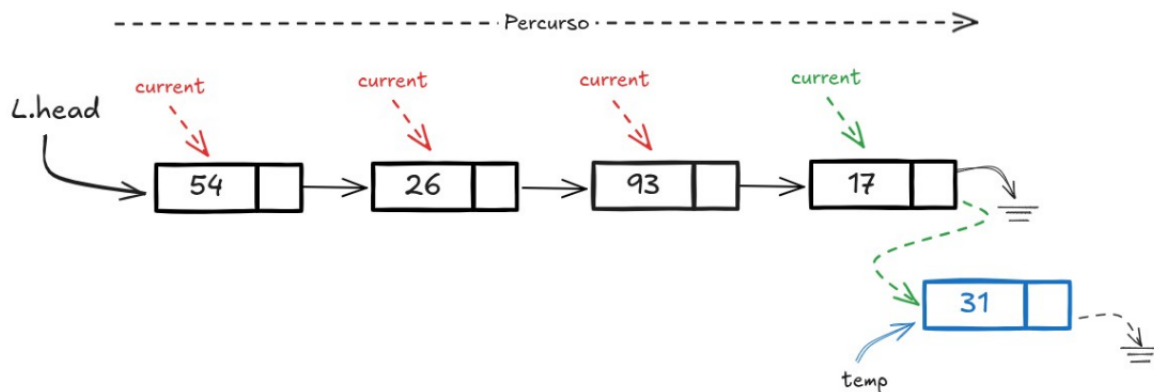


Figura 21: Ilustração da inserção de um novo nó no final de uma lista encadeada simples.

O algoritmo a seguir implementa essa funcionalidade:

```
add_tail(L, key) {
    // Cria novo nó
    tail = Node(key)
    if L.head == NIL
        L.head = tail
    else {
        // Referência temporária para percorrer a lista
        temp = L.head
        // Percorre até o último elemento
        while temp.next != NIL:
            temp = temp.next
        // Faz o último nó apontar para o novo nó
        temp.next = tail
    }
    tail.next = NIL
    L.size += 1
}
```

Observe que, nessa implementação, a inserção no final da lista possui complexidade $O(n)$, pois, no pior caso, é necessário percorrer todos os nós da lista para localizar o último elemento. Essa limitação pode ser superada mantendo-se uma referência adicional para o último nó (ponteiro *tail*), permitindo inserção no final em tempo $O(1)$. Precisamos ainda implementar uma função para retornar quantos elementos existem na lista encadeada. Note que é possível realizar essa operação em complexidade $O(1)$.

```
length(L) {
    return L.size
}
```

Observe que, ao adotar a estratégia de incrementar ou decrementar o número de elementos da lista

encadeada após cada inserção ou remoção, torna-se possível obter a quantidade de elementos em L com complexidade $O(1)$.

Outra funcionalidade importante consiste em buscar um elemento na lista encadeada, ou seja, verificar se um determinado valor pertence ao conjunto. Para isso, devemos percorrer a lista sequencialmente até encontrar o elemento desejado (retornando `True`, indicando que o elemento pertence à lista), ou até atingir o final da lista (retornando `False`, indicando que o elemento não pertence ao conjunto). A figura a seguir ilustra esse processo.

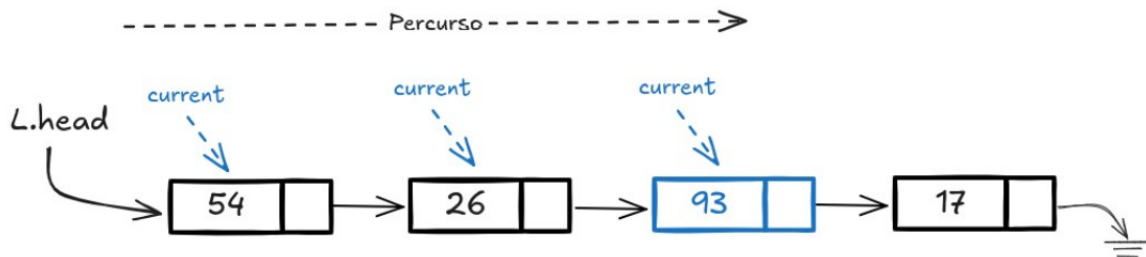


Figura 22: Ilustração da busca por um nó em uma lista encadeada simples.

O algoritmo a seguir apresenta uma possível implementação dessa funcionalidade:

```
// Busca pelo elemento na lista
search(L, key) {
    // Inicia na cabeça da lista
    temp = L.head
    // Percorre a lista até achar elemento ou chegar no final
    while temp != NIL {
        // Se o nó atual contém o elemento
        if temp.key == key
            return True
        else:
            temp = temp.next
    }

    return False
}
```

A operação de busca consiste em percorrer sequencialmente a lista a partir da referência `head`, comparando a chave de cada nó com o valor desejado. No melhor caso, o elemento procurado encontra-se na primeira posição, resultando em complexidade $O(1)$. No pior caso, o elemento está na última posição ou não pertence à lista, sendo necessário percorrer todos os n nós. Assim, a complexidade no pior caso é $O(n)$.

Por fim, uma operação fundamental é a remoção de um determinado elemento da lista encadeada. Ao remover um nó, é necessário religar corretamente o seu antecessor ao seu sucessor, de modo a preservar o encadeamento sequencial e evitar que parte da estrutura se torne inacessível.

Para isso, utilizamos duas referências que percorrem simultaneamente a lista: `current`, que aponta para o nó corrente, e `previous`, que aponta para o seu antecessor. Ambas devem avançar conjuntamente até que `current` aponte para o nó que contém a chave a ser removida. A figura a seguir ilustra esse processo.

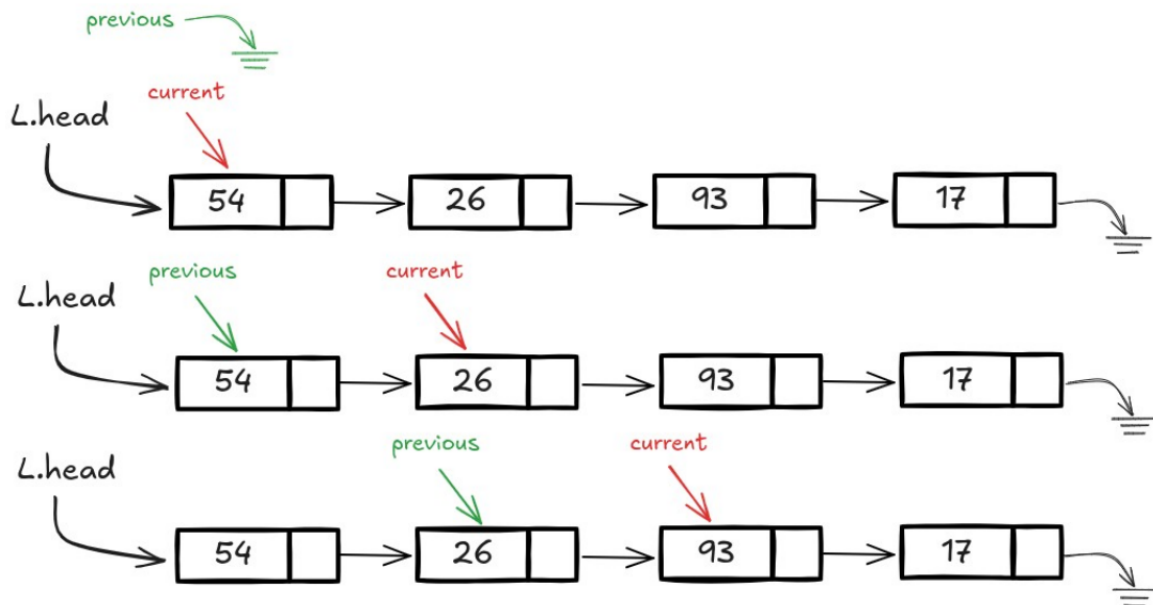


Figura 23: Durante a busca para a remoção de um nó de uma lista encadeada simples, é preciso de duas referências.

Uma vez localizado o nó, devemos atualizar o campo `next` da referência `previous`, fazendo-o apontar para o mesmo nó indicado por `current.next`. Dessa forma, o nó removido é contornado na estrutura. Em seguida, atribuímos `NIL` ao campo `next` do nó corrente, desligando-o completamente da lista. A figura a seguir apresenta uma ilustração gráfica desse procedimento.

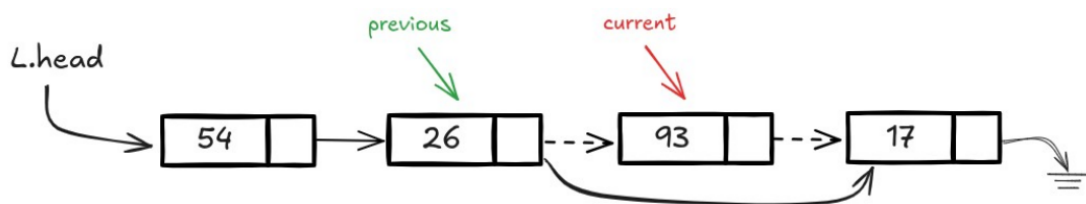


Figura 24: A referência `previous` deve apontar para o nó sucessor do nó a ser removido.

Existe, entretanto, um caso particular que deve ser tratado separadamente: quando o nó a ser removido é o primeiro da lista. Nesse cenário, `previous` permanece igual a `NIL`, e portanto devemos atualizar diretamente a referência `head` da lista, conforme ilustrado na figura correspondente.

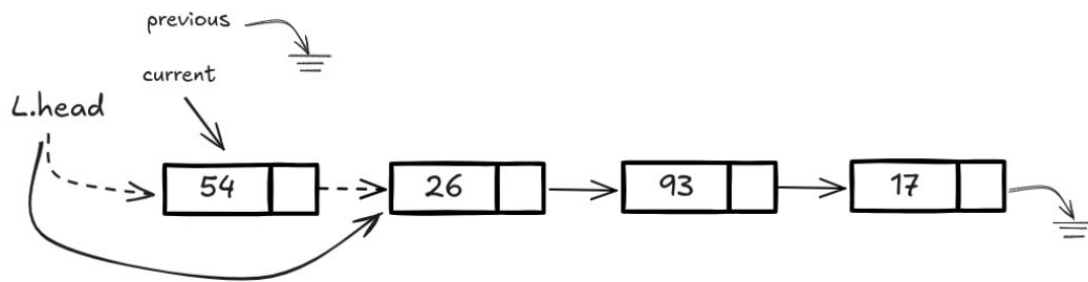


Figura 25: Na remoção da cabeça da lista, é preciso definir uma nova cabeça como sendo o nó sucessor.

O algoritmo a seguir apresenta uma possível implementação para o método `remove()`:

```
// Remove um nó da lista encadeada
remove(L, key) {
    current = L.head
    previous = NIL
    // Enquanto não encontrar o valor a ser removido
    while current != NIL {
        // Se nó corrente armazena a chave desejada
        if current.key == key
            break
        else {
            // Atualiza previous e current
            previous = current
            current = current.next
        }
    }
    // Se nó a ser removido for o primeiro da lista
    if previous == NIL
        L.head = current.next
    else
        // Liga previous ao sucessor de current
        previous.next = current.next
    // Desliga nó corrente
    current.next = NIL
}
```

A remoção de um elemento exige, primeiramente, a sua localização na lista. Portanto, há um custo associado à busca do nó a ser removido. No pior caso, é necessário percorrer todos os n elementos para encontrar o nó desejado, o que implica complexidade $O(n)$. Após a localização, a operação de religar os ponteiros é feita em tempo constante. Assim, a complexidade total da remoção é dominada pela etapa de busca, sendo $O(n)$. Se o nó a ser removido já for conhecido (isto é, se tivermos uma referência direta para ele e para seu antecessor), a operação de remoção pode ser realizada em $O(1)$.

De maneira geral, listas simplesmente encadeadas apresentam:

- Busca: $O(n)$
- Inserção no início: $O(1)$
- Inserção no final: $O(n)$ (ou $O(1)$ com ponteiro para cauda)
- Remoção: $O(n)$

Essas propriedades tornam as listas encadeadas adequadas para cenários com inserções frequentes no início da estrutura, mas menos eficientes quando há necessidade de acesso aleatório ou buscas recorrentes.

5.1.2 Pilhas com Estruturas Dinâmicas

É possível implementar uma pilha de forma dinâmica, utilizando uma lista encadeada simples como estrutura subjacente. Para isso, definimos o Tipo Abstrato de Dados `Dynamic_Stack`.

```
TAD Dynamic_Stack
    Node top
    int size    // número de elementos presentes na pilha
```

Durante a criação da pilha *DS*, devemos inicializar a referência `top` com `NIL`, indicando que a pilha está vazia, e definir o tamanho como zero.

```
init(DS) {
    DS.top = NIL
    DS.size = 0
}
```

A verificação se a pilha está vazia pode ser realizada consultando o atributo `size`:

```
is_empty(DS) {
    if DS.size == 0
        return True
    else
        return False
}
```

A operação `peek()` retorna o elemento presente no topo da pilha, sem removê-lo:

```
peek(DS) {
    return DS.top.key
}
```

A inserção de um novo elemento (`push`) é realizada no início do encadeamento:

```
// Insere elemento no início do encadeamento
push(DS, key) {
    temp = Node(key)
    temp.next = DS.top
    DS.top = temp
    DS.size += 1
    return DS.top
}
```

A remoção de um elemento (pop) também ocorre no início do encadeamento:

```
// Remove elemento no início do encadeamento
pop(DS) {
    if DS.size == 0
        error('underflow')
    else {
        key = DS.top.key
        temp = DS.top
        DS.top = DS.top.next
        temp.next = NIL
        DS.size -= 1
        return key
    }
}
```

Observe que, assim como na implementação estática baseada em vetor, as operações push e pop na versão dinâmica possuem complexidade assintótica $O(1)$, pois envolvem apenas atualizações de ponteiros e do contador de elementos, independentemente do número de elementos armazenados na pilha.

5.1.3 Filas com Estruturas Dinâmicas

Assim como no caso das pilhas, é possível implementar uma fila de maneira dinâmica utilizando listas encadeadas. Para isso, definimos o Tipo Abstrato de Dados `Dynamic_Queue`.

```
TAD Dynamic_Queue
    Node head
    int size // número de elementos presentes na fila
```

Durante a criação da fila DQ , devemos inicializar a referência head com NIL, indicando que a fila está vazia, e definir o tamanho como zero.

```
init(DQ) {
    DQ.head = NIL
```

```
DQ.size = 0
}
```

A verificação se a fila está vazia pode ser realizada consultando o atributo `size`:

```
is_empty(DQ) {
    if DQ.size == 0
        return True
    else
        return False
}
```

A operação `enqueue()` insere um novo elemento no final do encadeamento, isto é, na cauda da fila:

```
// Insere elemento no final do encadeamento
enqueue(DQ, key) {
    // Vai até o fim da fila (tail)
    temp = DQ.head
    while temp.next != NIL
        temp = temp.next
    new = Node(key)
    new.next = NIL
    temp.next = new
    DQ.size += 1
    return DQ.head
}
```

A operação `dequeue()` remove o elemento presente no início da fila:

```
// Remove elemento no início do encadeamento
dequeue(DQ) {
    if DQ.size == 0
        error('underflow')
    else {
        key = DQ.head.key
        temp = DQ.head
        DQ.head = DQ.head.next
        temp.next = NIL
        DQ.size -= 1
    }
    return key
}
```

Observe que, nessa implementação, a operação `dequeue()` possui complexidade $O(1)$, pois envolve apenas a atualização da referência `head`. Entretanto, a operação `enqueue()` possui complexidade $O(n)$, uma vez que é necessário percorrer toda a lista até alcançar o último nó para inserir o novo elemento.

Essa limitação pode ser eliminada mantendo-se explicitamente um ponteiro adicional para a cauda da fila, permitindo que tanto `enqueue()` quanto `dequeue()` sejam executadas em tempo $O(1)$. É possível fazer com que a operação `enqueue` tenha complexidade $O(1)$. Para isso, devemos alterar a definição do TAD `Dynamic_Queue`, de modo a incluir uma referência explícita para o último nó da lista (cauda).

```
TAD Dynamic_Queue
Node head
Node tail    // referência para o último nó da fila
int size     // número de elementos presentes na fila
```

Durante a inicialização da fila, tanto a referência para a cabeça quanto para a cauda são vazias:

```
init(DQ) {
    DQ.head = NIL
    DQ.tail = NIL
    DQ.size = 0
}
```

Com essa modificação estrutural, a inserção no final do encadeamento pode ser realizada em tempo constante, pois não é mais necessário percorrer toda a lista.

```
// Insere elemento no final do encadeamento em O(1)
enqueue(DQ, key) {
    if DQ.head == NIL {
        new = Node(key)
        new.next = NIL
        DQ.head = new
        DQ.tail = new
    }
    else {
        // Não precisa percorrer toda a estrutura
        new = Node(key)
        new.next = NIL
        DQ.tail.next = new
        DQ.tail = new
    }
    DQ.size += 1
    return DQ.tail
}
```

Dessa forma, a operação `enqueue()` passa a ter complexidade assintótica $O(1)$, pois envolve apenas a atualização de ponteiros, independentemente do número de elementos armazenados na fila.

5.2 Listas Encadeadas Ordenadas

Quando trabalhamos com listas encadeadas ordenadas, dois métodos precisam ser ajustados em relação à versão não ordenada: `search()` e `add()`. Na inserção de um novo nó, devemos primeiramente localizar sua posição correta na estrutura, de modo a preservar a ordenação. Já na operação de busca, não é necessário percorrer toda a lista, pois, se o elemento atual for maior que o valor procurado e ainda não o tivermos encontrado, podemos concluir que ele não pertence à lista.

A seguir, apresentamos uma implementação da função que verifica se um elemento pertence a uma lista ordenada.

```
search(OL, key) {
    // Início da lista
    current = OL.head
    // Enquanto não atingir o final da lista
    while current != NIL {
        # Se nó atual é o elemento, encontrou
        if current.key == key:
            return True
        else {
            // Se elemento atual é maior que valor buscado, pare
            if current.key > key
                break
            else
                current = current.next
        }
    }
    return False
}
```

Devemos também modificar o método `add()`, responsável por inserir um novo elemento na lista ordenada. A ideia consiste em encontrar a posição correta de inserção. Para isso, iniciamos pela cabeça da lista e percorremos a estrutura até localizar o ponto onde o novo elemento deve ser inserido.

Para determinar essa posição, utilizamos duas referências, `current` e `previous`, de forma análoga ao procedimento de remoção. A inserção ocorre quando o valor armazenado em `previous` é menor que a chave a ser inserida, e esta, por sua vez, é menor que o valor armazenado em `current`.

```
// Adiciona elemento na posição correta da lista
add(OL, key) {
    // Inicia na cabeça da lista
    current = OL.head
```

```

previous = NIL
// Enquanto não chegar ao final
while current != NIL {
    // Se valor do corrente for maior que o desejado, pare!
    if current.key > key
        break
    else {
        // Move previous e current para o próximo nó
        previous = current
        current = current.next
    }
}
// Cria novo nó
temp = Node(key)
// Se for o primeiro elemento
if previous == NIL {
    temp.next = OL.head
    OL.head = temp
}
else {
    // Inserção no meio ou no final
    temp.next = current
    previous.next = temp
}
}

```

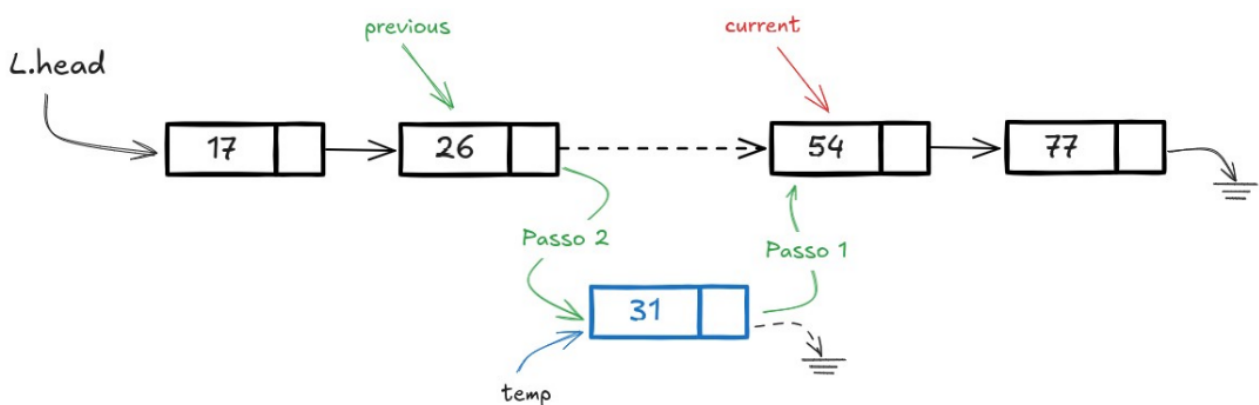


Figura 26: Ilustração da inserção de um novo nó em uma lista encadeada ordenada.

Em termos de complexidade, a principal diferença em relação às listas encadeadas não ordenadas está na operação de inserção. Em listas ordenadas, a inserção requer, no pior caso, a varredura de toda a estrutura para encontrar a posição correta, resultando em complexidade $O(n)$. Já na lista encadeada

não ordenada, a inserção no início pode ser realizada em $O(1)$, enquanto a inserção no final, na ausência de ponteiro para a cauda, possui complexidade $O(n)$.

5.3 Listas Duplamente Encadeadas

Listas duplamente encadeadas permitem navegação nos dois sentidos, isto é, tanto da esquerda para a direita quanto da direita para a esquerda. Isso é possível graças à inclusão de uma referência adicional para o nó anterior. Essa modificação estrutural traz algumas vantagens importantes, como a possibilidade de realizar remoções sem a necessidade de manter explicitamente uma referência auxiliar para o nó anterior.

```
TAD Node
    int key
    Node prev
    Node next

TAD Double_Linked_List
    Node head
    int size // número de elementos presentes na lista
```

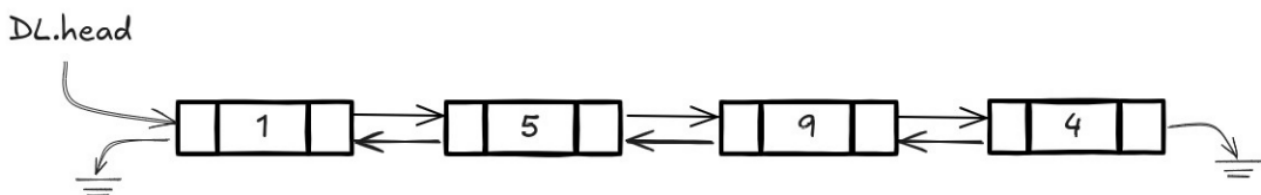


Figura 27: Ilustração de uma lista duplamente encadeada.

A seguir, apresentamos as primitivas de uma lista duplamente encadeada. As principais diferenças em relação à lista simplesmente encadeada estão nas operações de inserção no início, inserção no final e remoção.

```
init(DL) {
    DL.head = NIL
    DL.size = 0
}

// Adiciona no início de uma lista duplamente encadeada
add_head(DL, key) {
    // Cria novo nó
    temp = Node(key)
    if DL.head == NIL {
        DL.head = temp
    }
}
```

```

        temp.next = NIL
    }
    else {
        // Aponta novo nó para a cabeça da lista
        temp.next = DL.head
        DL.head.prev = temp
        // Atualiza a cabeça da lista
        DL.head = temp
    }
    temp.prev = NIL
    DL.size += 1
}

// Adiciona no fim de uma lista duplamente encadeada
add_tail(DL, key) {
    // Cria novo nó
    tail = Node(key)
    if DL.head == NIL {
        DL.head = tail
        tail.prev = NIL
    }
    else {
        // Usa referência temporária para percorrer a lista
        temp = DL.head
        // Percorre até o último elemento
        while temp.next != NIL:
            temp = temp.next
        // Conecta último nó ao novo nó
        temp.next = tail
        tail.prev = temp
    }
    tail.next = NIL
    DL.size += 1
}

```

A busca na lista duplamente encadeada pode ser realizada de maneira idêntica à busca em listas simplesmente encadeadas, percorrendo a estrutura a partir da cabeça. Entretanto, a remoção pode ser implementada sem a necessidade de uma referência auxiliar explícita, pois cada nó possui acesso direto ao seu predecessor.

```

remove(DL, key) {
    // Encontra a posição do nó a ser removido
    current = DL.head
    while current != NIL {
        if current.key == key
            break
        else
            current = current.next
    }
    if current == NIL
        return False
    elif current == DL.head {
        DL.head = current.next
        if DL.head != NIL
            DL.head.prev = NIL
    }
    else {
        current.prev.next = current.next
        if current.next != NIL
            current.next.prev = current.prev
    }
    # Desliga nó corrente
    current.next = NIL
    current.prev = NIL
}

```

Em listas duplamente encadeadas, a complexidade das operações de inserção e remoção depende fundamentalmente da necessidade (ou não) de localizar previamente a posição do elemento na estrutura. Quando a posição de inserção ou o nó a ser removido já são conhecidos por meio de uma referência direta, tanto a inserção quanto a remoção podem ser realizadas em tempo constante $O(1)$, pois envolvem apenas a atualização de um número fixo de ponteiros (`prev` e `next`). Em particular, inserções no início da lista possuem complexidade $O(1)$, assim como remoções no início ou no final, desde que haja acesso direto ao nó correspondente. Entretanto, caso seja necessário percorrer a lista para localizar a posição correta (por exemplo, em listas ordenadas ou na busca prévia ao elemento a ser removido), o custo passa a ser dominado pela etapa de varredura sequencial, resultando em complexidade $O(n)$ no pior caso. Assim, a principal vantagem da lista duplamente encadeada não está na redução da ordem assintótica, mas na simplificação das operações de atualização estrutural e na eliminação da necessidade de manter uma referência auxiliar explícita ao nó anterior.

5.3.1 Listas Duplamente Encadeadas com Sentinelas

Conforme discutido anteriormente, a inserção no final de uma lista duplamente encadeada tradicional possui complexidade $O(n)$ quando não há uma referência explícita para a cauda. Uma maneira de tornar essa operação mais eficiente consiste na utilização de listas duplamente encadeadas com sentinelas.

Em listas desse tipo, tanto a inserção quanto a remoção no final podem ser realizadas em tempo $O(1)$, pois a estrutura mantém duas extremidades fixas: `header` (início) e `trailer` (final). Esses nós especiais não armazenam elementos válidos da lista e não podem ser removidos. Por essa razão, são chamados de sentinelas.

Em uma lista duplamente encadeada com sentinelas, cada nó possui uma informação e duas referências: uma para o nó antecessor e outra para o nó sucessor.

```
TAD Node
    int key
    Node prev
    Node next

TAD Sentinel_Double_Linked_List
    Node header    // sentinela: nó que não pode ser removido
    Node trailer   // sentinela: nó que não pode ser removido
    int size       // número de elementos presentes na lista
```

A inicialização da lista cria explicitamente os dois nós sentinelas e estabelece o encadeamento básico entre eles.

```
init(DL) {
    DL.header = new Node()    // cria sentinela inicial (sem chave)
    DL.trailer = new Node()   // cria sentinela final (sem chave)
    DL.header.prev = NIL
    DL.header.next = DL.trailer
    DL.trailer.prev = DL.header
    DL.trailer.next = NIL
    DL.size = 0
}
```

Assim como a lista simplesmente encadeada possui uma referência `head` que aponta para o início do encadeamento lógico, a lista duplamente encadeada com sentinelas possui duas referências especiais: `header`, que representa o início estrutural da lista, e `trailer`, que representa o seu final. Os elementos válidos da lista são sempre inseridos entre essas duas sentinelas. A figura a seguir ilustra a organização estrutural dessa representação.

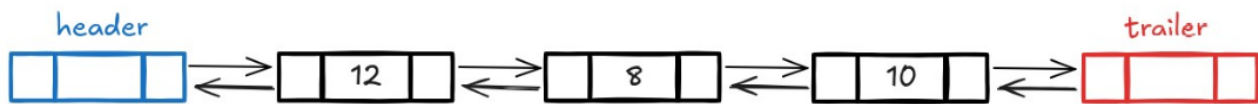


Figura 28: Ilustração de uma lista duplamente encadeada com sentinelas.

Para essa classe, adotaremos a estratégia de, a cada nó inserido, incrementar o atributo `size` em uma unidade e, a cada nó removido, decrementá-lo também em uma unidade. Dessa forma, não há necessidade de implementar uma função específica para contar quantos elementos existem na lista, pois essa informação é mantida atualizada dinamicamente, podendo ser obtida em tempo $O(1)$.

Com relação à operação de inserção, a principal diferença em relação à lista simplesmente encadeada é que, em uma lista duplamente encadeada, o novo nó deve ser conectado tanto ao seu sucessor quanto ao seu antecessor. Assim, além de atualizar o ponteiro `next` do nó anterior, é necessário também ajustar o ponteiro `prev` do nó sucessor, garantindo a consistência bidirecional do encadeamento, conforme ilustra a figura a seguir.

A mesma observação se aplica à operação de remoção. Para desconectar completamente um nó da lista duplamente encadeada, devemos religar o seu antecessor ao seu sucessor, ajustando simultaneamente os ponteiros `next` e `prev`. Somente após essa atualização estrutural o nó pode ser considerado efetivamente removido da lista.

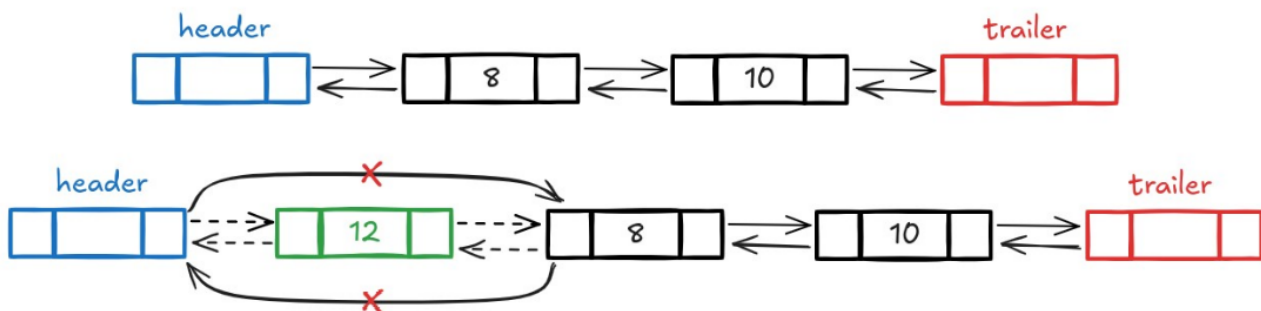


Figura 29: Ilustração da remoção no início em uma lista duplamente encadeada com sentinelas.

A mesma observação vale para a remoção de um nó. Para desconectá-lo completamente da lista duplamente encadeada, devemos ligar o antecessor ao sucessor e vice-versa.

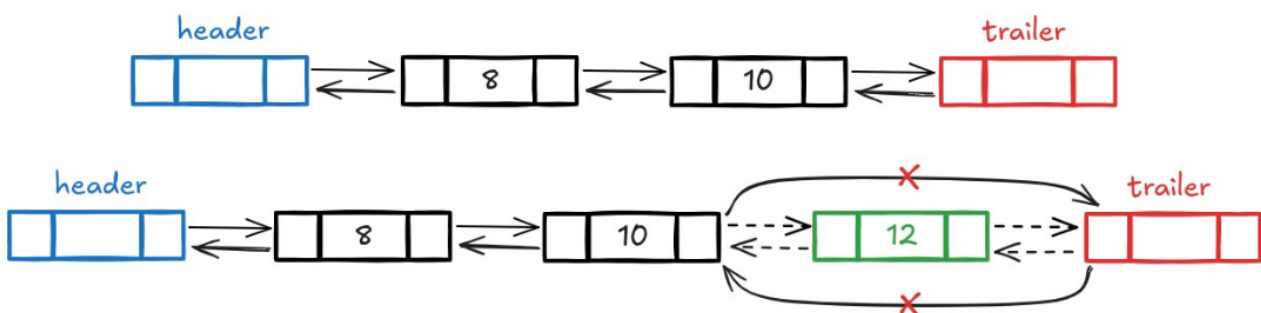


Figura 30: Ilustração da remoção no final em uma lista duplamente encadeada com sentinelas.

O algoritmo da função auxiliar `insert_between()` é apresentado a seguir. Essa função insere um novo nó entre dois nós já existentes na lista duplamente encadeada.

```
// Insere novo nó entre dois nós existentes
insert_between(DL, key, predecessor, successor) {
    new = Node(key)
    predecessor.next = new
    successor.prev = new
    new.prev = predecessor
    new.next = successor
    DL.size += 1
    return new
}
```

De posse da função anterior, a inserção no início da lista duplamente encadeada pode ser realizada conforme o algoritmo a seguir.

```
// Insere elemento no início
insert_first(DL, key) {
    // Nó deve ficar entre header e header.next
    insert_between(DL, key, DL.header, DL.header.next)
}
```

De maneira análoga, a inserção no final da lista duplamente encadeada pode ser implementada conforme descrito abaixo.

```
// Insere elemento no final
insert_last(DL, key) {
    // Nó deve ficar entre trailer.prev e trailer
    insert_between(DL, key, DL.trailer.prev, DL.trailer)
}
```

Note que, diferentemente das listas encadeadas simples, a inserção no final de uma lista duplamente encadeada com sentinelas possui complexidade $O(1)$, pois não é necessário percorrer a estrutura para localizar o último elemento. Essa característica pode representar uma vantagem significativa em diversas aplicações. A remoção de um nó pode ser realizada de maneira análoga à inserção, utilizando uma função auxiliar `delete_node()`. Observe que os nós sentinelas (`header` e `trailer`) nunca podem ser removidos.

```
// Remove um nó intermediário da lista
// OBS: header e trailer nunca podem ser removidos!
delete_node(DL, key) {
    node = search(DL, key)
    if node == NIL
        return False
```

```

else {
    predecessor = node.prev
    successor = node.next
    predecessor.next = successor
    successor.prev = predecessor
    DL.size -= 1
    // Desconecta completamente o nó removido
    node.prev = NIL
    node.next = NIL
    x = node.key
    return x
}
}

```

5.3.2 Listas Duplamente Encadeadas Circulares

Outra forma de tornar a inserção no final uma operação de complexidade $O(1)$ consiste em utilizar uma lista duplamente encadeada circular. Nesse tipo de estrutura, o último elemento da lista mantém uma referência para o primeiro, formando um ciclo, conforme ilustra a figura a seguir.

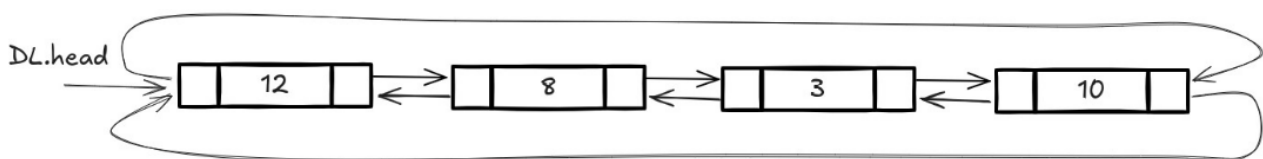


Figura 31: Ilustração de uma lista duplamente encadeada circular.

Em uma lista circular duplamente encadeada, o ponteiro `prev` do nó cabeça referencia o último nó da lista, e o ponteiro `next` do último nó referencia a cabeça. Dessa forma, não há extremidades com valor `NIL`, exceto no caso da lista vazia. A seguir, apresentamos os algoritmos de inserção no início e no final dessa estrutura.

```

// Insere no início da lista circular duplamente encadeada
add_head(DL, key) {
    temp = Node(key)
    if DL.head == NIL {
        DL.head = temp
        temp.next = temp
        temp.prev = temp
    }
    else {
        // Novo nó aponta para a cabeça atual

```

```

        temp.next = DL.head
        temp.prev = DL.head.prev
        // Ajusta conexões circulares
        DL.head.prev.next = temp
        DL.head.prev = temp
        // Atualiza a cabeça da lista
        DL.head = temp
    }
    DL.size += 1
}

// Insere no final da lista circular duplamente encadeada
add_tail(DL, key) {
    temp = Node(key)
    if DL.head == NIL {
        DL.head = temp
        temp.next = temp
        temp.prev = temp
    }
    else {
        // Novo nó aponta para a cabeça
        temp.next = DL.head
        temp.prev = DL.head.prev
        // Ajusta conexões circulares
        DL.head.prev.next = temp
        DL.head.prev = temp
        // Não é necessário atualizar a cabeça!
    }
    DL.size += 1
}

```

Observe que, nessa representação, tanto a inserção no início quanto a inserção no final podem ser realizadas em tempo constante $O(1)$, pois o acesso ao último elemento é obtido diretamente por meio da referência `DL.head.prev`. Isso elimina a necessidade de percorrer a lista para localizar a cauda, tornando a estrutura eficiente para aplicações que exigem inserções frequentes em ambas as extremidades.

5.4 Matrizes Esparsas com Arrays e Listas Encadeadas

Uma aplicação de grande relevância na Ciência da Computação consiste na representação eficiente de matrizes esparsas. Uma matriz é dita esparsa quando aproximadamente $2/3$ ou mais de seus elementos são nulos. Um exemplo de matriz esparsa é apresentado a seguir:

$$\begin{bmatrix} 0 & 0 & 3 & 0 & 4 \\ 0 & 0 & 5 & 7 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 6 & 0 & 0 \end{bmatrix}$$

Note que, dos 20 elementos da matriz, apenas 6 são não nulos, o que corresponde a apenas 30% do total. Representar matrizes esparsas da forma tradicional, utilizando um arranjo bidimensional com n linhas e m colunas, não é eficiente do ponto de vista de memória, pois há um grande desperdício de espaço para armazenar valores nulos.

Suponha que os elementos da matriz anterior sejam representados como números de ponto flutuante (*float*), e que cada elemento ocupe 64 bits. Nesse caso, a matriz completa ocupa

$$64 \times 20 = 1280 \text{ bits.}$$

Entretanto, como apenas 6 elementos são não nulos, grande parte dessa memória é utilizada para armazenar zeros, o que não é desejável. A seguir, estudaremos duas abordagens para armazenar matrizes esparsas de maneira mais eficiente, reduzindo significativamente o consumo de memória ao armazenar apenas os elementos não nulos e suas respectivas posições.

5.4.1 Método 1: Arrays Estáticos

Neste primeiro método, criamos três arrays estáticos: R (*rows*), C (*columns*) e V (*values*), todos de tamanho K , onde K é o número de elementos não nulos da matriz esparsa. A lógica da representação é a seguinte:

- $R[k]$ indica a linha (*row*) em que o k -ésimo elemento não nulo se encontra;
- $C[k]$ indica a coluna (*column*) em que o k -ésimo elemento não nulo se encontra;
- $V[k]$ indica o valor (*value*) do elemento não nulo localizado na posição $(R[k], C[k])$.

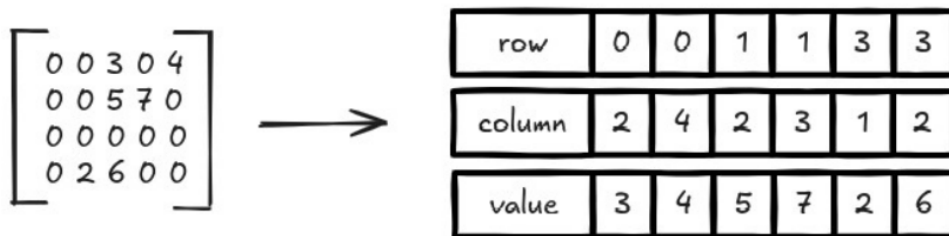


Figura 32: Ilustração de uma estrutura de dados estática para armazenar uma matriz esparsa.

Como os índices são inteiros, suponha que cada elemento dos arrays R e C ocupe 32 bits de memória. Já os elementos do array V devem ser representados como *float*, ocupando 64 bits. Considerando o exemplo anterior, no qual $K = 6$, o espaço total ocupado pelos três arrays é dado por:

$$32 \times 6 + 32 \times 6 + 64 \times 6 = 192 + 192 + 384 = 768 \text{ bits.}$$

Como a representação tradicional da matriz ocupava 1280 bits, temos:

$$\frac{768}{1280} = 0.6,$$

ou seja, uma economia de aproximadamente 40% de memória, o que representa uma redução significativa de espaço. A seguir, apresenta-se um algoritmo para codificar uma matriz esparsa A utilizando essa estratégia. Observe que, apesar da economia de memória, a estrutura empregada ainda é estática, pois os arrays possuem tamanho fixo determinado previamente.

Entrada: matriz esparsa A de dimensão $n \times m$.

```
sparse_matrix_encode(A, n, m) {
    int R[K]    // array de inteiros de tamanho K
    int C[K]    // array de inteiros de tamanho K
    float V[K]  // array de floats de tamanho K
    K = 0
    for i = 1 to n {
        for j = 1 to m {
            if A[i, j] != 0
                K += 1
        }
    }
    k = 1
    for i = 1 to n {
        for j = 1 to m {
            if A[i, j] != 0 {
                R[k] = i
                C[k] = j
                V[k] = A[i, j]
                k += 1
            }
        }
    }
}
```

5.4.2 Método 2: Listas Encadeadas

Uma segunda abordagem consiste na utilização de uma lista encadeada para armazenar apenas os elementos não nulos da matriz esparsa. Nesse caso, cada nó da lista contém três campos de informação (linha, coluna e valor) e uma referência para o próximo nó da lista.

TAD Node

```
int row      // índice da linha
int col      // índice da coluna
float data   // valor do elemento não nulo
Node next    // referência para o próximo nó (encadeamento lógico)
```

TAD Sparse_Matrix

```
Node head    // referência para o primeiro nó
Node tail    // referência para o último nó inserido
int size     // número de nós na lista
```

A seguir, apresenta-se o procedimento de inicialização da estrutura:

```
init(SM) {
    SM.head = NIL
    SM.tail = NIL
    SM.size = 0
}
```

O próximo procedimento cria um novo nó e o insere ao final da lista encadeada:

```
create_new_node(SM, row, col, data) {
    new_node = Node(row, col, data, NIL)
    if SM.size == 0 {
        SM.head = new_node
    }
    else {
        SM.tail.next = new_node
    }
    SM.tail = new_node
    SM.size += 1
}
```

Por fim, o algoritmo para codificar uma matriz esparsa A de dimensão $n \times m$ é dado por:

```
sparse_matrix_encode(SM, A, n, m) {
    for i = 1 to n {
        for j = 1 to m {
            if A[i, j] != 0 {
                create_new_node(SM, i, j, A[i, j])
            }
        }
    }
}
```

Observe que, em termos de espaço, a estrutura dinâmica é bastante semelhante à estrutura estática apresentada anteriormente, pois também armazena apenas os elementos não nulos e suas respectivas posições.



Figura 33: Ilustração de uma estrutura de dados dinâmica para armazenar uma matriz esparsa.

A principal diferença, contudo, reside na flexibilidade. Inserir ou remover nós em uma lista encadeada é consideravelmente mais simples do que redimensionar uma estrutura estática baseada em arrays. Dessa forma, a representação dinâmica torna-se mais adequada para aplicações em que os dados da matriz esparsa podem sofrer inserções, remoções ou atualizações frequentes.

5.5 Listas Ortogonais e Matrizes Dinâmicas

Uma lista ortogonal é uma estrutura dinâmica composta por nós que possuem quatro referências, correspondentes às direções: cima (up), baixo (down), esquerda (left) e direita (right). Assim como uma matriz pode ser vista como uma generalização bidimensional de um vetor (*array*), uma lista ortogonal pode ser interpretada como uma generalização bidimensional de uma lista encadeada.

```
TAD OLNode
    int key
    OLNode up
    OLNode down
    OLNode left
    OLNode right
```

```
TAD Orthogonal_List
    OLNode head
    int size
```

A inicialização da estrutura pode ser realizada a partir de uma matriz A de dimensões $n \times m$.

```
init(SM, A, n, m) {
    SM.head = build_orthogonal_list(A, n, m)
    SM.size = n * m
}
```

A função responsável por construir os encadeamentos ortogonais é apresentada a seguir.

```
build_orthogonal_list(A, n, m) {
    OLNode map[n, m]    // matriz auxiliar para encadeamentos
```

```

for i = 1 to n {
    for j = 1 to m {
        // Cria um novo nó para cada entrada da matriz
        new_node = OLNode(A[i, j])
        // Armazena referência para o novo nó
        map[i, j] = new_node
        // Ajusta referências de cima e de baixo
        if i != 1 {
            new_node.up = map[i-1, j]
            map[i-1, j].down = new_node
        }
        // Ajusta referências da esquerda e da direita
        if j != 1 {
            new_node.left = map[i, j-1]
            map[i, j-1].right = new_node
        }
    }
}
return map[1, 1]    // referência para nó inicial
}

```

O percurso da lista ortogonal pode ser realizado linha por linha, de maneira análoga à varredura de uma matriz bidimensional.

```

// Percorre a lista ortogonal (linha por linha)
traverse_orthogonal_list(OL) {
    current_row = OL.head
    while current_row != NIL {
        current_col = current_row
        while current_col != NIL {
            print(current_col.key)
            current_col = current_col.right
        }
        print("\n")
        current_row = current_row.down
    }
}

```

A figura a seguir ilustra uma representação gráfica de uma lista ortogonal, evidenciando os encaamentos nas quatro direções e sua correspondência estrutural com uma matriz dinâmica bidimensional.

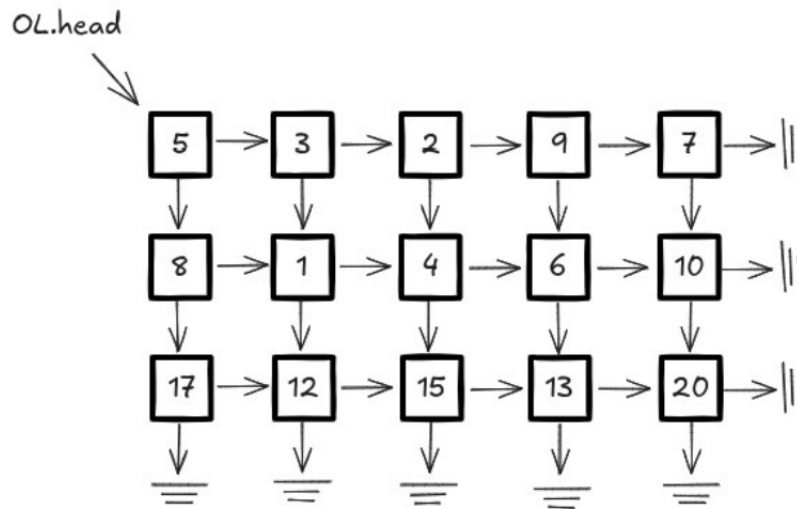


Figura 34: Ilustração de uma estrutura de dados para representar uma lista ortogonal, também conhecida como matriz esparsa.

5.6 Considerações Finais

Neste capítulo, foram apresentadas as principais estruturas de dados lineares dinâmicas, com ênfase em listas encadeadas simples, listas duplamente encadeadas e suas variações. Diferentemente das estruturas lineares estáticas, como vetores, as estruturas dinâmicas permitem alocação e desalocação de memória em tempo de execução, proporcionando maior flexibilidade na gestão de dados cuja cardinalidade pode variar ao longo do processamento.

Do ponto de vista conceitual, observou-se que a principal característica dessas estruturas reside no uso de nós interligados por referências. Essa organização elimina a necessidade de armazenamento contíguo em memória, mas impõe custos adicionais associados ao uso de ponteiros e à navegação sequencial. Como consequência, operações como busca possuem complexidade linear no pior caso, enquanto inserções e remoções podem ser realizadas em tempo constante quando a posição do nó é previamente conhecida.

A análise de complexidade apresentada ao longo do capítulo evidencia o compromisso clássico entre flexibilidade estrutural e eficiência de acesso. Enquanto vetores oferecem acesso direto em tempo constante, listas encadeadas favorecem modificações estruturais locais com baixo custo assintótico. Essa distinção é fundamental para a escolha adequada da estrutura de dados em projetos de software e na implementação de algoritmos.

Além disso, a extensão para estruturas bidimensionais dinâmicas, como listas ortogonais, demonstra como o princípio de encadeamento pode ser generalizado para modelar estruturas mais complexas, preservando a ideia central de organização por referências. Tal generalização amplia o repertório de soluções para problemas que envolvem dados estruturados em múltiplas dimensões.

Em síntese, o domínio das estruturas lineares dinâmicas constitui um passo essencial na formação em algoritmos e estruturas de dados, pois estabelece a base conceitual para o estudo de estruturas não lineares mais sofisticadas, como árvores e grafos, que serão abordadas nos capítulos seguintes.

6 Árvores Binárias

O estudo de árvores binárias marca a transição das estruturas lineares para estruturas hierárquicas, nas quais os dados deixam de ser organizados de forma sequencial e passam a obedecer a uma relação pai–filho. Uma árvore é formalmente definida como um conjunto finito de nós, podendo ser vazio, ou composto por um nó raiz e zero ou mais subárvores disjuntas. No caso particular das árvores binárias, cada nó possui no máximo dois filhos, usualmente denominados filho esquerdo e filho direito. Essa restrição estrutural, embora simples, é suficiente para modelar uma ampla variedade de problemas computacionais, desde representações sintáticas até estruturas eficientes de busca.

Do ponto de vista estrutural, uma árvore binária é composta por nós que armazenam uma chave (ou valor) e duas referências para subárvores. Diferentemente das listas encadeadas, nas quais a navegação é essencialmente sequencial, as árvores introduzem múltiplos caminhos possíveis a partir de cada nó, o que altera substancialmente o comportamento das operações fundamentais. Conceitos como altura, profundidade, nível, folhas e nós internos tornam-se centrais para a análise estrutural e para o estudo da complexidade dos algoritmos associados. Ao longo deste capítulo, serão exploradas as propriedades fundamentais das árvores binárias, suas formas de representação e os principais algoritmos de percurso, inserção e busca.

Árvores binárias são estruturas de dados dinâmicas baseadas em encadeamento lógico e que apresentam organização hierárquica. Em uma árvore binária de busca T , cada nó deve conter, no mínimo, quatro informações fundamentais:

- **Chave (*key*)**: valor a ser armazenado no nó de T ;
- ***p***: referência para o nó pai;
- ***left***: referência para o filho à esquerda;
- ***right***: referência para o filho à direita.

Uma árvore binária T possui sempre um nó especial denominado *raiz*, que representa o ponto de entrada da estrutura. Neste texto, a raiz será denotada por $T.root$. Opcionalmente, pode-se manter um atributo adicional *size*, responsável por armazenar o número total de nós presentes na árvore, facilitando operações que dependem dessa informação.

Um possível Tipo Abstrato de Dados (TAD) para representar os nós e a estrutura de uma árvore binária pode ser definido da seguinte forma:

```
TAD TreeNode
    int key
    TreeNode p
    TreeNode left
    TreeNode right
```

```
TAD Binary_Tree
  TreeNode root
  int size
```

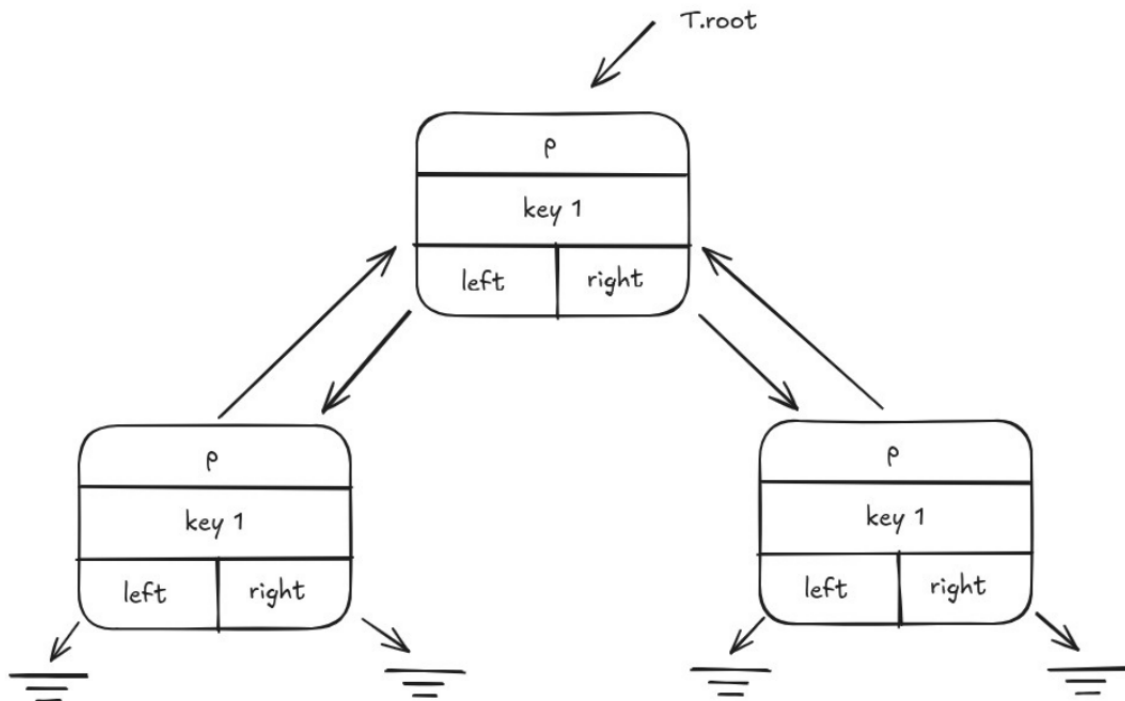


Figura 35: Ilustração de uma estrutura de dados do tipo árvore binária.

Uma pergunta natural a esta altura é: como podemos percorrer todos os nós de uma árvore binária? Utilizando uma abordagem recursiva, é possível definir algoritmos simples que visitam todos os nós da árvore. Esses percursos são conhecidos como *inorder*, *preorder* e *postorder*, dependendo da ordem em que o nó atual é processado em relação às chamadas recursivas para seus filhos.

Percurso Inorder

```
Tree_Walk_Inorder(x) {
  if x ≠ NIL {
    Tree_Walk_Inorder(x.left)
    print(x.key)
    Tree_Walk_Inorder(x.right)
  }
}
```

No percurso *inorder*, primeiro visitamos a subárvore esquerda, depois o próprio nó e, por fim, a subárvore direita.

Percurso Preorder

```
Tree_Walk_Preorder(x) {  
    if x ≠ NIL {  
        print(x.key)  
        Tree_Walk_Preorder(x.left)  
        Tree_Walk_Preorder(x.right)  
    }  
}
```

No percurso *preorder*, o nó atual é visitado antes de suas subárvores esquerda e direita.

Percurso Postorder

```
Tree_Walk_Postorder(x) {  
    if x ≠ NIL {  
        Tree_Walk_Postorder(x.left)  
        Tree_Walk_Postorder(x.right)  
        print(x.key)  
    }  
}
```

No percurso *postorder*, o nó é visitado apenas após a exploração das subárvores esquerda e direita.

Supondo a árvore binária ilustrada anteriormente, podemos analisar a ordem de visitação dos nós quando utilizamos os três percursos clássicos: *inorder*, *preorder* e *postorder*. Cada estratégia define uma ordem distinta de processamento dos nós da árvore.

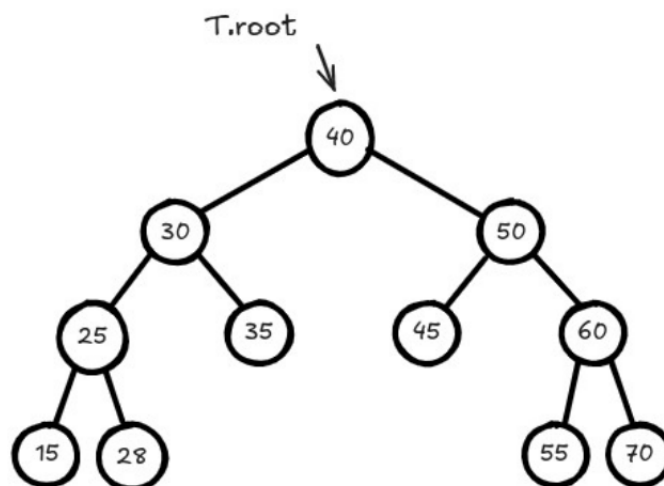


Figura 36: Ilustração de uma árvore binária.

- **Percurso inorder:** 15, 25, 28, 30, 35, 40, 45, 50, 55, 60, 70
- **Percurso preorder:** 40, 30, 25, 15, 28, 35, 50, 45, 60, 55, 70

- **Percurso postorder:** 15, 28, 25, 35, 30, 45, 55, 70, 60, 50, 40

Cada uma dessas estratégias de percurso possui aplicações específicas. Por exemplo, o percurso *inorder* em uma árvore binária de busca visita os elementos em ordem crescente de suas chaves.

Uma questão natural neste ponto é determinar qual é a complexidade computacional das funções responsáveis por percorrer os nós de uma árvore binária. A resposta para essa questão será discutida na seção a seguir.

Análise da Complexidade

Seja uma árvore binária com N nós, de modo que existam exatamente L nós na subárvore à esquerda da raiz e $N - L - 1$ nós na subárvore à direita.

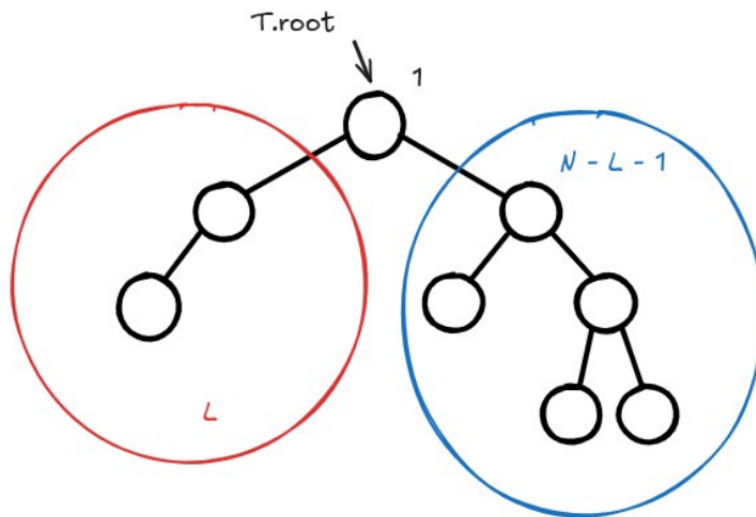


Figura 37: Árvore binária com N nós, sendo L nós na subárvore a esquerda e o restante na subárvore a direita.

Observe que o número total de nós pode ser escrito como

$$(N - L - 1 + L) + 1 = N.$$

Com base nessa decomposição, podemos definir a seguinte recorrência para o tempo de execução do algoritmo de percurso da árvore:

$$T(N) = T(L) + T(N - L - 1) + C$$

onde C representa uma constante associada à operação realizada no nó atual (por exemplo, a operação de impressão da chave).

Para obter um limite superior para essa recorrência, consideremos o pior caso, que ocorre quando a árvore está totalmente desbalanceada para a direita, aumentando sua profundidade. Nesse cenário temos $L = 0$, resultando na seguinte recorrência:

$$T(N) = T(0) + T(N - 1) + C$$

Expandindo recursivamente o termo $T(N - 1)$, obtemos:

$$T(N) = T(0) + T(0) + T(N - 2) + C + C$$

Repetindo o processo, temos:

$$T(N) = T(0) + T(0) + T(0) + T(N - 3) + C + C + C$$

Continuando a expansão até chegar ao caso base $T(1)$, teremos exatamente $N - 1$ passos. Assim, podemos escrever

$$T(N) = (N - 1)T(0) + T(1) + (N - 1)C$$

ou, reorganizando os termos,

$$T(N) = NT(0) - T(0) + T(1) + NC - C.$$

Sabendo que $T(0) = 1$ e que $T(1)$ é uma constante arbitrária K , obtemos

$$T(N) = N - 1 + K + NC - C.$$

Portanto, concluímos que o tempo de execução do percurso de uma árvore binária é linear no número de nós da árvore, isto é,

$$T(N) = O(N).$$

6.1 Limitações das Árvores Binárias Simples

Em uma árvore binária simples não existe nenhuma restrição quanto à posição em que novos elementos podem ser inseridos. Em outras palavras, uma nova chave pode ser adicionada em qualquer posição disponível da estrutura, desde que a propriedade básica da árvore binária seja respeitada, isto é, cada nó possua no máximo dois filhos. Essa flexibilidade na inserção, embora simplifique a construção da estrutura, traz consequências importantes para as operações de busca e remoção.

Como não há qualquer relação de ordem entre as chaves armazenadas nos nós, torna-se impossível descartar partes da árvore durante uma busca. Dessa forma, no pior caso, pode ser necessário percorrer todos os nós da árvore até encontrar o elemento desejado ou concluir que ele não pertence à estrutura. Consequentemente, a operação de busca em uma árvore binária simples possui complexidade $O(N)$, onde N é o número total de nós da árvore, o que é equivalente à busca em uma lista encadeada.

A remoção de um elemento também se torna mais trabalhosa. Após localizar o nó a ser removido, pode ser necessário reorganizar parte da estrutura para manter o encadeamento correto entre os nós.

Como não existe nenhuma propriedade de ordenação que auxilie na localização de sucessores ou predecessores naturais, o processo de remoção tende a exigir percursos adicionais na árvore.

Por essas razões, na maioria das aplicações práticas prefere-se utilizar *árvores binárias de busca* (Binary Search Trees — BST). Nesse tipo de estrutura, as chaves são organizadas de acordo com uma propriedade de ordem: para cada nó da árvore, todas as chaves da subárvore à esquerda são menores que a chave do nó, enquanto todas as chaves da subárvore à direita são maiores. Essa organização permite realizar buscas, inserções e remoções de forma muito mais eficiente, explorando a estrutura hierárquica da árvore para reduzir o número de nós visitados durante as operações.

6.2 Árvores Binárias de Busca (Binary Search Trees - BST)

Em árvores binárias de busca, cada chave possui uma localização bem definida dentro da estrutura, de maneira semelhante ao que ocorre em uma lista ordenada. A organização das chaves segue uma propriedade fundamental que garante a manutenção da ordem dentro da árvore.

Propriedade chave: Seja x um nó arbitrário de uma árvore binária de busca T . Se y é um nó pertencente à subárvore à esquerda de x , então $y.key \leq x.key$. Se y é um nó pertencente à subárvore à direita de x , então $y.key \geq x.key$.

Essa propriedade garante que cada chave tenha uma posição única dentro da árvore. Consequentemente, em uma árvore binária de busca não é possível inserir novos nós em posições arbitrárias. Toda inserção deve preservar a propriedade de ordenação descrita anteriormente, garantindo que a estrutura permaneça válida após cada operação.

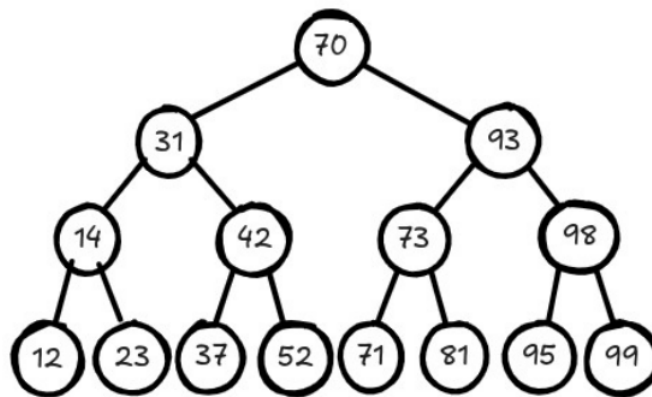


Figura 38: Ilustração de uma árvore binária de busca. Note que para toda chave, as chaves menores encontram-se na subárvore à esquerda e as chaves maiores encontram-se na subárvore à direita.

Uma consequência importante dessa propriedade é que o percurso *inorder* sempre imprime as chaves da árvore em ordem crescente. Por exemplo, considerando a árvore ilustrada na figura anterior, o percurso *inorder* produziria a seguinte sequência de chaves:

12, 14, 23, 31, 37, 42, 52, 70, 71, 73, 81, 93, 95, 98, 99.

Outra operação fundamental em árvores binárias de busca consiste em encontrar a menor e a maior chave armazenada na estrutura. Devido à propriedade de ordenação da árvore, essas operações são bastante simples. Para encontrar a menor chave, basta seguir continuamente pelos filhos à esquerda a partir da raiz da árvore. De maneira análoga, para encontrar a maior chave, basta seguir pelos filhos à direita. A seguir apresentamos os algoritmos para encontrar o mínimo e o máximo em uma árvore binária de busca.

```
// Passar o nó raiz como parâmetro
Tree_Minimum(x) {
    while x.left ≠ NIL
        x = x.left
    return x
}
```

```
// Passar o nó raiz como parâmetro
Tree_Maximum(x) {
    while x.right ≠ NIL
        x = x.right
    return x
}
```

6.2.1 Busca em Árvores Binárias de Busca

A operação de busca é uma das funcionalidades fundamentais de uma árvore binária de busca. Devido à propriedade de ordenação da estrutura, a busca pode ser realizada de forma eficiente seguindo apenas um dos ramos da árvore a cada passo. Existem duas formas principais de implementar esse algoritmo: uma abordagem recursiva e uma abordagem iterativa. Inicialmente apresentaremos a versão recursiva.

```
// Passar a raiz da árvore como parâmetro
Recursive_Tree_Search(x, k) {
    if x == NIL or k == x.key
        return x
    if k < x.key
        return Recursive_Tree_Search(x.left, k)
    else
        return Recursive_Tree_Search(x.right, k)
}
```

Note que a complexidade do algoritmo depende essencialmente da altura h da árvore. No melhor caso, quando a árvore encontra-se perfeitamente balanceada, ou seja, quando as alturas das subárvores à esquerda e à direita são aproximadamente iguais em todos os nós, obtemos uma estrutura semelhante à ilustrada na figura a seguir.

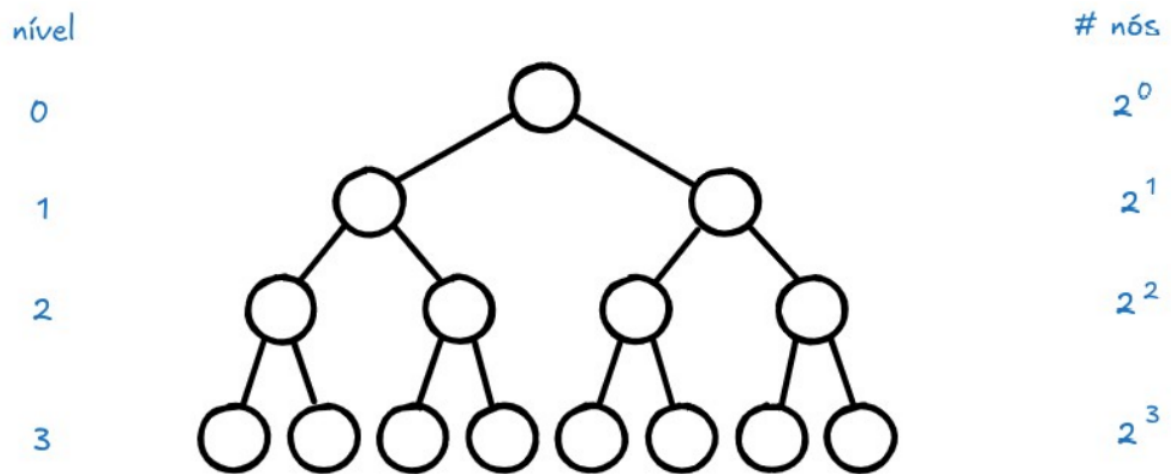


Figura 39: Em uma árvore binária de busca, o número máximo de nós que podem ser armazenados no nível k é igual a 2^k .

A relação entre o número de nós n e a altura da árvore pode ser obtida pela seguinte soma:

$$n = \sum_{k=0}^h 2^k = 2^0 + 2^1 + 2^2 + \dots + 2^h$$

Sabemos que

$$2^{k+1} = 2^k + 2^k$$

o que nos permite escrever

$$2^k = 2^{k+1} - 2^k$$

Assim, podemos calcular n utilizando uma soma telescópica:

$$n = \sum_{k=0}^h (2^{k+1} - 2^k) = 2^{h+1} - 1$$

Logo,

$$2^{h+1} = n + 1$$

Aplicando logaritmo em ambos os lados da equação:

$$h = \log_2(n + 1) - 1$$

Portanto, a menor altura possível para uma árvore binária de busca com n nós é da ordem de

$$h = O(\log_2 n)$$

Por exemplo, suponha que $n = 15$. Nesse caso temos:

$$h = \log_2(16) - 1 = 4 - 1 = 3$$

No pior caso, quando a árvore se degenera para uma estrutura semelhante a uma lista encadeada (isto é, quando todos os nós possuem apenas um filho), a altura da árvore torna-se igual ao número de nós, ou seja, $h = n$. Consequentemente, a complexidade da busca em árvores binárias de busca pode variar entre

$$O(\log n) \text{ e } O(n)$$

dependendo do grau de balanceamento da árvore. A seguir apresentamos a versão iterativa do algoritmo de busca.

```
// Passar a raiz da árvore como parâmetro
Iterative_Tree_Search(x, k) {
    while x ≠ NIL and k ≠ x.key {
        if k < x.key
            x = x.left
        else
            x = x.right
    }
    return x
}
```

6.2.2 Sucessor e Predecessor

Encontrar os elementos que precedem ou sucedem um nó x em uma árvore binária de busca é uma operação importante em diversos algoritmos. Assumiremos, por simplicidade, que não existem duas chaves idênticas na árvore T .

´ **Definição:** O sucessor de um nó x é o nó y de menor chave tal que $y.key > x.key$. Em outras palavras, trata-se do próximo nó visitado após x em um percurso *inorder* da árvore. Existem duas situações possíveis para encontrar o sucessor de um nó:

1. Se a subárvore à direita do nó x não é vazia, então o sucessor de x é o menor elemento dessa subárvore.

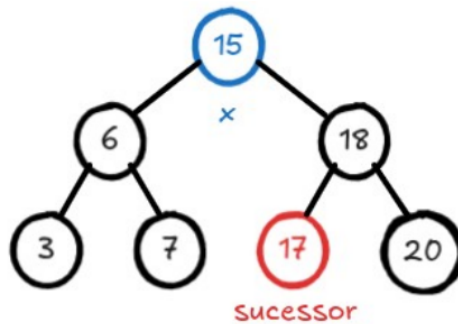


Figura 40: Se a subárvore a direita existe, então o sucessor é o mínimo desta subárvore.

2. Se a subárvore à direita de x é vazia e x possui um sucessor y , então y é o ancestral mais baixo de x cujo filho à esquerda também é um ancestral de x .

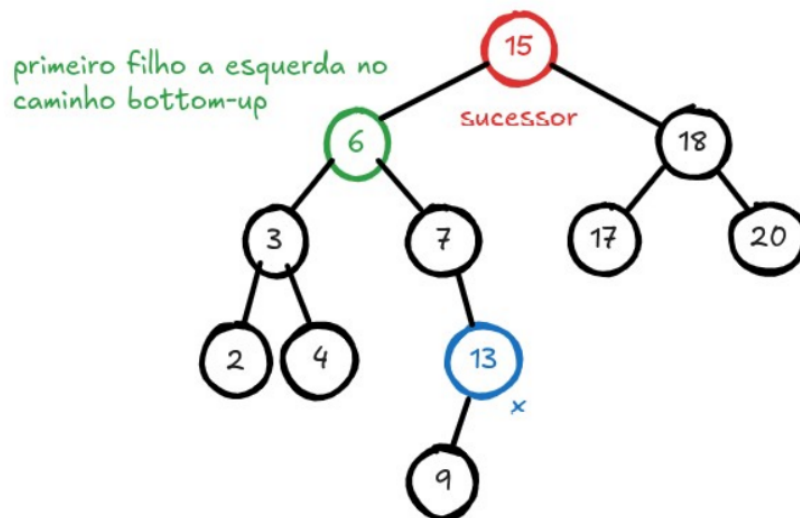


Figura 41: Se a subárvore a direita não existe, então o sucessor é o primeiro filho a esquerda no caminho bottom-up até a raiz.

O algoritmo a seguir encontra o sucessor de um nó x em uma árvore binária de busca.

```
Tree_Successor(x) {
    if x.right ≠ NIL
        return Tree_Minimum(x.right)
    else {
        y = x.p
        while y != NIL and x == y.right {
            x = y
            y = y.p
        }
        return y
    }
}
```

Note que a complexidade desse algoritmo depende da altura h da árvore. Assim, no melhor caso, quando a árvore encontra-se balanceada, o custo é $O(\log n)$. Já no pior caso, quando a árvore se degenera para uma lista encadeada, a altura pode chegar a n , levando a uma complexidade $O(n)$. Para encontrar o predecessor de um nó x em uma árvore binária de busca, devemos considerar duas situações possíveis:

1. Se a subárvore à esquerda de x não é vazia, então o predecessor de x é o maior elemento dessa subárvore.

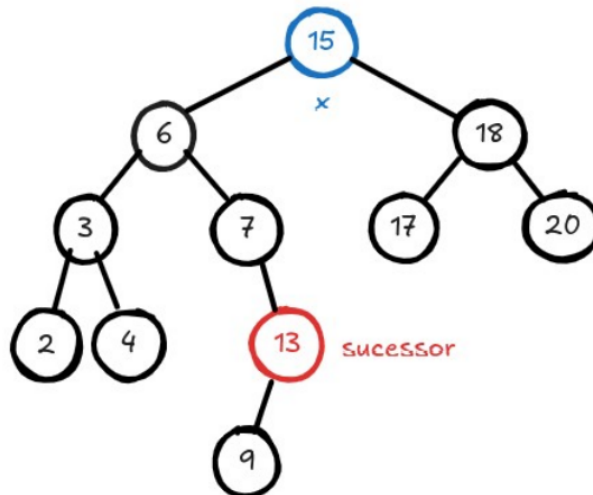


Figura 42: Se a subárvore à esquerda existe, então o sucessor é o máximo desta subárvore.

2. Se a subárvore à esquerda de x é vazia, então o predecessor y de x é o ancestral mais baixo em T cujo filho à direita também é um ancestral de x .
3. Se a subárvore à direita de x é vazia e x possui um predecessor y , então y é o ancestral mais baixo de x cujo filho à direita também é um ancestral de x .

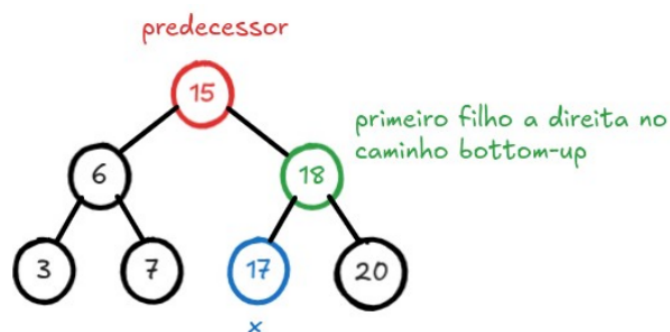


Figura 43: Se a subárvore à esquerda não existe, então o sucessor está é o primeiro filho a direita no caminho bottom-up até a raiz.

O algoritmo a seguir implementa a busca pelo predecessor de um nó em uma árvore binária de busca.

```

Tree_Predecessor(x) {
    if x.left ≠ NIL
        return Tree_Maximum(x.left)
    else {
        y = x.p
        while y != NIL and x == y.left {
            x = y
            y = y.p
        }
        return y
    }
}

```

Note que este problema é perfeitamente simétrico ao problema de encontrar o sucessor de um nó em uma árvore binária de busca. Portanto, sua complexidade também depende da altura h da árvore, podendo variar de $O(\log n)$ em árvores balanceadas até $O(n)$ no pior caso, quando a árvore se degenera para uma lista encadeada.

6.2.3 Inserção em Árvores Binárias de Busca

Para inserir um novo nó em uma árvore binária de busca não basta encontrar a primeira posição disponível. Devemos respeitar a *propriedade chave*, o que garante que cada chave possua uma posição correta dentro da estrutura. Assim, o processo de inserção consiste em localizar a posição apropriada para o novo nó e, em seguida, realizar o encadeamento lógico na árvore.

A função a seguir insere um novo nó z na árvore T . Inicialmente, precisamos encontrar a posição correta de z em T com base no valor de sua chave. Depois disso, realizamos o encadeamento lógico do novo nó na estrutura.

```

Tree_Insert(T, z) {
    x = T.root
    y = NIL
    // Encontra a posição do novo nó em T
    while x != NIL {
        y = x
        // y aponta para o pai de x
        if z.key < x.key
            x = x.left
        else
            x = x.right
    }
    // Realiza o encadeamento lógico
    z.p = y
}

```

```

if y == NIL
    // a raiz era vazia
    T.root = z
else {
    if z.key < y.key
        // adiciona a esquerda de y
        y.left = z
    else
        // adiciona a direita de y
        y.right = z
}
}

```

Assim como nos algoritmos apresentados anteriormente, a complexidade da inserção depende da altura h da árvore. No melhor caso, quando a árvore está balanceada, temos $h = O(\log n)$, o que resulta em uma inserção com custo $O(\log n)$. No pior caso, quando a árvore se degenera para uma estrutura semelhante a uma lista encadeada, temos $h = O(n)$, resultando em uma complexidade $O(n)$. A figura a seguir ilustra o processo de inserção da chave 13 em uma árvore binária de busca.

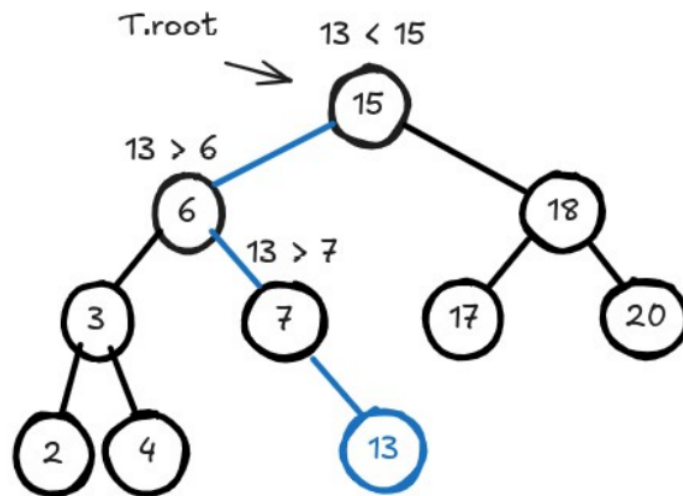


Figura 44: O processo de inserção de um nó em uma ABB baseia-se em encontrar sua posição correta na árvore a partir de um caminho top-down.

6.2.4 Remoção em Árvores Binárias de Busca

A remoção de um nó em uma árvore binária de busca é um processo mais complexo do que a inserção. Para facilitar essa operação, introduzimos uma primitiva auxiliar chamada *Transplant*. A função *Transplant*(T, u, v) substitui a subárvore enraizada no nó u pela subárvore enraizada no nó v .

A ideia central é que, quando a primitiva *Transplant* substitui uma subárvore enraizada em u por outra subárvore enraizada em v , o pai do nó u passa a se tornar o pai do nó v . Observe

que é possível também que $v = \text{NIL}$, o que significa que a subárvore original será removida sem substituição. A figura a seguir ilustra esse processo.

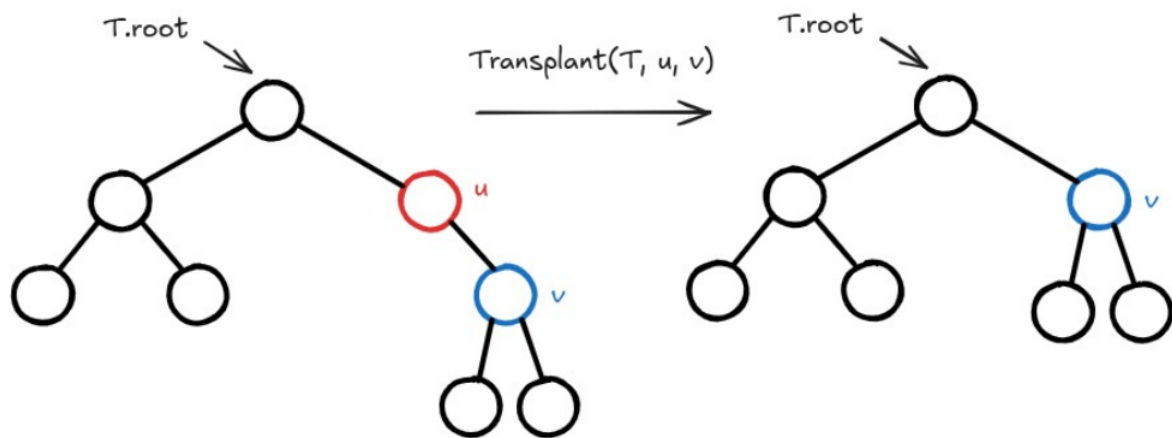


Figura 45: O processo de transplantar uma subárvore permite mover subárvores na ABB com custo $O(1)$.

Essa operação é particularmente importante porque permite mover subárvores inteiras com custo computacional $O(1)$, já que apenas algumas referências são atualizadas. Em linguagens de programação modernas com gerenciamento automático de memória, após o transplante, o nó u fica desconectado da estrutura e poderá ser desalocado automaticamente pelo coletor de lixo. Em linguagens sem esse recurso, é recomendável desalocar a memória manualmente para evitar vazamentos. O algoritmo a seguir apresenta a implementação da função `Transplant`.

```
Transplant(T, u, v) {
    if u.p == NIL
        T.root = v
    elif u == u.p.left {
        u.p.left = v
        u.left = NIL
    }
    else {
        u.p.right = v
        u.right = NIL
    }
    if v ≠ NIL {
        v.p = u.p
        u.p = NIL
    }
}
```

6.2.5 Remoção em Árvores Binárias de Busca

Seja z o nó a ser removido de uma árvore binária de busca T . A estratégia para remover z baseia-se na análise de dois casos principais.

a) z possui apenas um filho

- **a1)** Se z não possui filho à esquerda, então substituímos z por seu filho à direita utilizando a primitiva `Transplant`.

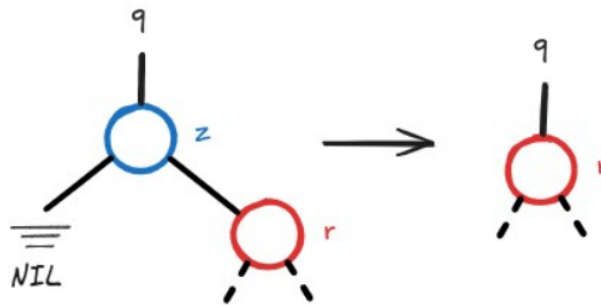


Figura 46: Se um nó possui um único filho, esse filho assume o lugar do pai.

- **a2)** Se z não possui filho à direita, então substituímos z por seu filho à esquerda utilizando a primitiva `Transplant`.

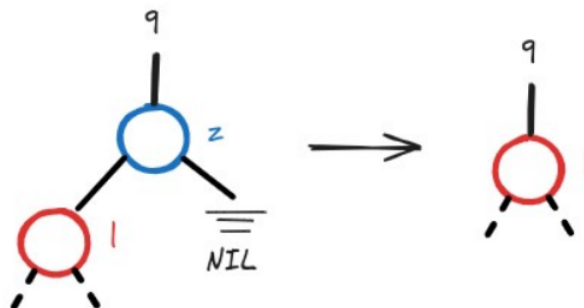


Figura 47: Se um nó possui um único filho, esse filho assume o lugar do pai.

b) z possui dois filhos

Nesse caso, devemos encontrar o sucessor de z na árvore T , que denotaremos por y . Podem ocorrer duas situações distintas:

- **b1)** y é o filho à direita de z . Nesse caso, basta transplantar y para o lugar de z e fazer com que o filho à esquerda de z se torne o filho à esquerda de y .

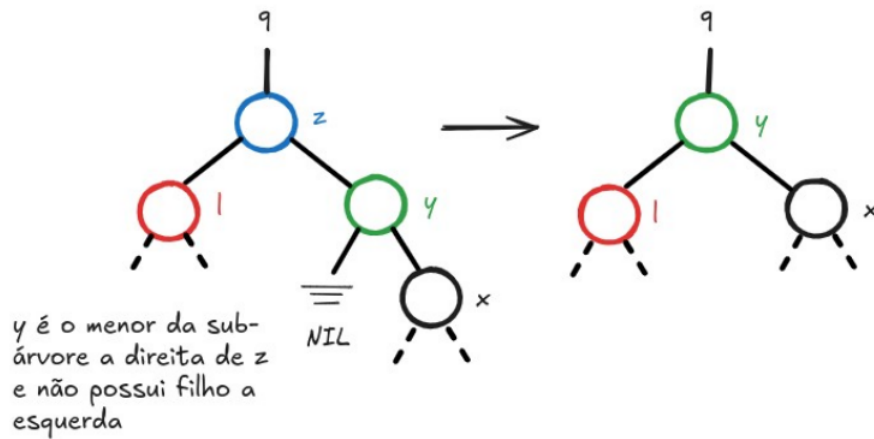


Figura 48: Se o sucessor é filho a direita do nó a ser removido, então basta substituir o nó a ser removido por ele.

- **b2)** y não é o filho à direita de z (ou seja, o sucessor está mais abaixo na subárvore direita de T). Nesse caso:
 1. Substituímos y por seu filho à direita, pois quando z for substituído por y , deve haver um nó ocupando a posição originalmente ocupada por y .

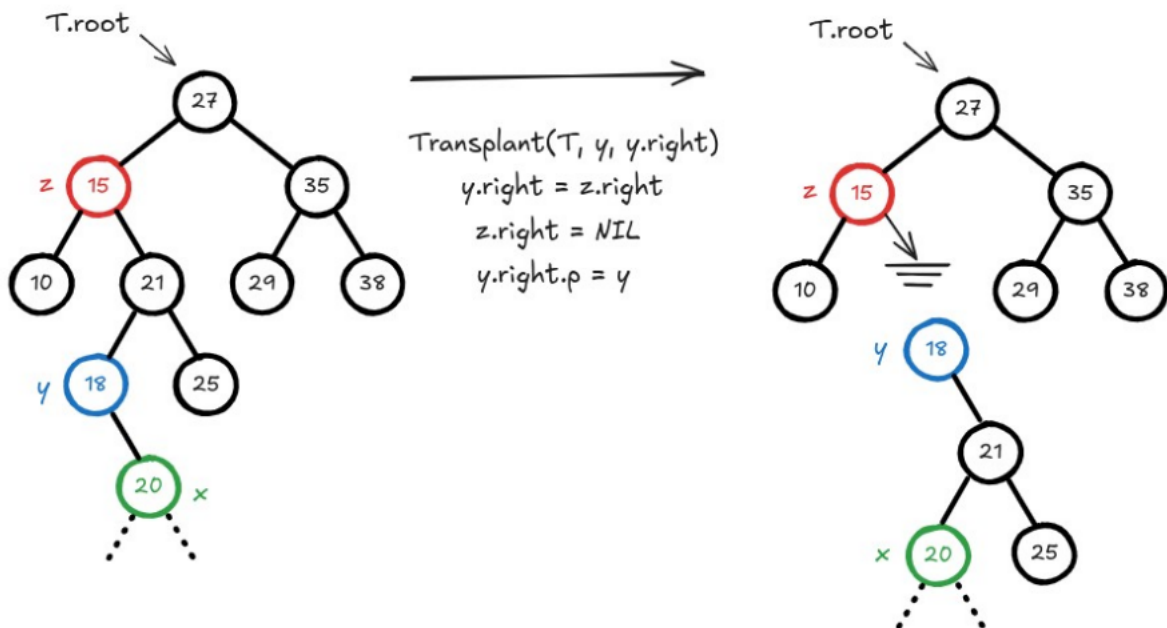


Figura 49: Se o sucessor do nó z a ser removido não é seu filho a direita, primeiro substituímos o sucessor y por seu filho a direita x . Lembre-se que y não possui filho a esquerda, pois ele é o menor elemento da subárvore a direita de z .

2. Em seguida, substituímos z por y .

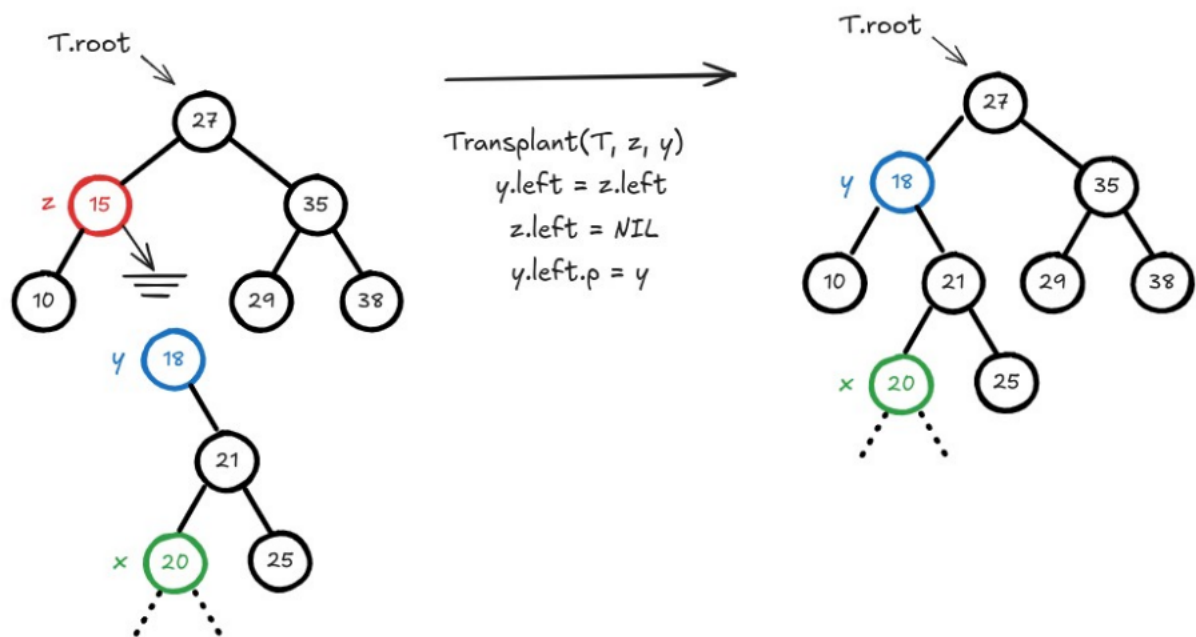


Figura 50: Se o sucessor do nó z a ser removido não é seu filho a direita, depois de substituí y por x , devemos substituir z por y .

A seguir apresentamos o algoritmo `Tree_Delete( $T, z$ )`, que remove o nó z de uma árvore binária de busca.

```
Tree_Delete( $T, z$ ) {
    if  $z.left == NIL$ 
        // substitui  $z$  pelo seu único filho a direita
        Transplant( $T, z, z.right$ )
    elseif  $z.right == NIL$ 
        // substitui  $z$  pelo seu único filho a esquerda
        Transplant( $T, z, z.left$ )
    else {
        //  $z$  possui dois filhos
        // encontra a menor chave da subárvore direita
         $y = \text{Tree\_Minimum}(z.right)$ 
        // verifica se o sucessor não é o filho direto
        if  $y \neq z.right$  {
            // sucessor está mais abaixo na árvore
            Transplant( $T, y, y.right$ )
             $y.right = z.right$ 
             $z.right = NIL$ 
             $y.right.p = y$ 
        }
        Transplant( $T, z, y$ ) // substitui  $z$  por  $y$ 
         $y.left = z.left$ 
    }
}
```

```
    z.left = NIL
    y.left.p = y
  }
}
```

6.3 Árvores AVL

Vimos anteriormente que as operações de busca, inserção e remoção de nós em árvores binárias de busca possuem complexidade $O(h)$, em que h é a altura da árvore T . Dessa forma, o desempenho dessas operações depende diretamente da altura da estrutura.

Um dos problemas das árvores binárias de busca é que seu padrão de crescimento pode se tornar bastante irregular. Por exemplo, após a criação da raiz da árvore, pode ocorrer de todos os elementos inseridos posteriormente serem maiores que os anteriores. Nesse caso, cada novo elemento é inserido sempre como filho à direita do nó previamente inserido.

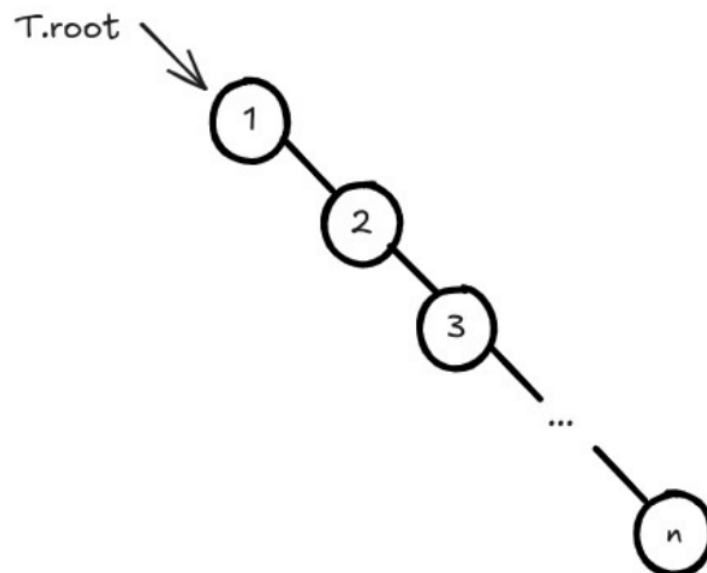


Figura 51: Se as chaves são inseridas em uma ABB na ordem crescente, a árvore resultante degenera-se para uma lista encadeada, fazendo com que a busca, inserção e remoção sejam $O(n)$.

Esse comportamento faz com que a árvore se degenere para uma estrutura semelhante a uma lista encadeada, cuja altura cresce linearmente com o número de elementos. Consequentemente, operações que deveriam ter custo próximo de $O(\log n)$ passam a apresentar custo $O(n)$.

Embora essa situação represente um caso extremo, não é incomum que alguns ramos da árvore cresçam significativamente mais do que outros. Isso torna a árvore desbalanceada e compromete a eficiência das operações de busca, inserção e remoção.

Para contornar esse problema, foram propostas as árvores AVL, em homenagem aos seus criadores, Georgy Adelson-Velsky e Evgenii Landis. Essas estruturas também são conhecidas como *árvores*

binárias de busca balanceadas. A ideia central é manter a altura das subárvores aproximadamente equilibrada após cada operação de inserção ou remoção.

Para cada nó x de uma árvore, define-se o *fator de balanço* como:

$$x.b = h_L - h_R$$

em que h_L representa a altura da subárvore à esquerda de x e h_R representa a altura da subárvore à direita.

Definição: Uma árvore T é dita uma árvore AVL (ou árvore balanceada) se

$$\forall x \in T \quad (x.b \in \{-1, 0, 1\})$$

Ou seja, não é permitido que nenhum nó apresente fator de balanço maior que 1 ou menor que -1 . Essa restrição garante que a árvore permaneça aproximadamente balanceada, mantendo sua altura proporcional a $O(\log n)$. A figura a seguir ilustra um exemplo de árvore AVL.

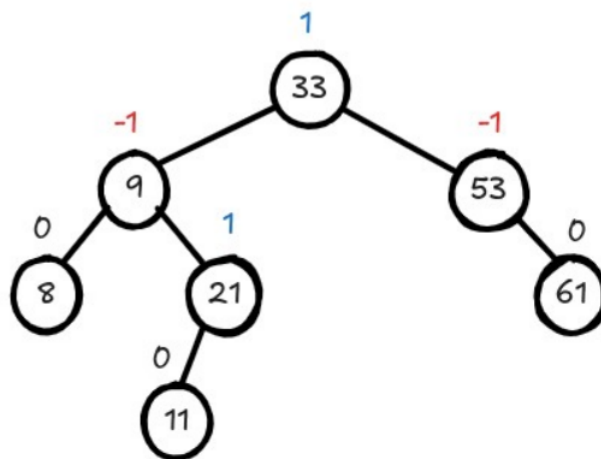


Figura 52: Exemplo de uma árvore AVL em que cada nó possui fator de balanço entre -1 e 1 .

Para otimizar o cálculo das alturas dos nós em uma árvore AVL, é comum adicionar um campo h em cada nó da estrutura para armazenar explicitamente essa informação. Dessa forma, evita-se o recálculo repetido da altura das subárvores durante as operações de inserção e remoção, o que tornaria o algoritmo menos eficiente.

Assim, cada nó de uma árvore AVL passa a possuir um campo adicional que armazena sua altura. Com isso, podemos definir uma função auxiliar responsável por retornar a altura de um nó da árvore. Por convenção, considera-se que a altura de um nó nulo (NIL) é igual a zero.

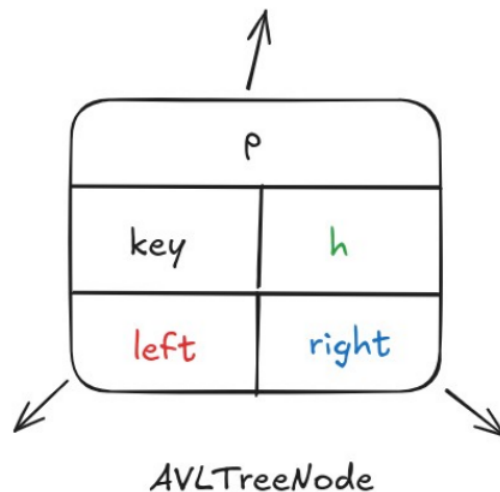


Figura 53: O campo h armazena a altura do nó em questão na árvore AVL.

```
// Função auxiliar que retorna a altura do nó z
// Altura é o número de nós no maior caminho da raiz a um nó folha
height(z) {
    if z == NIL
        return 0
    else
        return z.h
}
```

Nessa definição, a altura corresponde ao número de nós no maior caminho entre o nó considerado e um nó folha de sua subárvore.

6.4 Operações em Árvores AVL

Se uma árvore AVL torna-se desbalanceada após a remoção ou inserção de um nó, devemos executar uma operação chamada *rotação*. Há dois tipos básicos de rotação: **rotação à esquerda** e **rotação à direita**. É importante mencionar que tais operações preservam a propriedade fundamental das árvores binárias de busca, isto é, para qualquer nó x , todas as chaves da subárvore à esquerda permanecem menores que a chave de x e todas as chaves da subárvore à direita permanecem maiores.

6.4.1 Rotação à Esquerda

Seja x um nó desbalanceado tal que $x.b < -1$, conforme ilustrado na figura a seguir. Nesse caso, a altura da subárvore à direita é maior que a altura da subárvore à esquerda. Para restaurar o balanceamento da árvore, executa-se uma **rotação à esquerda** em x . Após a rotação à esquerda, a altura do nó x na árvore é decrementada, contribuindo para restaurar a propriedade de balanceamento da árvore AVL.

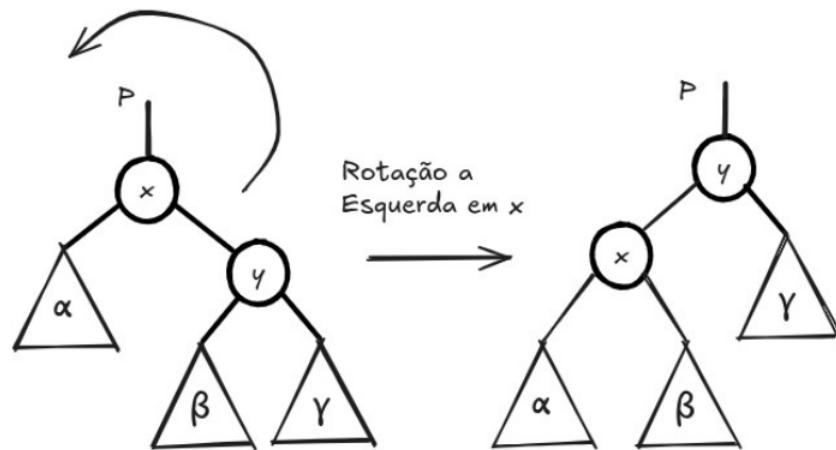


Figura 54: Ilustração da operação de rotação à esquerda em um nó de uma árvore AVL.

A função `leftRotate` a seguir apresenta os passos lógicos dessa operação:

```
leftRotate(x) {
    y = x.right
    beta = y.left
    x.right = beta
    beta.p = x
    y.left = x
    y.p = x.p
    if x == x.p.left
        x.p.left = y
    else
        x.p.right = y
    x.p = y
    // Recalcula as alturas
    x.h = 1 + max(height(x.left), height(x.right))
    y.h = 1 + max(height(y.left), height(y.right))
    return y
}
```

Em resumo, os passos principais dessa operação são:

1. Se y possui uma subárvore à esquerda β , então o filho à direita de x deve passar a apontar para β .
2. Se x é a raiz da árvore, então y passa a ser a nova raiz. Caso contrário, se x é o filho à esquerda do nó p , então y passa a ser o filho à esquerda de p . Caso contrário, y passa a ser o filho à direita de p .
3. Por fim, y torna-se o pai de x .

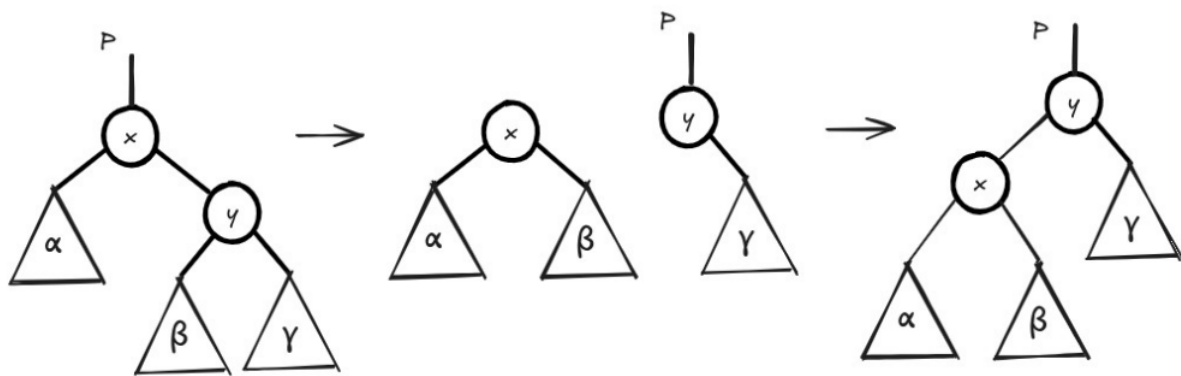


Figura 55: A relação de pai e filho entre os nós x e y é invertida após a rotação.

Observe que as chaves da subárvore α permanecem menores que x , as chaves da subárvore β continuam sendo maiores que x e menores que y , e as chaves da subárvore γ permanecem maiores que y . Dessa forma, a propriedade fundamental das árvores binárias de busca é preservada.

6.4.2 Rotação à Direita

Seja y um nó desbalanceado tal que $y.b > 1$, conforme ilustrado na figura a seguir. Nesse caso, a altura da subárvore à esquerda é maior que a altura da subárvore à direita. Para restaurar o balanceamento da árvore, executa-se uma **rotação à direita** em y . Após a rotação à direita, a altura do nó y na árvore é decrementada, contribuindo para restaurar a propriedade de balanceamento da árvore AVL.

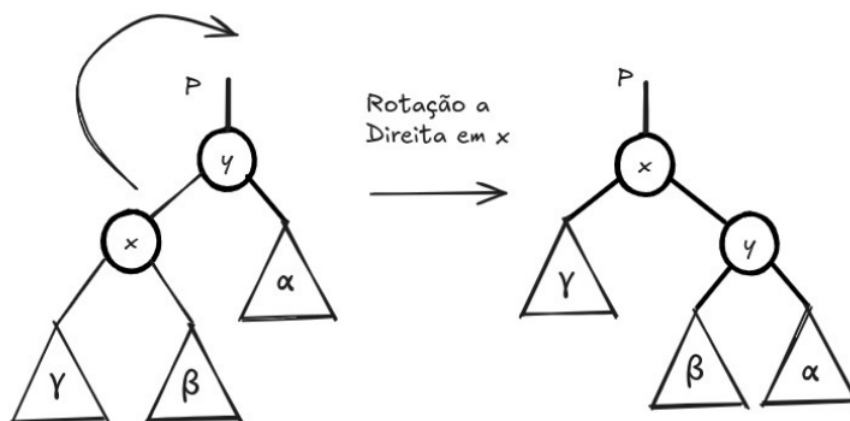


Figura 56: Ilustração da operação de rotação à direita em um nó de uma árvore AVL.

A função `rightRotate` a seguir apresenta os passos lógicos dessa operação:

```
rightRotate(y) {
    x = y.left
    beta = x.right
    y.left = beta
    beta.p = y
    x.right = y
}
```

```

x.p = y.p
if y == y.p.left
    y.p.left = x
else
    y.p.right = x
y.p = x
// Atualiza as alturas
y.h = 1 + max(height(y.left), height(y.right))
x.h = 1 + max(height(x.left), height(x.right))
return x
}

```

Em resumo, os passos principais dessa operação são:

1. Se x possui uma subárvore à direita β , então o filho à esquerda de y deve passar a apontar para β .
2. Se y é a raiz da árvore, então x passa a ser a nova raiz da árvore. Caso contrário, se y é o filho à direita do nó p , então x passa a ser o filho à direita de p . Caso contrário, x passa a ser o filho à esquerda de p .
3. Por fim, x torna-se o pai de y .

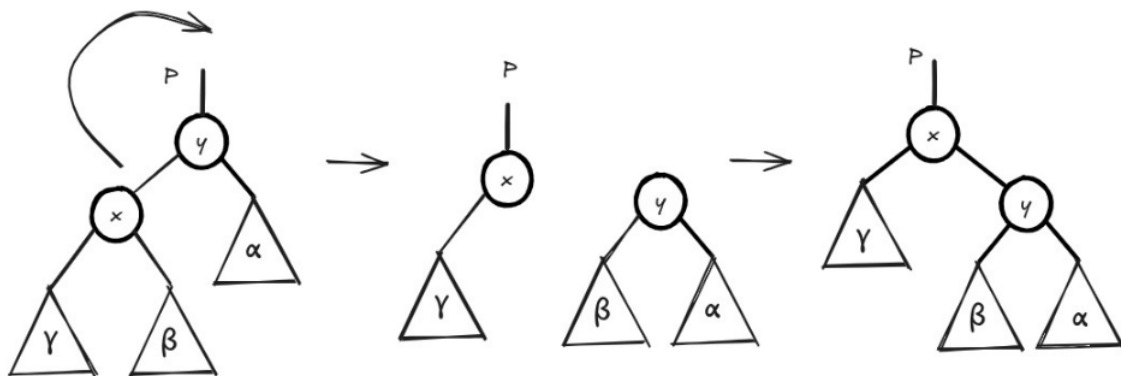


Figura 57: A relação de pai e filho entre os nós x e y é invertida após a rotação.

Observe que, após a rotação, as chaves da subárvore α permanecem menores que x , as chaves da subárvore β continuam sendo maiores que x e menores que y , e as chaves da subárvore γ permanecem maiores que y . Dessa forma, a propriedade fundamental das árvores binárias de busca é preservada.

Em alguns casos mais complexos, pode ser necessário realizar duas rotações básicas em sequência após a inserção ou remoção de um nó na árvore AVL. Essas operações são conhecidas como **rotação esquerda-direita** e **rotação direita-esquerda**, e serão definidas a seguir.

6.4.3 Rotação Esquerda-Direita

Em algumas situações, uma única rotação não é suficiente para restaurar o balanceamento da árvore AVL. Nesses casos, é necessário realizar duas rotações básicas em sequência. Um desses casos corresponde à **rotação esquerda-direita**. Esse tipo de rotação ocorre quando um nó z está desbalanceado ($z.b > 1$), mas seu filho à esquerda x possui fator de balanço negativo, indicando que a subárvore mais profunda está à direita de x . A operação consiste em duas etapas:

1. Inicialmente, realiza-se uma **rotação à esquerda** em x , o que faz com que o nó y seja promovido acima de x na árvore.

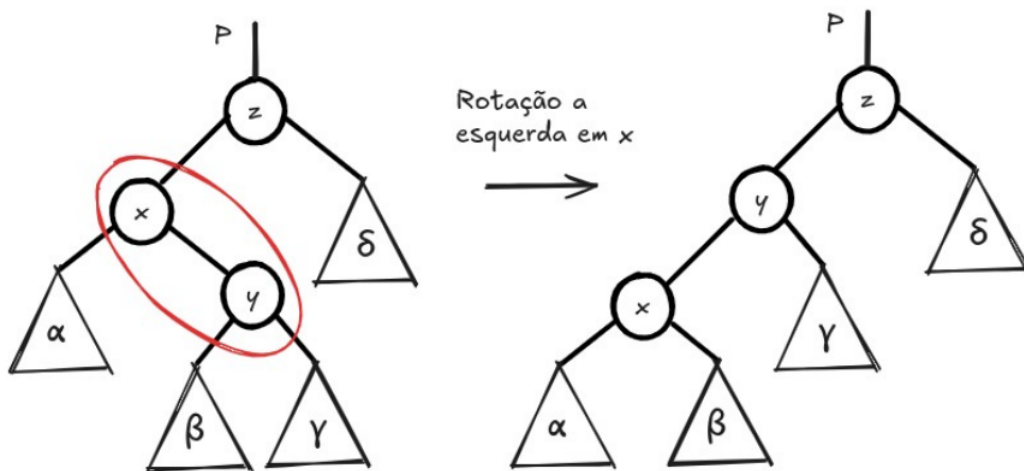


Figura 58: Primeiro, executamos uma rotação a esquerda em x .

2. Em seguida, realiza-se uma **rotação à direita** em z , promovendo o nó y acima de z .

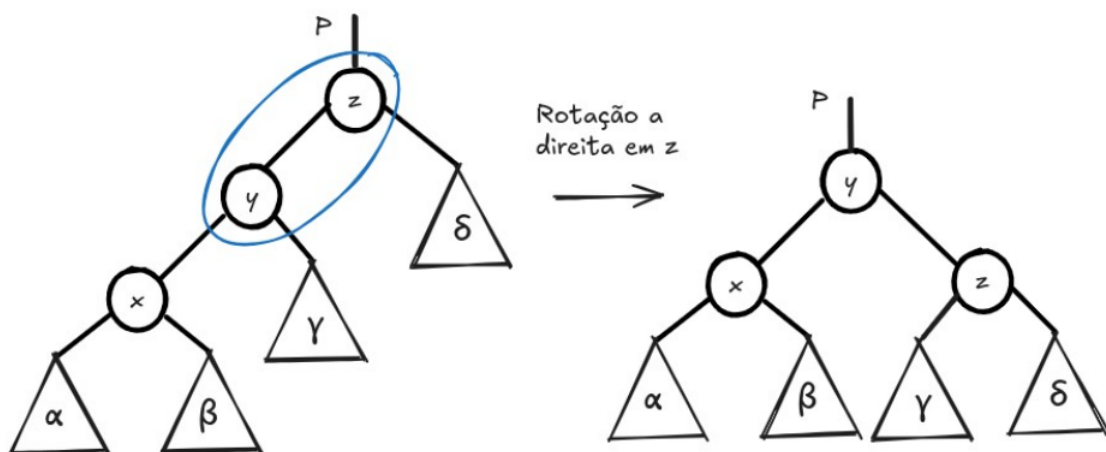


Figura 59: Depois, executamos uma rotação a direita em z .

A função a seguir ilustra os passos necessários para realizar uma rotação esquerda-direita.

```

leftRightRotate(z) {
    x = z.left
    u = leftRotate(x)
    w = rightRotate(z)
    return w
}

```

Após a execução dessas duas rotações, o nó intermediário y passa a ocupar a posição anteriormente ocupada por z , restaurando o balanceamento local da árvore e preservando a propriedade fundamental das árvores binárias de busca.

6.4.4 Rotação Direita-Esquerda

Outro caso em que duas rotações são necessárias ocorre quando um nó z está desbalanceado para a direita ($z.b < -1$), mas seu filho à direita x possui fator de balanço positivo. Isso indica que a subárvore mais profunda está à esquerda de x , exigindo uma rotação composta chamada **rotação direita-esquerda**. Essa operação também ocorre em duas etapas:

1. Primeiramente, realiza-se uma **rotação à direita** em x , promovendo o nó y acima de x .

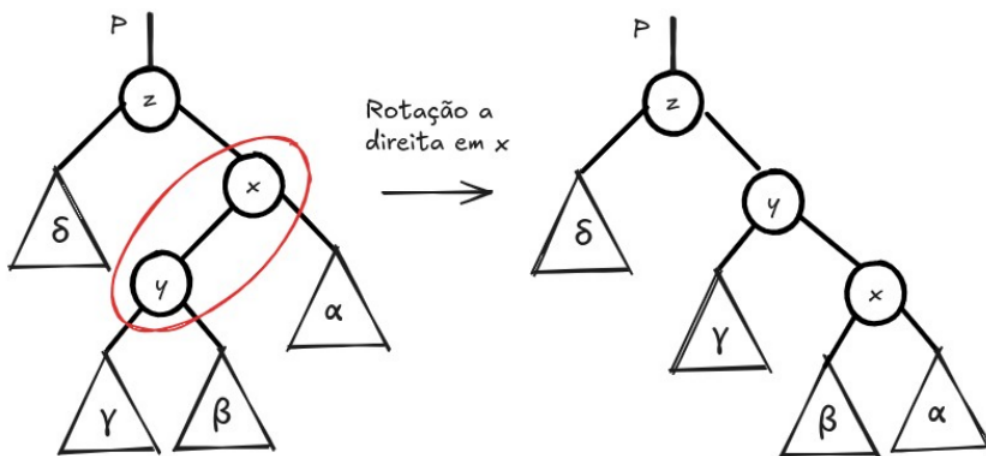


Figura 60: Primeiro, executamos uma rotação a direita em x .

2. Em seguida, realiza-se uma **rotação à esquerda** em z , promovendo o nó y acima de z .

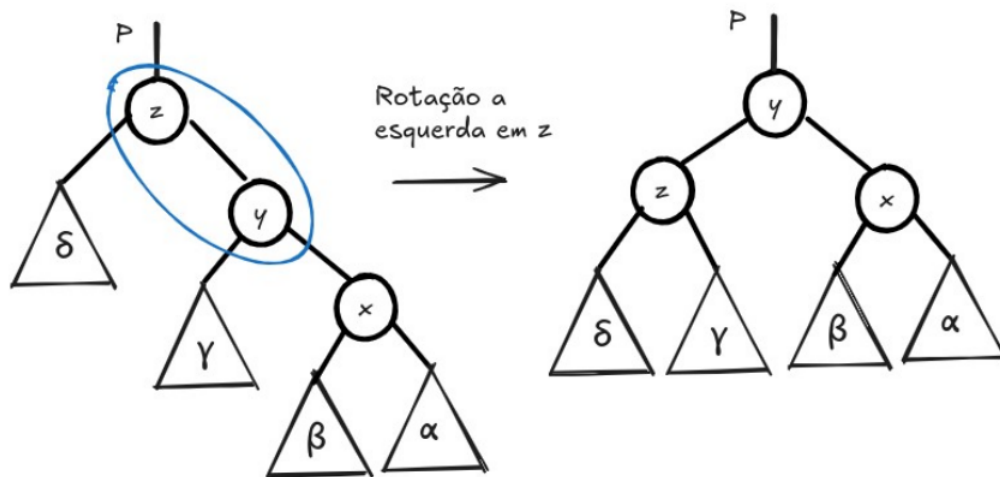


Figura 61: Depois, executamos uma rotação a esquerda em z .

A função a seguir ilustra os passos necessários para realizar essa rotação composta.

```
rightLeftRotate(z) {
    x = z.right
    u = rightRotate(x)
    w = leftRotate(z)
    return w
}
```

Após a execução dessas duas rotações, o nó intermediário y assume a posição anteriormente ocupada por z , restaurando o balanceamento da árvore e mantendo a propriedade fundamental das árvores binárias de busca.

6.5 Inserção de nós em árvores AVL

Quando um novo nó é inserido em uma árvore AVL, ele é inicialmente colocado como uma folha da árvore. Como folhas possuem altura mínima, o fator de balanço do novo nó é igual a zero. No entanto, a inserção pode alterar as alturas das subárvores de seus ancestrais, fazendo com que algum deles se torne desbalanceado. Nesses casos, torna-se necessário aplicar uma operação de rotação para restaurar a propriedade de balanceamento da árvore.

Considere, por exemplo, a inserção da chave 9 em uma árvore AVL. Embora apenas um nó seja inserido, vários outros nós ao longo do caminho até a raiz podem ter suas alturas modificadas. A estratégia consiste em identificar o primeiro nó desbalanceado no caminho percorrido da folha recém-inserida até a raiz da árvore. A rotação corretiva deve ser aplicada exatamente nesse nó.

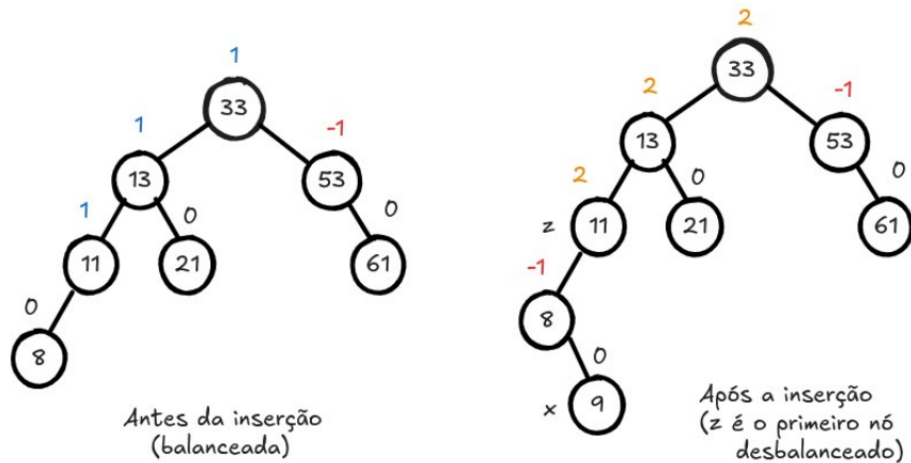


Figura 62: Em uma árvore AVL, a inserção de um novo nó pode desbalancear a árvore. Nesses casos, devemos aplicar um dos tipos de rotação existentes.

Partindo do princípio de que todo nó inserido é inicialmente uma folha (denotado por x), percorremos a árvore no sentido *bottom-up*, atualizando as alturas dos nós até encontrar o primeiro nó desbalanceado ou alcançar o valor NIL. Suponha que z seja o primeiro nó desbalanceado encontrado nesse percurso. Caso o fator de balanço seja positivo ($z.b > 1$), aplicamos a seguinte regra:

```

if z.b > 1 {
    if x.key < z.left.key
        rightRotate(z)
    else
        leftRightRotate(z)
}

```

Após a aplicação da rotação apropriada, a estrutura da árvore é reorganizada de forma a restaurar o balanceamento AVL. Dessa forma, a árvore resultante volta a satisfazer a propriedade de que o fator de balanço de todos os seus nós pertence ao conjunto $\{-1, 0, 1\}$.

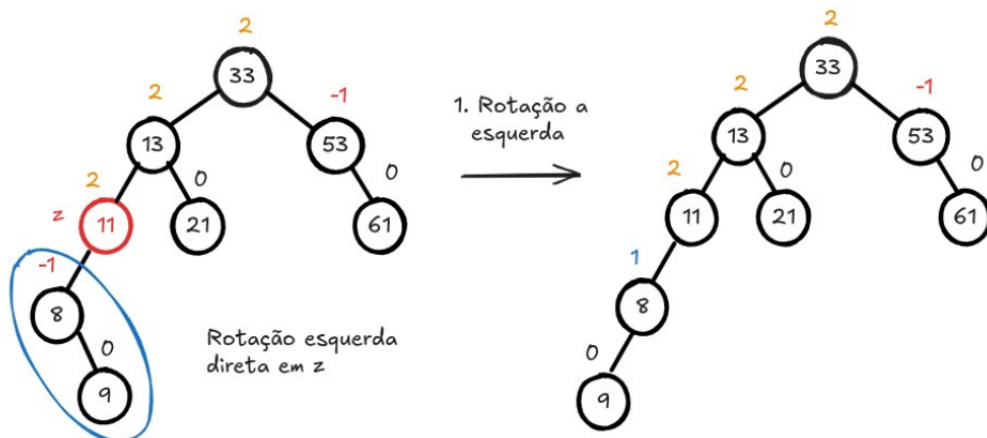


Figura 63: Primeiro, aplica-se uma rotação a esquerda.

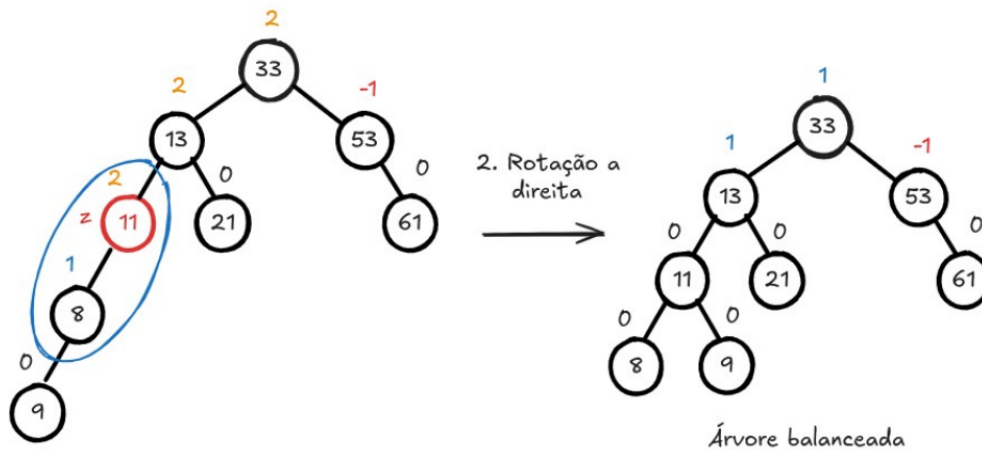


Figura 64: Em seguida, aplica-se uma rotação a direita.

Analogamente, existe o caso simétrico ao anterior, quando o fator de balanço do nó é negativo. Nesse caso, a subárvore direita possui altura maior que a subárvore esquerda, indicando que devemos aplicar rotações que movam nós da direita para restaurar o equilíbrio. A regra geral é descrita a seguir:

```
if z.b < -1 {
    if x.key > z.right.key
        leftRotate(z)
    else
        rightLeftRotate(z)
}
```

A figura a seguir ilustra um exemplo simples desse processo de rebalanceamento.

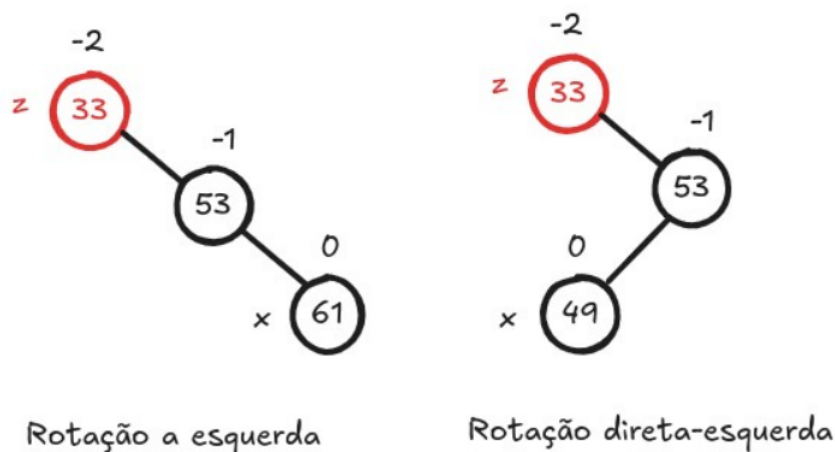


Figura 65: Exemplo de rotações em um nó com fator de balanço negativo.

De maneira geral, a operação completa de inserção com rebalanceamento em uma árvore AVL pode ser implementada por meio de um algoritmo recursivo. Nesse procedimento, durante a descida da recursão percorremos o caminho de inserção na árvore (*top-down*). Já no retorno da recursão realizamos a atualização das alturas dos nós e verificamos possíveis violações da propriedade de

balanceamento (*bottom-up*). Para isso, utilizamos uma função auxiliar `balance`, que calcula o fator de balanço de um nó.

```
# Funcao para retornar o fator de balanco de um no
balance(z) {
    if z == NIL
        return 0
    else
        return height(z.left) - height(z.right)
}
```

A seguir apresentamos o algoritmo completo de inserção em uma árvore AVL. O procedimento recebe como parâmetro o nó raiz da árvore (ou subárvore) e a chave a ser inserida.

```
// Funcao recursiva para inserir um no em uma arvore AVL
// Deve-se passar T.root como parametro em node
AVL_Tree_Insert(node, key) {
    if node == NIL {
        node = AVLTreeNode(key)
        return node
    }
    // Caminho TOP-DOWN da recursao
    if key < node.key
        node.left = AVL_Tree_Insert(node.left, key)
    elif key > node.key
        node.right = AVL_Tree_Insert(node.right, key)
    else
        // se chave ja existe em T, nao insere novamente
        return node
    // Caminho BOTTOM-UP na volta da recursao
    // Atualiza a altura do no atual
    node.h = 1 + max(height(node.left), height(node.right))
    // Calcula o fator de balanco
    b = balance(node)
    // Analise dos quatro casos possiveis
    // Caso 1: Rotacao a direita (RE)
    if b > 1 and key < node.left.key
        return rightRotate(node)
    // Caso 2: Rotacao a esquerda (RD)
    if b < -1 and key > node.right.key
        return leftRotate(node)
    // Caso 3: Rotacao esquerda-direita (RED)
    if b > 1 and key > node.left.key
```

```

    return leftRightRotate(node)
// Caso 4: Rotacao direita-esquerda (RDE)
if b < -1 and key < node.right.key
    return rightLeftRotate(node)
// Se estiver balanceado, retorna o no sem modificacoes
return node
}

```

Pode-se demonstrar que, após a inserção de um novo nó em uma árvore AVL, são necessárias no máximo duas rotações para restaurar completamente a propriedade de balanceamento da estrutura. Isso garante que a altura da árvore permaneça $O(\log n)$, assegurando eficiência nas operações de busca, inserção e remoção.

6.6 Remoção em Árvores AVL

Assim como vimos anteriormente, durante a remoção de um nó em uma árvore binária de busca, há três casos principais a serem considerados. A análise desses casos é importante para garantir que a estrutura da árvore permaneça válida após a remoção do elemento.

1. O nó a ser removido é uma folha, ou seja, não possui filhos.
2. O nó a ser removido possui apenas um único filho.
3. O nó a ser removido possui dois filhos.

Por conveniência, denominaremos por w o nó que será removido da árvore.

Caso 1: O nó removido é uma folha.

Após a remoção, devemos apenas verificar o fator de balanço dos antecessores do nó removido. Iniciando no pai de w (denotado por y), percorremos um caminho ascendente na árvore até encontrar o primeiro nó cujo fator de balanço seja maior que 1 ou menor que -1 , ou até atingirmos NIL . Esse primeiro nó desbalanceado será denominado por z .

Caso 2: O nó removido possui apenas um único filho.

Nesse caso, o nó removido é substituído diretamente pelo seu único filho. Após essa substituição, devemos verificar o fator de balanço dos antecessores. Iniciando no nó que ocupou a posição do nó removido, denotado por y , percorremos novamente um caminho ascendente até encontrar o primeiro nó desbalanceado z .

Caso 3: O nó removido possui dois filhos.

Nesse cenário, devemos inicialmente encontrar o sucessor do nó na árvore. Como o sucessor é sempre o menor elemento da subárvore à direita, ele terá no máximo um único filho. Assim, podemos

substituir o nó removido por seu sucessor. Após essa substituição, é necessário verificar o fator de balanço dos antecessores.

Caminho Bottom-Up: Iniciando no nó que substituiu o removido, denotado por y , percorremos um caminho ascendente na árvore até encontrar o primeiro nó desbalanceado z . A regra de rebalanceamento consiste em aplicar uma rotação no ancestral desbalanceado mais próximo. Para o caso em que o fator de balanço é positivo, temos a seguinte regra:

```
if z.b > 1 {
  if z.left.b >= 0
    rightRotate(z)
  else
    leftRightRotate(z)
}
```

As figuras a seguir ilustram um exemplo didático do processo de remoção em árvores AVL. Nesse exemplo, observa-se que é necessário realizar uma rotação à direita para restaurar o balanceamento da árvore, conforme ilustrado na figura correspondente.

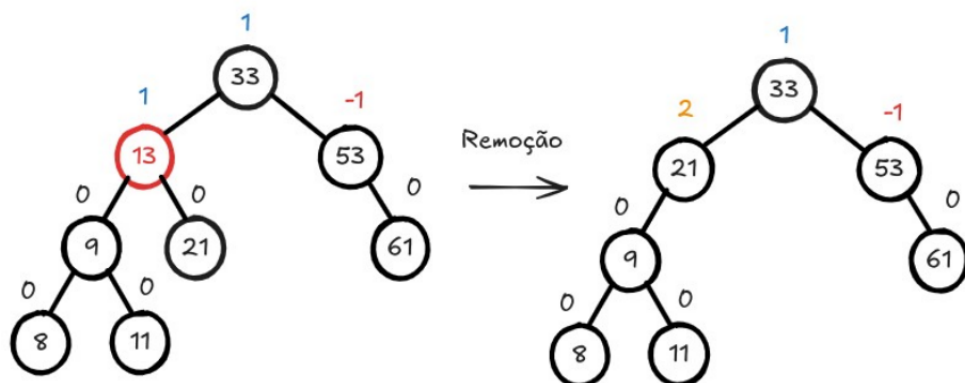


Figura 66: Após a remoção de um nó w em uma árvore AVL, o fator de balanço dos nós no caminho bottom-up de w até a raiz podem ser afetados e a árvore se torna desbalanceada.

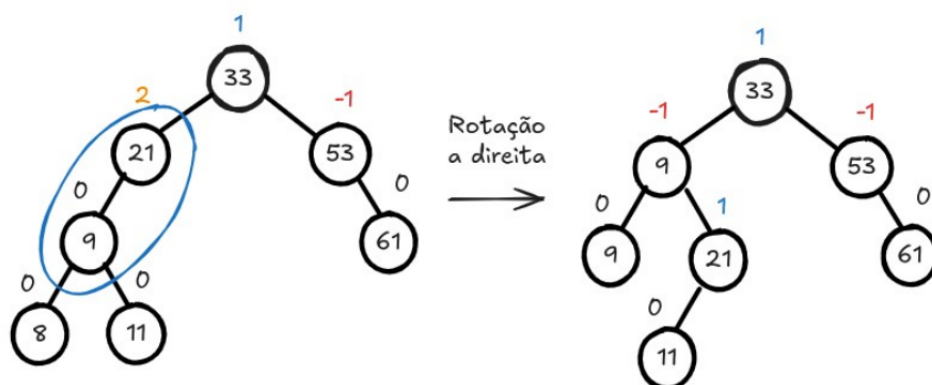


Figura 67: Devemos identificar o primeiro nó desbalanceado no caminho bottom-up e realizar as rotações necessárias para restaurar o balanceamento da árvore.

Para o caso simétrico, quando o fator de balanço do nó é negativo, temos uma regra análoga para restaurar o balanceamento da árvore:

```
if z.b < -1 {  
    if z.right.b <= 0  
        leftRotate(z)  
    else  
        rightLeftRotate(z)  
}
```

A função a seguir mostra como podemos realizar a remoção de um nó em uma árvore AVL de maneira recursiva. O algoritmo segue duas fases principais: um caminho *top-down* para localizar o nó a ser removido e um caminho *bottom-up* para restaurar o balanceamento da árvore durante o retorno da recursão.

```
AVL_Tree_Delete(root, key) {  
    // Caminho TOP-DOWN para encontrar o nó  
    if root == NIL  
        return root  
    if key < root.key  
        root.left = AVL_Tree_Delete(root.left, key)  
    elif key > root.key  
        root.right = AVL_Tree_Delete(root.right, key)  
    else {  
        // Se entrou aqui, encontrou o nó a ser removido  
        if root.left == NIL or root.right == NIL {  
            if root.left != NIL  
                temp = root.left  
            else  
                temp = root.right  
            // Se o nó removido é uma folha  
            if temp == NIL  
                root = NIL  
            else  
                // Nó possui um único filho  
                root = temp  
        }  
        else {  
            // Nó possui dois filhos  
            // Encontrar o sucessor (menor da subárvore à direita)  
            temp = Tree_Minimum(root.right)  
            // Copia a chave do sucessor  
            root.key = temp.key
```

```

        // Remove o sucessor
        root.right = AVL_Tree_Delete(root.right, temp.key)
    }
}
// Condição de parada da recursão
if root == NIL
    return root
// Caminho BOTTOM-UP para rebalanceamento
// Atualiza a altura do nó
root.h = 1 + max(height(root.left), height(root.right))
// Calcula o fator de balanço
b = balance(root)
// Análise dos quatro casos possíveis
// Caso 1: Rotação à direita
if b > 1 and balance(root.left) >= 0
    return rightRotate(root)
// Caso 2: Rotação esquerda-direita
if b > 1 and balance(root.left) < 0
    return leftRightRotate(root)
// Caso 3: Rotação à esquerda
if b < -1 and balance(root.right) <= 0
    return leftRotate(root)
// Caso 4: Rotação direita-esquerda
if b < -1 and balance(root.right) > 0
    return rightLeftRotate(root)
// Se estiver balanceado, retorna o nó sem modificações
return root
}

```

6.7 Complexidade das Operações em Árvores AVL

Uma das principais vantagens das árvores AVL é que elas garantem que a altura da árvore permaneça limitada por $O(\log n)$, onde n é o número de nós da estrutura. Como as operações fundamentais, busca, inserção e remoção, percorrem um caminho da raiz até um nó folha ou até a posição onde a chave deve estar localizada, o custo dessas operações é proporcional à altura da árvore.

Em uma árvore binária de busca comum, a altura pode crescer até n no pior caso, especialmente quando as chaves são inseridas em ordem crescente ou decrescente. Nesse cenário, a árvore se degenera para uma lista encadeada e as operações passam a ter complexidade $O(n)$. Entretanto, em árvores AVL o fator de balanço de cada nó é mantido no intervalo $\{-1, 0, 1\}$, garantindo que a árvore permaneça aproximadamente balanceada após inserções e remoções. Como consequência, a altura da

árvore cresce de forma logarítmica em relação ao número de elementos armazenados.

Uma forma intuitiva de compreender essa eficiência é por meio de uma analogia com o algoritmo de busca binária. Na busca binária, a cada iteração eliminamos aproximadamente metade das chaves do conjunto de busca. Assim, se começarmos com n elementos, após uma comparação restam aproximadamente $n/2$ candidatos, depois $n/4$, depois $n/8$ e assim sucessivamente, até restar apenas um elemento. O número de etapas necessárias para esse processo é proporcional a $\log_2 n$.

Algo conceitualmente semelhante ocorre em uma árvore AVL. Quando realizamos uma busca, cada comparação entre a chave procurada e a chave armazenada em um nó determina qual subárvore deve ser explorada. Em outras palavras, a cada passo descartamos aproximadamente metade da estrutura restante, seguindo apenas para a subárvore esquerda ou direita. Como a árvore permanece balanceada, sua altura é proporcional a $\log n$, garantindo que o número máximo de nós visitados em qualquer operação seja também $O(\log n)$.

Assim, podemos concluir que as operações fundamentais em árvores AVL apresentam as seguintes complexidades assintóticas:

$$\text{Busca} = O(\log n), \quad \text{Inserção} = O(\log n), \quad \text{Remoção} = O(\log n).$$

Essas garantias tornam as árvores AVL estruturas extremamente eficientes para manter conjuntos dinâmicos de dados ordenados, permitindo acesso rápido mesmo em cenários com grande volume de elementos.

6.8 Considerações Finais

Neste capítulo foram apresentadas as árvores binárias, uma importante classe de estruturas de dados dinâmicas que organizam informações de forma hierárquica. Inicialmente discutimos as árvores binárias simples e os principais algoritmos de percurso, como *inorder*, *preorder* e *postorder*. Esses percursos permitem visitar todos os nós da estrutura de maneira sistemática e são fundamentais para diversas aplicações, como impressão ordenada de chaves, avaliação de expressões e manipulação de estruturas hierárquicas.

Em seguida, introduzimos as árvores binárias de busca (Binary Search Trees – BST), nas quais existe uma propriedade de ordenação que determina a posição de cada chave dentro da árvore. Essa propriedade permite que operações fundamentais como busca, inserção e remoção sejam realizadas de forma eficiente, com custo proporcional à altura da árvore. No entanto, vimos que o desempenho dessas operações depende fortemente do formato da árvore. Em situações desfavoráveis, como quando as chaves são inseridas em ordem crescente ou decrescente, a árvore pode se degenerar para uma estrutura semelhante a uma lista encadeada, fazendo com que a complexidade das operações aumente para $O(n)$.

Para contornar esse problema, apresentamos as árvores AVL, uma forma de árvore binária de busca auto-balanceada. Nessas estruturas, a diferença de altura entre as subárvores esquerda e direita de cada nó é limitada, garantindo que a árvore permaneça aproximadamente balanceada após opera-

ções de inserção e remoção. Mostramos que o balanceamento é mantido por meio de operações de rotação, que reorganizam localmente a estrutura da árvore sem violar a propriedade de ordenação das chaves.

Como consequência desse mecanismo de balanceamento, a altura de uma árvore AVL cresce de forma logarítmica em relação ao número de elementos armazenados. Isso garante que as operações fundamentais de busca, inserção e remoção possuam complexidade $O(\log n)$, tornando essa estrutura adequada para aplicações que exigem acesso eficiente a grandes conjuntos de dados dinâmicos. Dessa forma, as árvores AVL representam uma solução elegante e eficiente para manter estruturas hierárquicas ordenadas e balanceadas.

7 Heaps Binários e Filas de Prioridade

Em diversos problemas computacionais é necessário manipular conjuntos de elementos nos quais cada item possui uma prioridade associada. Nesses cenários, não basta apenas inserir ou remover elementos de uma estrutura de dados: é necessário também acessar rapidamente o elemento de maior ou menor prioridade. Estruturas projetadas especificamente para esse tipo de operação são conhecidas como *filas de prioridade* (*priority queues*). Em uma fila de prioridade, cada elemento possui uma chave que determina sua prioridade relativa no conjunto, permitindo que operações como inserção, consulta ao elemento de maior prioridade e remoção desse elemento sejam realizadas de forma eficiente.

Uma das implementações mais utilizadas de filas de prioridade é baseada em uma estrutura chamada *heap binário*. Um heap binário é uma árvore binária quase completa que satisfaz uma propriedade de ordenação específica entre pais e filhos. Dependendo da aplicação, essa propriedade pode assumir duas formas: no *max-heap*, a chave armazenada em cada nó é maior ou igual às chaves de seus filhos, enquanto no *min-heap* ocorre o inverso, ou seja, cada nó possui chave menor ou igual às chaves de seus descendentes diretos. Essa organização garante que o elemento de maior (ou menor) prioridade esteja sempre localizado na raiz da árvore.

Na prática, heaps binários são frequentemente implementados utilizando vetores, explorando o fato de que árvores binárias quase completas podem ser representadas de forma compacta em memória sem a necessidade de ponteiros explícitos. Essa representação permite acessar eficientemente o pai e os filhos de um nó por meio de simples operações aritméticas sobre índices do vetor. Como veremos nas seções seguintes, essa estrutura possibilita a implementação eficiente de operações fundamentais como *insert*, *extract-max*, *extract-min* e *heapify*, que constituem a base para diversos algoritmos importantes, incluindo o *Heapsort* e aplicações envolvendo escalonamento e seleção de elementos com prioridade.

7.1 Árvores Binárias Completas

Para entender a lógica das estruturas de dados conhecidas como *heaps*, devemos primeiramente definir alguns conceitos importantes relacionados a árvores binárias. **Definição:** Uma árvore binária é dita *cheia* (*full*) se todos os nós intermediários (isto é, nós que não são folhas) possuem exatamente dois filhos.

Definição: Uma árvore binária é dita *perfeita* (*perfect*) se ela é cheia e, além disso, todos os nós folha encontram-se no mesmo nível da árvore.

A ideia por trás de um heap é que podemos manipular uma árvore binária completa (ABC) contendo n nós como se fosse um vetor de n elementos. Isso significa que não é necessário utilizar referências ou ponteiros para alocação dinâmica de memória. Em vez disso, dado um nó da árvore, podemos encontrar facilmente seus dois filhos e seu pai utilizando simples operações aritméticas sobre os índices do vetor.

Mas afinal, o que é uma árvore binária completa? Dizemos que uma árvore binária é *completa* quando todos os seus níveis estão completamente preenchidos, exceto possivelmente o último nível, que deve ser preenchido da esquerda para a direita.

Uma árvore binária completa difere de uma árvore binária cheia em dois aspectos principais:

1. Todo nó folha que não possui irmão é necessariamente um filho à esquerda.
2. A última folha pode não possuir um irmão à direita.

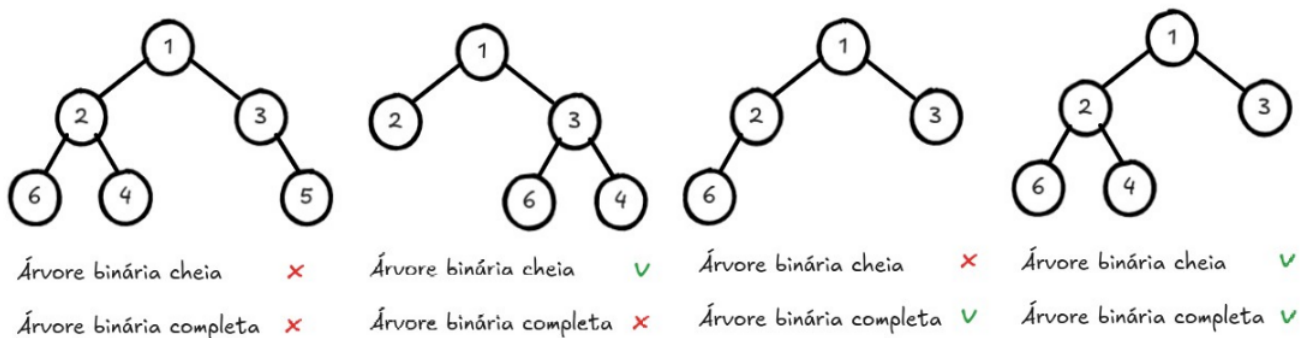


Figura 68: Diferenças entre árvores binárias completas e árvores binárias cheias.

Veremos adiante que uma das maneiras mais eficientes de implementar uma fila de prioridades é através da criação de um *heap binário*, cuja estrutura baseia-se justamente na relação existente entre árvores binárias completas e vetores.

Note que, para uma árvore binária ser cheia, não é necessário que todas as folhas estejam no mesmo nível; basta que todo nó interno possua exatamente dois filhos.

Em uma árvore binária completa com n nós, se $T[i]$ denota o i -ésimo elemento (nó) do vetor que representa a árvore, então podemos determinar as posições dos filhos e do pai da seguinte forma:

- Filho à esquerda: $T[2i + 1]$, se $2i + 1 \leq n$
- Filho à direita: $T[2i + 2]$, se $2i + 2 \leq n$
- Pai: $T[\lfloor (i - 1)/2 \rfloor]$

onde $\lfloor \cdot \rfloor$ denota a função piso. Um exemplo ilustrativo dessa correspondência entre uma árvore binária completa e sua representação em vetor pode ser encontrado na figura a seguir.

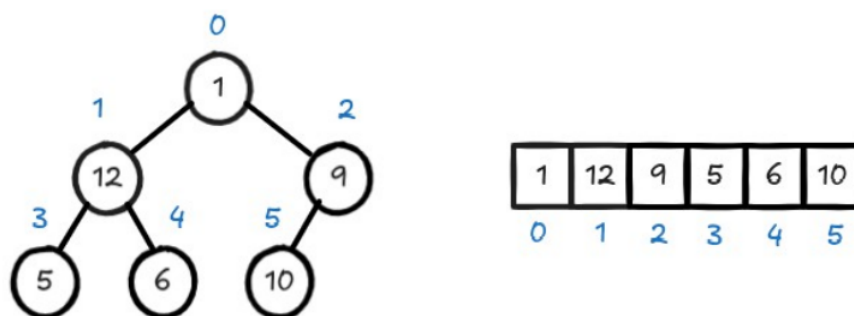


Figura 69: Exemplo de uma árvore binária completa representada como um array.

7.2 A Estrutura de Dados Heap

Um *heap* é uma estrutura de dados especializada baseada em árvores binárias completas. Como discutido anteriormente, uma árvore binária completa possui todos os níveis totalmente preenchidos, exceto possivelmente o último, que é preenchido da esquerda para a direita. Essa propriedade permite que a árvore seja representada eficientemente em um vetor, dispensando o uso de ponteiros ou referências explícitas entre os nós.

Uma árvore binária completa é considerada um *heap* quando satisfaz uma propriedade de ordenação conhecida como *propriedade do heap*. Essa propriedade estabelece uma relação entre cada nó da árvore e seus filhos. Dependendo da natureza dessa relação, temos dois tipos principais de heaps:

1. **Min-heap:** todo nó i possui uma chave menor ou igual às chaves de seus filhos. Formalmente, se j é filho de i , então

$$T[i] \leq T[j].$$

2. **Max-heap:** todo nó i possui uma chave maior ou igual às chaves de seus filhos. Formalmente, se j é filho de i , então

$$T[i] \geq T[j].$$

É importante observar que a propriedade do heap impõe apenas uma relação entre cada nó e seus filhos imediatos, não garantindo necessariamente uma ordenação global entre todos os elementos da estrutura. Em outras palavras, um heap não é uma árvore de busca binária, pois não existe uma restrição direta entre os elementos das subárvores esquerda e direita.

Uma consequência direta da propriedade do heap é que o elemento armazenado na raiz da árvore possui uma característica especial. Em um *min-heap*, a raiz contém o menor elemento do conjunto, enquanto em um *max-heap*, a raiz contém o maior elemento. Essa característica torna os heaps particularmente úteis para a implementação eficiente de *filas de prioridade*, nas quais é necessário acessar rapidamente o elemento de maior ou menor prioridade. A figura a seguir ilustra exemplos de um *max-heap* e de um *min-heap*.

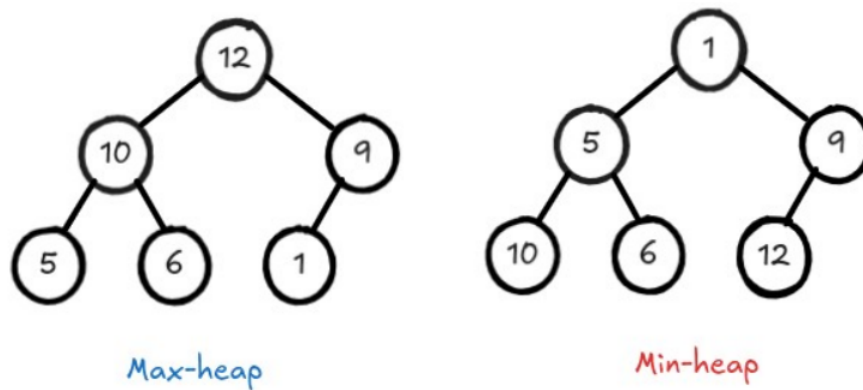


Figura 70: Exemplo de estruturas do tipo min-heap e max-heap.

7.3 Filas de Prioridade com min-heaps

Podemos utilizar um *min-heap* ou *max-heap* para implementar uma fila de prioridades de maneira eficiente. Em uma fila de prioridades, cada elemento possui um valor associado que representa sua prioridade. Dependendo da aplicação, pode ser necessário acessar rapidamente o elemento de maior prioridade (max-priority queue) ou o elemento de menor prioridade (min-priority queue).

A ideia central da implementação baseada em heaps é manter sempre válida a propriedade do *heap* após cada operação realizada na estrutura. Como os heaps são armazenados em um vetor que representa uma árvore binária completa, as operações de inserção e remoção exigem procedimentos auxiliares para restaurar essa propriedade. Em resumo, a metodologia adotada segue a lógica a seguir:

- Após cada `enqueue()` (inserção no final da estrutura), devemos aplicar a operação auxiliar `sift_up()` para mover o elemento recém-inserido em direção à raiz do heap até que a propriedade do heap seja restaurada.
- Após cada `dequeue()` (remoção da raiz do heap), trazemos a última chave do vetor para a posição da raiz e aplicamos a operação auxiliar `sift_down()` para deslocar esse elemento para baixo na árvore até sua posição correta.

Essas operações garantem que a propriedade estrutural da árvore binária completa e a propriedade de ordenação do heap sejam preservadas após cada modificação da estrutura.

Sendo assim, podemos definir um TAD (*Tipo Abstrato de Dados*) para uma fila de prioridade baseada em heaps da seguinte forma:

```

TAD Node
    int key
    int p        // prioridade

TAD PriorityQueue
    Node array[0..n-1] data
    int tail

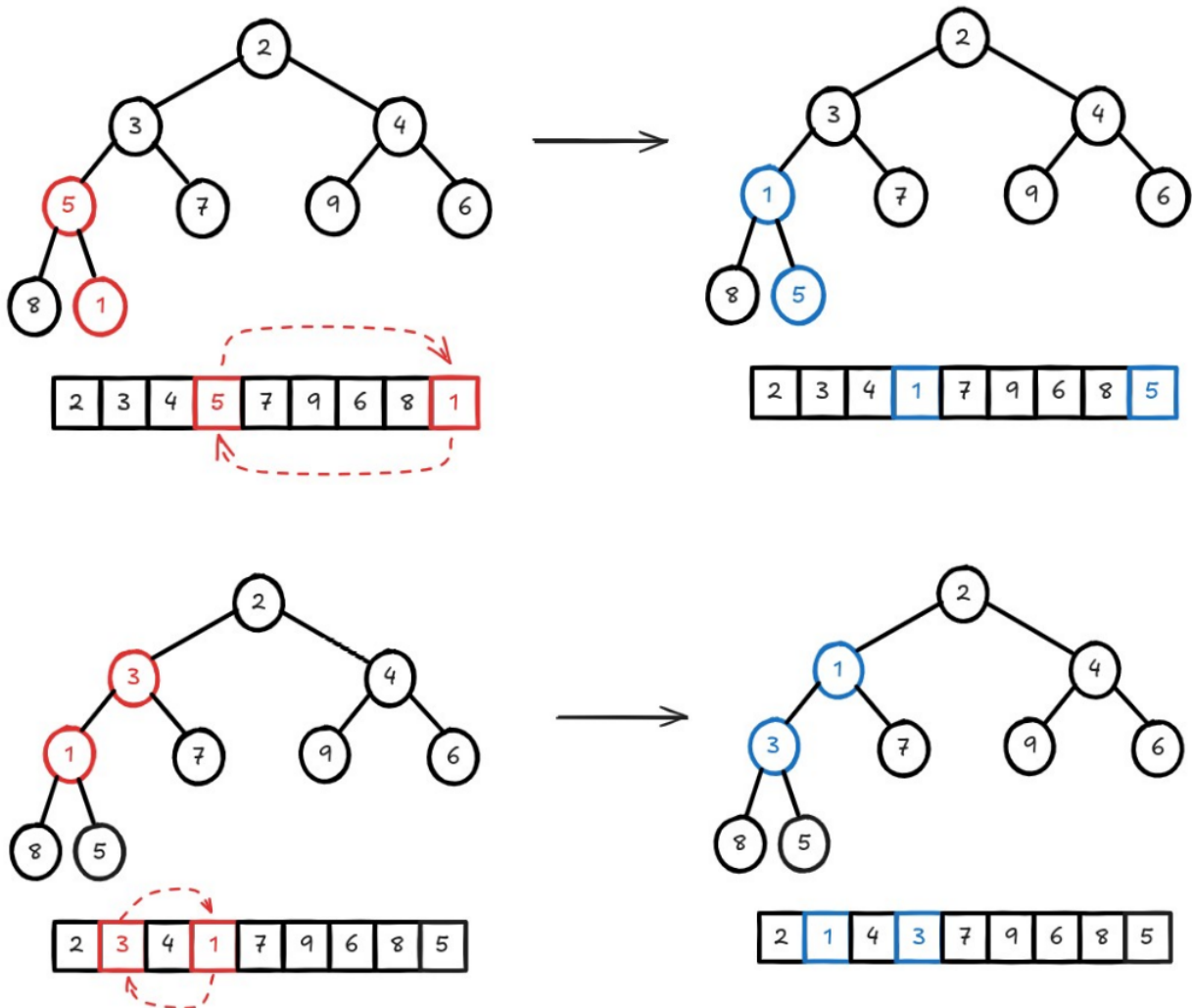
```

```

init(PQ) {
    PQ.tail = -1
}

// incrementa tail e depois insere
// Adiciona chave com prioridade p no final e realiza um sift_up
enqueue(PQ, key, p) {
    if PQ.tail == n-1
        error('overflow')
    else {
        PQ.tail += 1
        PQ.data[PQ.tail].key = key
        PQ.data[PQ.tail].p = p
        sift_up(PQ)      // sobe última chave no heap
    }
}

```



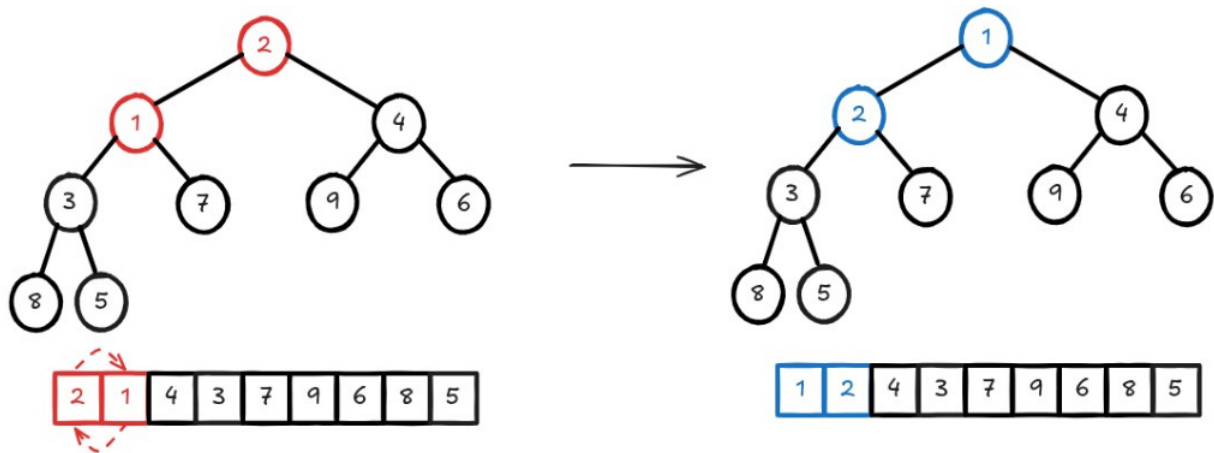


Figura 71: Ilustração gráfica do processo `sift_up` após o `enqueue` em um `min-heap`.

Note que após todo o processo de `enqueue()` seguido da operação `sift_up()`, a propriedade do *min-heap* é restaurada. Ou seja, o elemento de menor prioridade permanece na raiz da estrutura.

A operação `sift_up()` realiza um processo *bottom-up*, no qual o elemento recém-inserido é comparado com seu pai. Caso a prioridade do filho seja menor que a do pai (em um *min-heap*), os dois nós são trocados. Esse procedimento é repetido até que o elemento alcance sua posição correta na árvore.

```
// Sobe o menor para raiz do heap (bottom-up)
sift_up(PQ) {
    son = PQ.tail
    son_p = PQ.data[son].p
    parent = (son-1)//2
    parent_p = PQ.data[parent].p
    while son > 0 and son_p < parent_p {
        swap(PQ.data[son], PQ.data[parent])
        son = parent
        parent = (son-1)//2
    }
}
```

A operação de remoção da fila de prioridades é realizada pela função `dequeue()`. Como estamos utilizando um *min-heap*, o elemento removido será sempre aquele com menor valor de prioridade, que se encontra na raiz da estrutura.

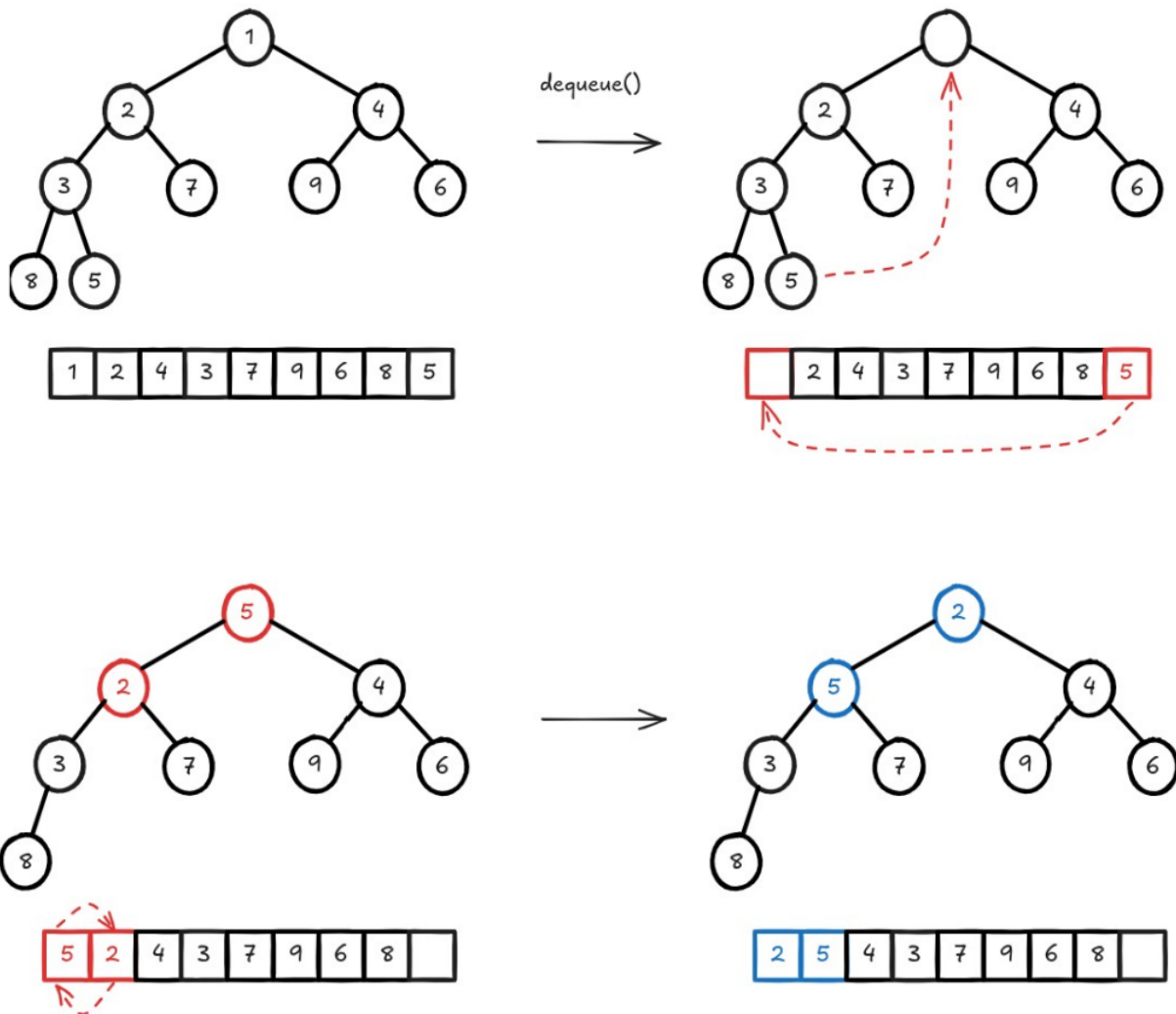
Para manter a propriedade estrutural da árvore binária completa, trocamos o último elemento do vetor com a raiz e reduzimos o tamanho do heap. Em seguida, aplicamos a operação `sift_down()` para reposicionar o novo elemento da raiz na posição correta.

```
// Remove chave de menor p (menor p = maior prioridade)
dequeue(PQ) {
    if PQ.tail == -1
```

```

error('underflow')
else {
    key = PQ.data[0]           // Remove a raiz
    swap(PQ.data[0], PQ.data[PQ.tail]) // Troca último com raiz
    PQ.tail -= 1
    shift_down(PQ, 0)         // Desce nova raiz no heap
}
return key
}

```



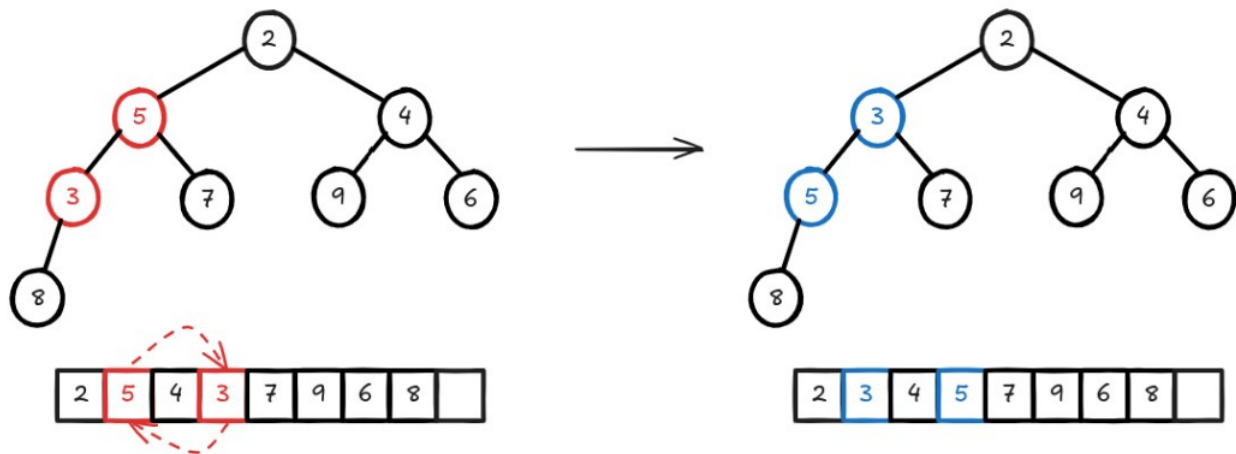


Figura 72: Ilustração gráfica do processo `sift_down` após o `dequeue` em um `min-heap`.

Note que após todo o processo de `dequeue()` seguido da operação `sift_down()`, a propriedade do *min-heap* é restaurada novamente.

A função `sift_down()` realiza o procedimento inverso do `sift_up()`. Nesse caso, partimos da raiz e comparamos o nó atual com seus dois filhos, trocando-o com o filho de menor prioridade sempre que necessário. Esse processo é executado de forma *top-down* até que o elemento alcance sua posição correta no heap.

```
// Desce nova raiz no heap (top-down)
sift_down(PQ, i) {
    parent = i                // i = 0 -> raiz da árvore
    parent_p = PQ.data[parent].p
    left = 2*parent + 1
    right = 2*parent + 2
    smallest = parent
    if left < PQ.tail and parent_p > PQ.data[left].p
        smallest = left
    if right < PQ.tail and PQ.data[smallest].p > PQ.data[right].p
        smallest = right
    if smallest != parent {
        swap(PQ.data[parent], PQ.data[smallest])
        sift_down(PQ, smallest)
    }
}
```

Graças à estrutura de árvore binária completa, tanto as operações de inserção (`enqueue`) quanto de remoção (`dequeue`) possuem complexidade $O(\log n)$, pois no pior caso o elemento percorre a altura do heap durante as operações de subida ou descida na árvore.

7.4 Análise da Complexidade

Iremos considerar inicialmente a análise no pior caso. Primeiramente, note que em uma árvore binária completa o número de nós em cada nível dobra em relação ao nível anterior. Essa propriedade decorre do fato de que cada nó pode possuir até dois filhos.

Considere, por exemplo, uma árvore binária completa cuja altura é $d = 2$ (adotando a raiz no nível 0). O número máximo de elementos armazenados na árvore é dado por

$$n = 2^0 + 2^1 + 2^2 = 1 + 2 + 4 = 7 = 2^{d+1} - 1.$$

Ao passarmos para uma árvore de altura $d = 3$, o número máximo de elementos armazenados torna-se

$$n = 2^0 + 2^1 + 2^2 + 2^3 = 1 + 2 + 4 + 8 = 15 = 2^4 - 1.$$

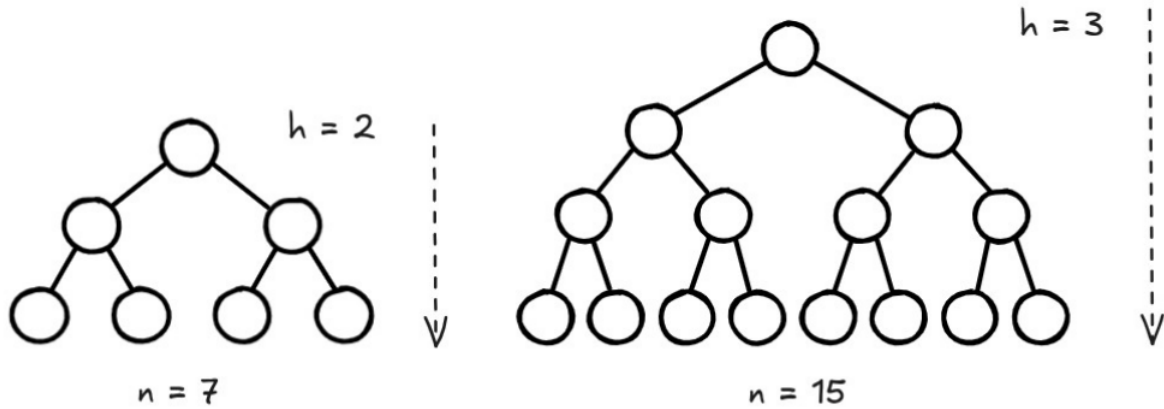


Figura 73: O número total de nós que podem ser armazenados em uma árvore binária completa com k níveis é $2^{k+1} - 1$.

De forma geral, para uma árvore binária completa de altura genérica k , o número máximo de elementos que podem ser armazenados é dado pela soma geométrica

$$n = 2^0 + 2^1 + 2^2 + \dots + 2^k = \sum_{i=0}^k 2^i = 2^{k+1} - 1.$$

A partir dessa expressão, podemos obter a altura da árvore em função do número de elementos n . Manipulando a equação, temos

$$2^{k+1} = n + 1$$

$$\log_2(n + 1) = k + 1$$

$$k = \log_2(n + 1) - 1.$$

Portanto, a altura de uma árvore binária completa cresce de forma logarítmica em relação ao número de elementos armazenados, ou seja,

$$k = O(\log n).$$

No pior caso, as operações auxiliares `sift_up()` e `sift_down()` percorrem toda a altura da árvore. Isso significa que o elemento pode subir ou descer desde uma folha até a raiz (ou vice-versa), realizando no máximo k trocas entre nós pai e filho.

Como $k = O(\log n)$, concluímos que as operações fundamentais do heap possuem complexidade logarítmica. Em particular, as funções

`sift_up()`, `sift_down()`, `enqueue()`, `dequeue()`

possuem complexidade de tempo no pior caso igual a

$$O(\log n).$$

Essa propriedade torna os heaps uma estrutura de dados extremamente eficiente para a implementação de filas de prioridade.

7.5 Considerações Finais

Neste capítulo foram apresentados os conceitos fundamentais relacionados aos *heaps binários* e à implementação eficiente de *filas de prioridade*. Inicialmente discutimos as propriedades estruturais das árvores binárias cheias, perfeitas e completas, destacando que a estrutura de um heap baseia-se especificamente em árvores binárias completas. Essa característica é particularmente importante, pois permite representar a árvore de maneira compacta utilizando apenas um vetor, dispensando o uso de ponteiros e simplificando significativamente a manipulação da estrutura.

Em seguida, foram introduzidos os conceitos de *min-heap* e *max-heap*, definidos a partir da relação de ordem entre cada nó e seus filhos. Essa propriedade garante que o elemento de maior ou menor prioridade esteja sempre armazenado na raiz da árvore, permitindo acesso imediato a esse elemento. Tal característica torna os heaps extremamente adequados para a implementação de filas de prioridade, nas quais elementos são processados de acordo com um critério de prioridade em vez da ordem de chegada.

Também foram apresentadas as operações fundamentais que mantêm a propriedade do heap após inserções e remoções. Em particular, discutimos os procedimentos `sift_up` e `sift_down`, responsáveis por restaurar a estrutura do heap após operações de inserção (`enqueue`) e remoção (`dequeue`), respectivamente. Essas operações realizam ajustes locais na árvore, deslocando elementos ao longo

de um caminho vertical até que a propriedade de ordenação seja novamente satisfeita.

Por fim, analisamos a complexidade dessas operações, mostrando que, devido à altura logarítmica das árvores binárias completas, tanto a inserção quanto a remoção em um heap possuem complexidade $O(\log n)$ no pior caso. Essa eficiência faz com que heaps sejam amplamente utilizados em diversos algoritmos e aplicações importantes, como escalonamento de tarefas, simulações de eventos discretos, algoritmos de busca em grafos (por exemplo, o algoritmo de Dijkstra) e procedimentos de ordenação, como o *Heapsort*. Dessa forma, heaps constituem uma das estruturas de dados mais relevantes para o desenvolvimento de algoritmos eficientes.

Epílogo

Ao longo desta apostila percorremos alguns dos conceitos fundamentais da área de Algoritmos e Estruturas de Dados. Iniciamos com os princípios básicos de análise de algoritmos e notação assintótica, avançamos pelo estudo de estruturas lineares como vetores, listas, pilhas e filas, e posteriormente exploramos estruturas hierárquicas e técnicas algorítmicas mais elaboradas, como árvores, heaps e algoritmos de ordenação e busca.

O objetivo principal deste material foi apresentar não apenas implementações práticas, mas também a forma de pensar necessária para projetar algoritmos eficientes. Em Ciência da Computação, a escolha adequada de uma estrutura de dados pode transformar um problema aparentemente complexo em uma solução simples e elegante. Da mesma forma, compreender o custo computacional de um algoritmo permite avaliar sua viabilidade em cenários reais, nos quais tempo de execução e uso de memória são recursos limitados.

Mais do que memorizar implementações específicas, espera-se que o estudante tenha desenvolvido a capacidade de analisar problemas computacionais, identificar padrões e selecionar ferramentas adequadas para resolvê-los. As estruturas de dados e técnicas apresentadas aqui constituem a base sobre a qual se apoiam diversas áreas da computação moderna, como bancos de dados, inteligência artificial, computação gráfica, sistemas distribuídos e aprendizado de máquina.

Naturalmente, o conteúdo abordado neste material representa apenas o primeiro passo em uma jornada mais ampla. Disciplinas posteriores aprofundarão temas como grafos, programação dinâmica, algoritmos avançados e análise mais sofisticada de complexidade. No entanto, os fundamentos apresentados aqui permanecerão presentes em praticamente qualquer sistema computacional que venha a ser desenvolvido ou estudado.

Por fim, vale lembrar que algoritmos são, em essência, uma forma estruturada de organizar ideias. Aprender algoritmos significa aprender a raciocinar de forma lógica, precisa e sistemática, habilidades que transcendem a programação e se estendem a inúmeros problemas do mundo real.

Espera-se que esta apostila sirva não apenas como material de apoio à disciplina, mas também como referência inicial para estudos futuros. Que os conceitos aqui apresentados incentivem a curiosidade, o pensamento crítico e o interesse contínuo pelo fascinante campo dos algoritmos e das estruturas de dados.

Informações sobre o autor

Alexandre L. M. Levada é Bacharel em Ciência da Computação pela Universidade Estadual Paulista (UNESP) em 2002. Concluiu seu Mestrado em Ciência da Computação pela Universidade Federal de São Carlos (UFSCar) em 2006 e seu Doutorado em Física Computacional pela Universidade de São Paulo (USP) em 2010. No mesmo ano, ingressou no Departamento de Computação da Universidade Federal de São Carlos. De 2010 a 2024, foi professor adjunto no Departamento de Computação, onde lecionou diversas disciplinas de graduação e pós-graduação como lógica matemática, teoria dos grafos, matemática discreta, processamento digital de imagens, análise de sinais e sistemas, reconhecimento de padrões, visão computacional, algoritmos e estruturas de dados 1 e 2, projeto e análise de algoritmos, programação científica e otimização matemática. Desde 2024 é professor associado. Seus principais temas de pesquisa incluem filtragem de ruído em sinais e imagens, reconhecimento de padrões e aprendizado de máquina.

“Se você tiver um pão e eu tiver um euro, e eu uso o meu euro para comprar o seu pão, no final da troca eu terei o pão e você o euro. Parece um equilíbrio perfeito, não? A tem um euro, B tem um pão; depois, A tem o pão e B o euro. É uma transação justa, mas meramente material. Agora, imagine que você tem um soneto de Verlaine ou conhece o teorema de Pitágoras, e eu não tenho nada. Se me ensinar, no final dessa troca, eu terei aprendido o soneto e o teorema, mas você ainda os terá também. Nesse caso, não há apenas equilíbrio, mas crescimento. No primeiro, trocamos mercadorias. No segundo, compartilhamos conhecimento. E enquanto a mercadoria se consome, a cultura se expande infinitamente.”

Michel Serres, filósofo francês.

Referências

- W. CELES, R. CERQUEIRA, J. RANGEL. Introdução a Estrutura de Dados, 2a edição, Elsevier, 2016.
- CORMEN, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. Introduction to Algorithms. 4 a edição, The MIT Press, 2022.
- ROUGHGARDEN, T. Algorithms Illuminated: The Omnibus Edition. SoundLikeYourself, 2022.
- KARUMANCHI, N. Data Structures and Algorithms Made Easy, CareerMonk Publications, 2010.
- LAMBERT, K. A. Fundamentals of Python: Data Structures, 2a edição, Cengage Learning, 2019.
- ALTHOFF, C. The Self-Taught Computer Scientist: The Beginners Guide to Data Structures & Algorithms, Wiley, 2022.
- BHARGAVA, A. Y. Grokking Algorithms: A Illustrated Guide for Programmers and Other Curious People, Manning Publications, 2016.
- KLEINBERG, J.; TARDOS, E. Algorithmic Design, Addison Wesley, 2005.
- U. AGARWAL, Algorithms Design and Analysis, Dhanpat Rai Publications, 2012.
- R. SEDGEWICK, K. WAYNE. Algorithms, 4th. ed., Addison-Wesley, 2011.
- LEVITIN, A. Introduction to the Design and Analysis of Algorithms, 3a edição, Pearson, 2012.
- SKIENA, S. S. The Algorithm Design Manual, 2a edição, Springer, 2008.
- S. DASGUPTA, C.H. PAPADIMITRIOU, U.V. VAZIRANI. Algorithms, McGraw-Hill, 2007.
- N. ZIVIANI. Projetos de algoritmos: com implementações em Pascal e C. 3. ed. rev. e ampl. São Paulo: Cengage Learning, 2012.
- R. SEDGEWICK. Algorithms in C, Parts 1-4: fundamentals, data structures, sorting, searching. 3rd. ed., Addison-Wesley, 1998.
- FELICE, M. C. S. Material e aulas das disciplinas Algoritmos e Estruturas de Dados I e II. Disponíveis em: <http://www.aloc.ufscar.br/felice/>
- Geeks for Geeks – A computer science portal for geeks. <https://www.geeksforgeeks.org>